

Relationale Datenbanksysteme

Eine formal fundierte Einführung mit einem
tiefenorientierten Entwurfsparadigma

Themengebiet: Datenbanktheorie, Schemaentwurf, SQL, PL/SQL, Anwendungsentwicklung
Schwerpunkt: Tiefenorientierter Entwurf – Tiefenpfade vs. sternförmige Strukturen
Autor: Stephan Epp
Datum: 11. Juni 2026

Inhaltsverzeichnis

Vorwort	5
1 Was sind Datenbanksysteme?	9
1.1 Unterschiede von DBMS zu klassischen Dateisystemen	9
1.2 Forderungen an ein DBMS	10
1.3 Abstraktionsebenen im DBMS	10
1.4 Aufbau und Leitidee dieses Buches	11
2 Formale Modelle für Datenbanken	12
2.1 Das Relationenmodell	12
2.2 Das Entity-Relationship-Modell	12
2.3 Vom Entity-Relationship- zum Relationenmodell	13
2.4 Wichtige Operationen auf Relationen (Tabellen)	14
2.5 Zusatzforderungen für relationale DBMS	14
3 Datenbankoperationen mit SQL	16
3.1 Anforderungen an einen Datenbank-Server	16
3.2 SQL-Standards	16
3.3 Sprachstruktur von SQL	16
3.4 Abrufen von Informationen mit SELECT	17
3.5 Das Sprachmittel DISTINCT ON: Gruppenrepräsentanten entlang des Tiefenpfads	17
3.5.1 Motivation: Standard-DISTINCT versus Gruppenrepräsentanten .	17
3.5.2 Das Standard-SQL-Vorgehen: Fensterfunktion mit ROW_NUMBER()	18
3.5.3 DISTINCT ON: Direkte deklarative Lösung in PostgreSQL	19
3.5.4 Effizienzvergleich und Bezug zur Tiefenpfadarchitektur	21
3.5.5 Weitere Anwendungsfälle entlang des Tiefenpfads	22
3.6 Datenmodifikationen	23
3.7 Darstellung von Datenbankinhalten	23
4 Datenbank-Entwurf	24
4.1 Anomalien in Datenbanken	24
4.2 Normalformen von Relationen	24
4.3 Dekomposition	24
4.4 Minimalcover und der Synthesealgorithmus zur Schemaminimierung . .	25
4.4.1 Hüllenbildung und Äquivalenz von FD-Mengen	25
4.4.2 Definition des Minimalcovers	26

4.4.3	Algorithmus zur Berechnung eines Minimalcovers	26
4.4.4	Laufzeitanalyse	27
4.4.5	Der Synthesealgorithmus zur Schemaminimierung	28
4.4.6	Korrektheit des Synthesealgorithmus	29
5	Datenbankbetrieb	33
5.1	Das Transaktionskonzept in Datenbanksystemen	33
5.2	Mehrbenutzerbetrieb	33
5.3	Datenschutz und Zugriffsrechte	34
5.4	Benutzung von Systemtabellen	34
6	PL/SQL	35
6.1	Blockstruktur	35
6.2	Ein Beispiel	35
6.3	Kontrollstrukturen	35
6.4	Das Cursor-Konzept	35
6.5	Fehlerbehandlung	36
6.6	Prozeduren und Funktionen	36
6.7	Trigger	36
7	Datenbankzugriff in höheren Programmiersprachen	38
7.1	Das Problem der Impedanzdiskrepanz	38
7.2	Embedded SQL	39
7.2.1	Deklarationsabschnitt und Hostvariablen	39
7.2.2	Einzeltupel-Zugriff	39
7.2.3	Mengenorientierter Zugriff: Cursor in Embedded SQL	39
7.2.4	Transaktionssteuerung und Fehlerbehandlung	40
7.3	Cursor- und API-basierter Zugriff: ODBC und JDBC	41
7.3.1	Grundstruktur einer JDBC-Anwendung	41
7.3.2	Vorbereitete Anweisungen (Prepared Statements)	42
7.3.3	Fehlerbehandlung und Ressourcenverwaltung	42
7.4	Die DBI-Schnittstelle für Perl	43
7.4.1	Fehlerbehandlung mit <code>RaiseError</code> und <code>eval</code>	43
7.4.2	Transaktionsklammern und <code>AutoCommit</code>	43
7.5	Portabilität der Zugriffsverfahren	44
7.6	Objektrelationales Mapping (ORM) und das Tiefenpfadparadigma	44
7.6.1	Lazy Loading entlang des Tiefenpfads	44
7.6.2	Kaskadierende Operationen im ORM	45
7.7	Modularisierung von Anwendungscode entlang des Fremdschlüsselgraphen	45
7.7.1	Das Repository-Pattern	45
7.7.2	Service-Schicht für Mehrfach-Pfad-Operationen	46
7.8	Zusammenfassung	47
8	WWW-Integration von Datenbanken	48
8.1	Kommandozeilen- und graphische Benutzeroberflächen	48
8.2	Benutzeroberflächen im World Wide Web	48
8.3	Architektur einer WWW-Oberfläche mit CGI-Skripten	48

8.4	Sammeln von Eingaben mit Web-Formularen	48
8.5	Auswertung von Formulareingaben mit CGI-Skripten	49
8.6	Erzeugen von Formularen mit CGI-Skripten	49
8.7	Interaktion mit Web-Formularen	49
8.8	Effizienz von CGI-Skripten	49
8.9	CGI-Skripten im Mehrbenutzerbetrieb	49
8.10	Implementierung umfangreicher Anwendungen	50
8.11	Formale Grundlagen	50
8.11.1	Relationales Modell	50
8.11.2	Referenzgraph	51
8.11.3	Komplexitätsmaße	51
8.12	Topologien: Tiefe vs. Breite	51
8.12.1	Formale Charakterisierung	51
8.12.2	Semantische Motivation	52
8.13	Normalisierungstheorie und Schematopologie	52
8.13.1	Normalformen und ihre Anforderungen	52
8.13.2	Transitivität und Schematopologie	53
8.13.3	Mehrwertige Abhängigkeiten und 4NF	53
8.14	Anomalievermeidung	53
8.14.1	Klassifikation von Anomalien	53
8.14.2	Quantitative Analyse der Redundanz	54
8.15	Abfragekomplexität und Optimierbarkeit	54
8.15.1	Join-Komplexität	54
8.15.2	Selektivität und Filtereffizienz	55
8.15.3	Schreibkomplexität und Konsistenzerhaltung	55
8.16	Wartbarkeit und Erweiterbarkeit	56
8.16.1	Schemaevolution	56
8.16.2	Zugriffsrechteverwaltung	56
8.16.3	Testbarkeit und Datenqualität	56
8.17	Konsolidierter formaler Nachweis	56
8.17.1	Gütekriterien	56
8.18	Graphentheoretische Fundierung: DFS, BFS und die Asymmetrie der Suchalgorithmen	57
8.18.1	Tiefensuche und der DFS-Wald	58
8.18.2	Breitensuche und das Fehlen eines BFS-Walds	58
8.18.3	Die fundamentale Asymmetrie von DFS und BFS	59
8.18.4	Datenbanktheoretische Konsequenz der Asymmetrie	60
8.19	Durchgängiges Praxisbeispiel	61
8.19.1	Domäne: Auftragsabwicklung	61
8.19.2	Sternschema (anti-pattern)	61
8.19.3	Tiefenpfadschema (BCNF-konform)	61
8.19.4	Vergleichstabelle	63

9	Fazit und Ausblick	64
9.1	Zusammenfassung der Ergebnisse	64
9.2	Ausblick	65
9.2.1	Automatisierte Schemaanalyse und Refactoring-Werkzeuge . . .	65
9.2.2	Erweiterung auf NoSQL- und NewSQL-Systeme	66
9.2.3	Quantitative empirische Validierung	66
9.2.4	Integration in den Datenbankoptimierer	67
9.2.5	Lehre und curriculare Integration	67
9.2.6	Herstellerspezifische Erweiterungen am Beispiel DISTINCT ON: Spannungsfeld zu Portabilität und Tiefenpfadarchitektur	68
9.2.7	Temporale Datenbanken und die zeitliche Dimension des Tiefenpfads	69
9.2.8	Verteilte Transaktionen, Konsistenzmodelle und der Tiefenpfad in Microservice-Architekturen	70
9.2.9	Sicherheits- und Datenschutzaspekte: Verschlüsselung, Mandan- tenfähigkeit und Anonymisierung entlang des Tiefenpfads	72
9.2.10	Formale Verifikation von Schemaeigenschaften mit Beweisassistenten	73
9.2.11	Maschinelles Lernen auf Tiefenpfadschemata	74
9.2.12	Hybride Graph-relationale Systeme und alternative Anfragesprachen	76
9.2.13	Ein Forschungsprogramm: Vom Gütekriterium zur Theorie der Schema-Topologie	77
9.2.14	Schlussbemerkung	77
A	Syntaxübersicht und ergänzende Materialien	79
A.1	Übersicht zentraler SQL-Sprachelemente	79
A.2	Glossar der formalen Symbole	80
A.3	Hinweise zu Software-Bezugsquellen	80

Vorwort

Das vorliegende Buch entwickelt eine vollständige, formal fundierte Einführung in relationale Datenbanksysteme, deren Gliederung sich an dem klassischen Lehrwerk *Relationale Datenbanksysteme – Eine praktische Einführung* von Kleinschmidt und Rank orientiert. Im Unterschied zu rein praxisorientierten Darstellungen wird in diesem Buch ein durchgängiges formales Entwurfparadigma – der *tiefenorientierte Datenbankentwurf* – in den Mittelpunkt gestellt: Es wird formal nachgewiesen, dass die strukturelle Orientierung von Fremdschlüsselrelationen in die *Tiefe* (lineare bzw. baumartige Ketten) gegenüber der *Breite* (sternförmige Hub-Anordnungen) hinsichtlich Normalisierungsgrad, funktionaler Abhängigkeitsstruktur, Anomalievermeidung, Abfragekomplexität und Wartbarkeit überlegen ist. Die Argumentation stützt sich auf die klassische relationale Algebra nach Codd, die Normalformtheorie bis zur 4NF sowie graphentheoretische Resultate zu Tiefen- und Breitensuche. Alle Kapitel des klassischen Kanons – formale Modelle, SQL, Datenbankentwurf, Datenbankbetrieb, PL/SQL, Programmiersprachenanbindung und WWW-Integration – werden konsequent unter diesem Paradigma neu beleuchtet und mit vollständigen formalen Beweisen versehen.

Motivation und Entstehungshintergrund

Die Idee zu diesem Buch entstand aus einer wiederkehrenden Beobachtung: In zahlreichen produktiven Datenbankschemata – gleich ob in kleinen Anwendungen oder in gewachsenen Unternehmenssystemen – findet sich immer wieder dasselbe strukturelle Muster: eine zentrale „Hub“-Tabelle, an die eine Vielzahl weiterer Tabellen über Fremdschlüssel angebunden ist, ohne dass die natürliche, oft tief geschachtelte Abhängigkeitsstruktur der modellierten Domäne abgebildet wird. Dieses Muster – in diesem Buch als *Sternschema* formalisiert (Definition 8.7) – wird in der Praxis häufig aus pragmatischen Gründen gewählt: Es erscheint auf den ersten Blick einfacher zu verstehen, da „alles an einer Stelle“ zu finden ist, und es vermeidet scheinbar die „Umständlichkeit“ mehrstufiger Verbundoperationen.

Die in diesem Buch entwickelte Theorie zeigt, dass dieser scheinbare Vorteil bei genauerer Betrachtung trügerisch ist. Sobald die modellierte Domäne – wie praktisch immer – transitive Abhängigkeiten aufweist (ein **Auftrag** gehört zu einem **Kunden**, der einer **Region** zugeordnet ist, die zu einem **Land** gehört), erzwingt bereits die klassische Normalisierungstheorie nach Codd eine *tiefe*, kettenförmige Zerlegung (Theorem 8.1). Ein Sternschema, das diese Tiefe künstlich in die Breite presst, verletzt zwangsläufig die dritte Normalform und erkaufte sich die scheinbare Einfachheit mit Redundanz, Anomalieanfälligkeit und eingeschränkter Wartbarkeit.

Dieses Buch versteht sich daher nicht als bloße Wiederholung etablierter Lehrbuchinhalte, sondern als ein *durchgängiges Argument*: Jedes Kapitel des klassischen Kanons – von den formalen Grundlagen über SQL, Datenbankentwurf und Datenbankbetrieb bis hin zu PL/SQL, Hostsprachenanbindung und WWW-Integration – wird daraufhin untersucht, welche *zusätzliche* Erkenntnis sich ergibt, wenn man es konsequent durch die Brille des tiefenorientierten Entwurfsparadigmas betrachtet. Die Antwort ist in jedem Fall dieselbe: Die Tiefenstruktur des Schemas – formal erfasst durch den Fremdschlüsselgraphen G_S und die in Definition 8.6 eingeführten Maße *depth*, *breadth* und τ – ist nicht nur ein Entwurfsdetail, sondern die zentrale Determinante für nahezu jede praktisch relevante Eigenschaft des Systems.

An wen sich dieses Buch richtet

Dieses Buch wendet sich an Studierende der Wirtschaftsinformatik, der Informatik und verwandter Studiengänge, die bereits eine erste Einführung in relationale Datenbanken erhalten haben oder parallel zu diesem Buch erhalten, ebenso wie an Praktikerinnen und Praktiker, die Datenbankschemata entwerfen, warten oder refaktorisieren und ein formales Vokabular suchen, um intuitiv empfundene „Code-Smells“ in Datenbankschemata präzise zu benennen und zu begründen. Vorausgesetzt werden grundlegende Kenntnisse der Mengenlehre und ein erster Kontakt mit SQL, wie er typischerweise in einer einführenden Lehrveranstaltung vermittelt wird; die formalen Beweise dieses Buches sind jedoch – in Anlehnung an die in [29] gepflegte Tradition – so gestaltet, dass sie auch ohne Vorkenntnisse in mathematischer Logik nachvollziehbar sind.

Da die Gliederung dieses Buches bewusst eng an das Lehrwerk von Kleinschmidt und Rank [11] angelehnt ist, eignet es sich insbesondere als *begleitende* oder *vertiefende* Lektüre zu Lehrveranstaltungen, die sich an jenem Werk orientieren: Jedes Kapitel dieses Buches kann parallel zum entsprechenden Kapitel des Vorbilds gelesen werden, wobei dieses Buch jeweils die zusätzliche, formal-beweisende Perspektive des tiefenorientierten Entwurfs beisteuert.

Wie dieses Buch zu lesen ist

Die Kapitel 1 bis 8 folgen der klassischen Gliederung eines einführenden Datenbanklehrbuchs und können – mit Ausnahme der jeweils am Ende eingestreuten Querverweise auf Kapitel 8.10 – weitgehend unabhängig voneinander gelesen werden, sofern die Leserin oder der Leser bereits über grundlegende SQL-Kenntnisse verfügt. Wer hingegen die zentrale formale Argumentation dieses Buches *zuerst* kennenlernen möchte, kann direkt mit Kapitel 8.10 („Tiefenorientierter Datenbankentwurf: Ein formales Gütekriterium“) beginnen, das in sich geschlossen die zentralen Definitionen, Sätze und Beweise dieses Buches enthält, und anschließend in den übrigen Kapiteln verfolgen, wie sich diese Theorie auf SQL (Kapitel 3), Datenbankentwurf (Kapitel 4), Datenbankbetrieb (Kapitel 5), PL/SQL (Kapitel 6), Hostsprachenanbindung (Kapitel 7) und WWW-Integration (Kapitel 8) auswirkt.

Sämtliche Definitionen, Sätze, Lemmata, Korollare, Propositionen und Bemerkungen sind kapitelweise nummeriert und werden im gesamten Buch konsistent referenziert;

ein Symbolverzeichnis findet sich in Anhang A.2. Beweise sind durch das Symbol ■ abgeschlossen. SQL-Beispiele orientieren sich, sofern nicht anders angegeben, an PostgreSQL, da dieses System sowohl die in Kapitel 3 behandelten Standardkonstrukte als auch die in Abschnitt 3.5 behandelte Erweiterung `DISTINCT ON` unterstützt.

Überblick über die Kapitel

Kapitel 1 („Was sind Datenbanksysteme?“) führt die grundlegenden Begriffe ein – DBMS, ACID-Eigenschaften, die Drei-Ebenen-Architektur nach ANSI/SPARC – und schließt mit einer Übersicht über Aufbau und Leitidee des Buches.

Kapitel 2 („Formale Modelle für Datenbanken“) behandelt das Relationenmodell und das Entity-Relationship-Modell und führt mit Definition 2.4 den Fremdschlüsselgraphen G_S ein, der im weiteren Verlauf des Buches als zentrales formales Objekt dient.

Kapitel 3 („Datenbankoperationen mit SQL“) behandelt `SELECT`-Anfragen, Views und Datenmodifikationen und enthält mit Abschnitt 3.5 eine ausführliche, formal untermauerte Behandlung des PostgreSQL-spezifischen Sprachmittels `DISTINCT ON` und seiner Beziehung zum Gruppenrepräsentantenproblem.

Kapitel 4 („Datenbank-Entwurf“) behandelt Anomalien, Normalformen und Dekomposition und enthält mit Abschnitt 4.4 einen vollständigen Synthesealgorithmus zur Schemaminimierung samt Pseudocode, Laufzeitanalyse und Korrektheitsbeweis.

Kapitel 5 („Datenbankbetrieb“) behandelt Transaktionen, Mehrbenutzerbetrieb, Zugriffsrechte und Systemtabellen und zeigt, wie sich die Tiefenpfadstruktur auf Sperrgranularität und Rechtevergabe auswirkt.

Kapitel 6 („PL/SQL“) behandelt prozedurale SQL-Erweiterungen, Cursor, Fehlerbehandlung, Prozeduren, Funktionen und Trigger.

Kapitel 7 („Datenbankzugriff in höheren Programmiersprachen“) behandelt die Impedanzdiskrepanz, Embedded SQL, ODBC/JDBC, die Perl-DBI-Schnittstelle und objektrelationales Mapping und enthält mit Theorem 7.3 einen formalen Nachweis, dass ein Tiefenpfadschema eine eindeutige, lokal konsistente Zerlegung des Anwendungscodes in Repository-Module induziert.

Kapitel 8 („WWW-Integration von Datenbanken“) behandelt die Architektur webbasierter Datenbankanwendungen, CGI-Skripte, Web-Formulare und Mehrbenutzerbetrieb im Web-Kontext.

Kapitel 8.10 („Tiefenorientierter Datenbankentwurf: Ein formales Gütekriterium“) bildet das formale Herzstück dieses Buches und enthält die vollständige, in sich geschlossene Argumentation für das Prinzip „Tiefe vor Breite“, einschließlich der graphentheoretischen Fundierung mittels Tiefen- und Breitensuche und eines durchgängigen Praxisbeispiels.

Kapitel 9 („Fazit und Ausblick“) fasst die Ergebnisse zusammen und entwickelt ein umfangreiches Forschungsprogramm, das von automatisiertem Schema-Tooling über temporale und verteilte Datenbanken, Sicherheits- und Datenschutzaspekte, formale Verifikation und maschinelles Lernen bis hin zu hybriden Graph-relationalen Systemen reicht.

Danksagung

Dieses Buch wäre ohne die in Abschnitt 1.4 genannte strukturelle Anlehnung an das Lehrwerk von Kleinschmidt und Rank [11] nicht in dieser Form entstanden; ihm gebührt der Dank für eine Gliederung, die sich seit nunmehr über zwei Jahrzehnten in der Lehre bewährt hat und die sich – wie dieses Buch zeigt – auch als Gerüst für eine deutlich formālere, beweisorientierte Darstellung eignet. Für verbleibende Fehler und Unzulänglichkeiten trägt allein der Autor die Verantwortung.

Bielefeld, 11. Juni 2026

Stephan Epp

Kapitel 1

Was sind Datenbanksysteme?

1.1 Unterschiede von DBMS zu klassischen Dateisystemen

Klassische dateiorientierte Anwendungssysteme verwalten Daten in applikationsspezifischen Dateien, deren Struktur fest in der Anwendung kodiert ist. Dies führt zu einer Reihe struktureller Probleme, die in einem Datenbankmanagementsystem (DBMS) systematisch vermieden werden:

- **Datenredundanz:** Dieselbe Information wird in mehreren Dateien dupliziert, was Speicherplatz verschwendet und Inkonsistenzen begünstigt.
- **Programm-Daten-Abhängigkeit:** Änderungen am Datenformat erfordern Anpassungen aller zugreifenden Programme.
- **Fehlende Mehrbenutzerfähigkeit:** Der gleichzeitige Zugriff mehrerer Anwendungen auf dieselben Daten ist nicht oder nur eingeschränkt möglich.
- **Fehlende Konsistenzsicherung:** Es existiert kein zentraler Mechanismus, der die Einhaltung von Integritätsbedingungen über Dateigrenzen hinweg garantiert.

Ein DBMS hingegen stellt eine zentrale Verwaltungskomponente bereit, die Daten unabhängig von den zugreifenden Anwendungsprogrammen organisiert, speichert und konsistent hält. Die Daten werden dabei in einem gemeinsamen, formal beschriebenen Schema abgelegt, auf das mehrere Anwendungen über eine deklarative Anfragesprache zugreifen.

Definition 1.1 (Datenbankmanagementsystem). Ein Datenbankmanagementsystem (DBMS) ist ein Softwaresystem, das die Definition, Speicherung, Manipulation und Verwaltung einer Datenbank $\mathcal{D}$ derart bereitstellt, dass

1. die logische Struktur der Daten (das *Schema*) von ihrer physischen Speicherung getrennt ist (Datenunabhängigkeit),
2. mehrere Anwendungsprogramme nebenläufig und konsistent auf $\mathcal{D}$ zugreifen können,

3. die Einhaltung definierter Integritätsbedingungen $\mathcal{I}$ für jeden Zustand von $\mathcal{D}$ garantiert wird,
4. ein Wiederanlauf nach Fehlern in einen konsistenten Zustand möglich ist (Persistenz und Recovery).

1.2 Forderungen an ein DBMS

Aus Definition 1.1 lassen sich die klassischen Forderungen an ein DBMS ableiten, die in der Literatur häufig unter dem Akronym *ACID* für Transaktionseigenschaften zusammengefasst werden (siehe Kapitel 5):

- **Atomarität (Atomicity):** Eine Transaktion wird entweder vollständig oder gar nicht ausgeführt.
- **Konsistenz (Consistency):** Jede Transaktion überführt die Datenbank von einem konsistenten in einen konsistenten Zustand.
- **Isolation:** Nebenläufig ausgeführte Transaktionen dürfen sich nicht gegenseitig beeinflussen, als ob sie seriell ausgeführt würden.
- **Dauerhaftigkeit (Durability):** Die Effekte einer erfolgreich abgeschlossenen Transaktion bleiben auch im Fehlerfall erhalten.

Darüber hinaus fordert man von einem modernen DBMS Effizienz bei Anfragen über große Datenmengen, Skalierbarkeit, ein feingranulares Zugriffsrechtssystem sowie Erweiterbarkeit des Schemas ohne Unterbrechung des laufenden Betriebs.

1.3 Abstraktionsebenen im DBMS

Die Architektur eines DBMS wird klassisch in drei Abstraktionsebenen gegliedert (ANSI/SPARC-Architektur):

1. **Externe Ebene:** Sichten (Views) auf die Daten, die für einzelne Anwendungen oder Benutzergruppen zugeschnitten sind.
2. **Konzeptuelle Ebene:** Das logische Gesamtschema der Datenbank, unabhängig von physischer Speicherung und individuellen Sichten.
3. **Interne Ebene:** Die physische Speicherorganisation, Indexstrukturen, Zugriffspfade.

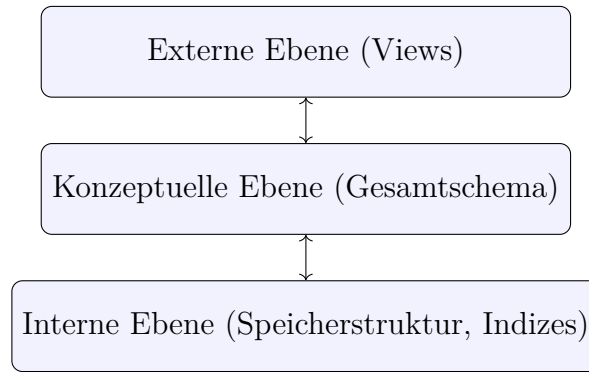


Abbildung 1.1: Drei-Ebenen-Architektur eines DBMS (ANSI/SPARC)

Diese strikte Trennung garantiert *logische* und *physische Datenunabhängigkeit*: Änderungen an der internen Speicherorganisation bleiben für die externe Ebene unsichtbar, und Änderungen am konzeptuellen Schema – sofern sie additiv erfolgen – beeinträchtigen bestehende externe Sichten nicht. Genau diese Eigenschaft wird in Kapitel 8.10 zum formalen Ausgangspunkt für den Nachweis genommen, dass tiefenorientierte Schemata eine überlegene Schemaevolution ermöglichen.

1.4 Aufbau und Leitidee dieses Buches

Die Gliederung dieses Buches folgt bewusst der Struktur des klassischen Lehrwerks von Kleinschmidt und Rank [11]: Kapitel 2 behandelt die formalen Modelle (Relationenmodell, Entity-Relationship-Modell), Kapitel 3 die Datenbankoperationen mit SQL, Kapitel 4 den Datenbankentwurf einschließlich Normalisierung, Kapitel 5 den Datenbankbetrieb (Transaktionen, Mehrbenutzerbetrieb, Zugriffsrechte), Kapitel 6 PL/SQL, Kapitel 7 die Anbindung an höhere Programmiersprachen und Kapitel 8 die WWW-Integration von Datenbanken.

Im Unterschied zum Vorbild wird jedoch in jedem dieser Kapitel ein zusätzlicher, durchgängiger formaler Strang verfolgt: das *tiefenorientierte Entwurfsparadigma*. Dieses Paradigma – in Kapitel 8.10 vollständig und in sich geschlossen hergeleitet – besagt, dass relationale Schemata, deren Fremdschlüsselstruktur als gerichteter, azyklischer *Tiefenpfad* (im Sinne eines DFS-Walds) organisiert ist, sternförmigen (BFS-artigen) Schemata in allen klassischen Gütekriterien überlegen sind. Die Kapitel 2 bis 8 verweisen an den jeweils relevanten Stellen auf dieses Paradigma und illustrieren es konkret an SQL-Beispielen, Normalformen, Transaktionsanalysen und WWW-Architekturen.

Kapitel 2

Formale Modelle für Datenbanken

2.1 Das Relationenmodell

Das Relationenmodell, eingeführt von Codd [4], bildet die mathematische Grundlage relationaler Datenbanksysteme. Es basiert auf dem Begriff der *Relation* im Sinne der Mengenlehre.

Definition 2.1 (Relation). Sei $D_1, D_2, \dots, D_n$ eine Folge von *Domänen* (Wertebereichen). Eine Relation $\mathcal{R}$ über dem Schema $(A_1 : D_1, \dots, A_n : D_n)$ ist eine endliche Teilmenge des kartesischen Produkts

$$\mathcal{R} \subseteq D_1 \times D_2 \times \dots \times D_n.$$

Die Elemente $A_1, \dots, A_n$ heißen *Attribute*, ein Element $t \in \mathcal{R}$ heißt *Tupel*.

Definition 2.2 (Relationenschema, Datenbankschema). Ein Relationenschema ist ein Paar $R = (attr(R), \Sigma_R)$, bestehend aus einer endlichen Menge von Attributen $attr(R) = \{A_1, \dots, A_n\}$ und einer Menge von Integritätsbedingungen Σ_R (insbesondere funktionalen Abhängigkeiten, vgl. Abschnitt 8.13). Ein Datenbankschema $\mathcal{S} = \{R_1, \dots, R_m\}$ ist eine endliche Menge von Relationenschemata.

Definition 2.3 (Schlüssel). Eine Menge $K \subseteq attr(R)$ heißt *Schlüssel* von R , wenn

1. K jedes Tupel von R eindeutig identifiziert (Eindeutigkeit), und
2. keine echte Teilmenge $K' \subsetneq K$ diese Eigenschaft besitzt (Minimalität).

Eine Menge $A \subseteq attr(R')$ in einem anderen Schema R' heißt *Fremdschlüssel* bezüglich R , wenn A und der Primärschlüssel von R über derselben Domäne definiert sind und jeder in A vorkommende Wert entweder NULL ist oder als Primärschlüsselwert in R existiert (referentielle Integrität).

2.2 Das Entity-Relationship-Modell

Das von Chen eingeführte Entity-Relationship-Modell (ER-Modell) dient der konzeptuellen Modellierung auf einer höheren Abstraktionsebene als das Relationenmodell. Zentrale Konstrukte sind:

- **Entitätstypen** repräsentieren Klassen von Objekten der Anwendungswelt (z. B. *Kunde*, *Auftrag*).
- **Attribute** beschreiben Eigenschaften von Entitäten oder Beziehungen.
- **Beziehungstypen** (Relationships) modellieren Assoziationen zwischen Entitätstypen, charakterisiert durch ihre *Kardinalität* (1:1, 1:n, n:m).

Definition 2.4 (ER-Schema als gerichteter Graph). Ein ER-Schema lässt sich als gerichteter Graph $G_{ER} = (V, E)$ auffassen, wobei V die Menge der Entitätstypen ist und $(E_i, E_j) \in E$ genau dann gilt, wenn ein Beziehungstyp zwischen E_i und E_j mit Kardinalität 1:n oder n:1 existiert, gerichtet von der Seite mit Kardinalität n (der „vielen“ Seite, die den Fremdschlüssel trägt) zur Seite mit Kardinalität 1.

Definition 2.4 ist der zentrale Bezugspunkt für das in Kapitel 8.10 entwickelte tiefenorientierte Entwurfparadigma: Die Frage, ob ein Datenbankschema „in die Tiefe“ oder „in die Breite“ orientiert ist, wird formal als eine Eigenschaft des Graphen G_{ER} – nämlich seiner Tiefe $depth(G_{ER})$ im Sinne von Definition 8.6 – charakterisiert.

2.3 Vom Entity-Relationship- zum Relationenmodell

Die Transformation eines ER-Schemas in ein Relationenschema folgt festen Regeln:

1. Jeder Entitätstyp wird zu einem Relationenschema mit den Attributen der Entität und einem Primärschlüssel.
2. Ein Beziehungstyp mit Kardinalität 1:n wird durch Aufnahme des Primärschlüssels der „1“-Seite als Fremdschlüssel in das Relationenschema der „n“-Seite realisiert.
3. Ein Beziehungstyp mit Kardinalität n:m erfordert ein eigenes Relationenschema, dessen Schlüssel sich aus den Primärschlüsseln beider beteiligter Entitätstypen zusammensetzt.

Proposition 2.1. Sei $G_{ER} = (V, E)$ ein ER-Schema gemäß Definition 2.4 und $\mathcal{S}$ das nach obigen Regeln daraus abgeleitete relationale Schema. Dann existiert für jede Kante $(R_i, R_j) \in E$ ein Fremdschlüssel in $attr(R_i)$, der auf den Primärschlüssel von R_j verweist, sofern die Beziehung nicht n:m ist (in diesem Fall wird die Kante in zwei Kanten über ein zusätzliches Verbindungsschema zerlegt). Insbesondere bleibt die Graphstruktur von G_{ER} – bis auf diese Zerlegung von n:m-Kanten – im Fremdschlüsselgraphen $G_{\mathcal{S}}$ des relationalen Schemas erhalten.

Beweis. Für 1:n-Beziehungen folgt die Behauptung unmittelbar aus Transformationsregel 2: Der Primärschlüssel von R_j (der „1“-Seite) wird als Attribut in $attr(R_i)$ aufgenommen und erfüllt nach Definition 2.3 die Bedingungen eines Fremdschlüssels, da seine Werte aus dem Wertebereich des Primärschlüssels von R_j stammen und referentielle Integrität gefordert wird. Für n:m-Beziehungen erzeugt Regel 3 ein neues Schema R_{ij} mit Fremdschlüsseln auf *beide* Seiten R_i und R_j ; die ursprüngliche Kante (R_i, R_j)

wird somit durch den Pfad $R_i \leftarrow R_{ij} \rightarrow R_j$ ersetzt, was strukturell einer Aufspaltung in zwei gerichtete Kanten mit gemeinsamem Zielknoten R_{ij} entspricht. In beiden Fällen bleibt die Adjazenzstruktur – modulo dieser lokalen Zerlegung – erhalten, womit die Behauptung gezeigt ist. $\square$

Diese Proposition ist von zentraler Bedeutung, da sie es erlaubt, alle graphentheoretischen Aussagen über G_{ER} (insbesondere die in Kapitel 8.10 bewiesenen Sätze über Tiefe und Breite) direkt auf den Fremdschlüsselgraphen des relationalen Schemas zu übertragen.

2.4 Wichtige Operationen auf Relationen (Tabellen)

Die relationale Algebra definiert eine Menge von Operationen, die Relationen auf neue Relationen abbilden und damit abgeschlossen („relational vollständig“) sind:

- **Selektion** $\sigma_\varphi(\mathcal{R})$: Auswahl der Tupel aus $\mathcal{R}$, die die Bedingung φ erfüllen.
- **Projektion** $\pi_{A_1, \dots, A_k}(\mathcal{R})$: Beschränkung auf die angegebenen Attribute, mit anschließender Duplikateliminierung.
- **Vereinigung, Differenz, Durchschnitt**: mengentheoretische Operationen auf Relationen mit gleichem Schema.
- **Kartesisches Produkt** $\mathcal{R} \times \mathcal{S}$: Kombination aller Tupelpaare.
- **Verbund (Join)** $\mathcal{R} \bowtie_\varphi \mathcal{S}$: Selektion auf dem kartesischen Produkt gemäß einer Verbundbedingung φ , typischerweise einer Gleichheit zwischen Fremdschlüssel- und Primärschlüsselattributen (*Equi-Join*).
- **Umbenennung** $\rho_{A \rightarrow B}(\mathcal{R})$: Umbenennung von Attributen.

Definition 2.5 (Natürlicher Verbund über Fremdschlüsselkante). Sei (R_i, R_j) eine Kante im Fremdschlüsselgraphen G_S gemäß Proposition 2.1, mit Fremdschlüssel $F \subseteq \text{attr}(R_i)$, der auf den Primärschlüssel $K \subseteq \text{attr}(R_j)$ verweist. Der *kanonische Verbund* entlang dieser Kante ist

$$R_i \bowtie_{F=K} R_j.$$

Definition 2.5 liefert die formale Grundlage für die in Abschnitt 8.15 und Kapitel 8.10 durchgeführte Analyse der Join-Komplexität entlang von Tiefenpfaden im Vergleich zu Sternschemas.

2.5 Zusatzforderungen für relationale DBMS

Codd formulierte zwölf Regeln, die ein DBMS erfüllen muss, um als „vollständig relational“ zu gelten [4, 5]. Für die vorliegende Arbeit sind insbesondere die folgenden Forderungen relevant:

- **Informationsregel**: Sämtliche Information – inklusive Metadaten – wird ausschließlich in Form von Werten in Tabellen dargestellt.

- **Garantierter Zugriff:** Jeder Datenwert ist eindeutig über Tabellennamen, Primärschlüsselwert und Attributnamen adressierbar.
- **Systematische Behandlung von Nullwerten:** NULL repräsentiert fehlende oder unanwendbare Information unabhängig vom Datentyp.
- **Strukturunabhängigkeit:** Die Anfragesprache darf nicht von der physischen Speicherstruktur abhängen.
- **Integritätsunabhängigkeit:** Integritätsbedingungen (insbesondere referentielle Integrität gemäß Definition 2.3) werden im Schema selbst definiert und vom DBMS automatisch durchgesetzt, unabhängig von Anwendungsprogrammen.

Die letztgenannte Forderung – Integritätsunabhängigkeit – ist von besonderer Bedeutung für die in Kapitel 8.10 diskutierte Wartbarkeit: Da Fremdschlüsselbeziehungen im Schema selbst deklariert werden, lässt sich die Tiefe eines Schemas $depth(G_S)$ direkt aus dem Datenbankkatalog (vgl. Abschnitt 5.4) ablesen, ohne den Anwendungsquellcode analysieren zu müssen.

Kapitel 3

Datenbankoperationen mit SQL

3.1 Anforderungen an einen Datenbank-Server

Ein Datenbank-Server muss nebenläufige Verbindungen mehrerer Clients verwalten, Anfragen optimieren und ausführen sowie Transaktionen gemäß den ACID-Eigenschaften (vgl. Kapitel 1) garantieren. Die in den folgenden Abschnitten vorgestellten SQL-Konstrukte werden in diesem Buch konsequent anhand des in Kapitel 8.10 entwickelten Tiefenpfadschemas illustriert (vgl. Abschnitt 8.19).

3.2 SQL-Standards

SQL (Structured Query Language) wurde mit ANSI SQL-86 erstmals standardisiert und seitdem mehrfach erweitert: SQL-92 (umfangreiche Erweiterung der Datentypen und Integritätsbedingungen), SQL:1999 (rekursive Anfragen, objektrelationale Erweiterungen, Trigger), sowie nachfolgende Standards (SQL:2003, SQL:2008, SQL:2011, SQL:2016) mit Erweiterungen für XML, Window-Funktionen, temporale Tabellen und JSON-Unterstützung. Die in diesem Buch verwendeten Konstrukte orientieren sich am SQL:1999-Standard mit punktuellen Hinweisen auf spätere Erweiterungen.

3.3 Sprachstruktur von SQL

SQL gliedert sich in mehrere Teilsprachen:

- **DDL** (Data Definition Language): `CREATE`, `ALTER`, `DROP` – Definition von Schemaobjekten.
- **DML** (Data Manipulation Language): `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- **DCL** (Data Control Language): `GRANT`, `REVOKE` – Verwaltung von Zugriffsrechten.
- **TCL** (Transaction Control Language): `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

3.4 Abrufen von Informationen mit SELECT

Die allgemeine Form einer SELECT-Anweisung lautet:

```
1 SELECT [DISTINCT] Ausdrucksliste
2 FROM   Tabellenliste
3 [WHERE Bedingung]
4 [GROUP BY Attributliste]
5 [HAVING Gruppenbedingung]
6 [ORDER BY Attributliste]
```

Semantisch entspricht eine SELECT-Anweisung einer Komposition relationaler Algebra-Operatoren gemäß Abschnitt 2.4:

$$\pi_{\text{Ausdrucksliste}} \left(\sigma_{\text{Bedingung}} \left(R_1 \times R_2 \times \dots \times R_k \right) \right)$$

mit anschließender optionaler Gruppierung, Aggregation und Sortierung.

Beispiel 3.1 (Verbund über eine Fremdschlüsselkante). Sei AUFTRAG ein Relationenschema mit Fremdschlüssel `kunden_id` auf KUNDE (vgl. Definition 2.5). Die Anfrage

```
1 SELECT a.auftrag_id, k.name, a.datum
2 FROM   Auftrag a
3 JOIN   Kunde k ON a.kunden_id = k.kunden_id
4 WHERE  a.datum >= DATE '2026-01-01'
```

realisiert den kanonischen Verbund $\text{AUFTRAG} \bowtie_{\text{kunden_id=kunden_id}} \text{KUNDE}$ mit anschließender Selektion und Projektion. Im Tiefenpfadschema aus Abschnitt 8.19 entspricht dieser Verbund genau einer Kante des DFS-Walds.

Mehrstufige Verbunde entlang eines Tiefenpfads $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_k$ werden durch Verkettung mehrerer JOIN-Klauseln realisiert; die Komplexitätsanalyse hierzu findet sich in Abschnitt 8.15.

3.5 Das Sprachmittel DISTINCT ON: Gruppenrepräsentanten entlang des Tiefenpfads

Im Folgenden wird ein Sprachmittel vorgestellt, das zwar nicht Teil des SQL-Standards ist, aber für die in diesem Buch entwickelte Tiefenpfadarchitektur (Kapitel 8.10) von besonderer Bedeutung ist: die PostgreSQL-spezifische Erweiterung `SELECT DISTINCT ON (...)`. Dieses Sprachmittel löst auf deklarative, mengenorientierte Weise eine Aufgabe, die in der Praxis außerordentlich häufig auftritt: das Auffinden eines *Gruppenrepräsentanten* – z. B. des jeweils neuesten Tupels pro Fremdschlüsselwert.

3.5.1 Motivation: Standard-DISTINCT versus Gruppenrepräsentanten

Der SQL-Standard kennt nur das einfache Schlüsselwort `DISTINCT`, das gemäß Abschnitt 3.4 als Duplikateliminierung nach der Projektion π wirkt:

Definition 3.1 (Standard-DISTINCT). Sei $\mathcal{R}$ eine Relation und $A_1, \dots, A_k \in \text{attr}(\mathcal{R})$. Die Anweisung `SELECT DISTINCT  $A_1, \dots, A_k$  FROM  $\mathcal{R}$` liefert das Ergebnis

$$\pi_{A_1, \dots, A_k}(\mathcal{R})$$

im Sinne der relationalen Algebra (Abschnitt 2.4), d. h. die Menge *aller verschiedenen Wertekombinationen* $(t[A_1], \dots, t[A_k])$, die in $\mathcal{R}$ auftreten – ohne Information darüber, *welches* Tupel von $\mathcal{R}$ diese Kombination geliefert hat, falls mehrere Tupel dieselbe Kombination aufweisen.

Häufig stellt sich jedoch eine andere, semantisch verwandte, aber algebraisch verschiedene Aufgabe: Gegeben eine Gruppierung von $\mathcal{R}$ nach einer Attributmenge $G \subseteq \text{attr}(\mathcal{R})$ – etwa $G = \{\text{kunden_id}\}$ –, soll für jede Gruppe *ein vollständiges Tupel* zurückgegeben werden, nämlich dasjenige, das bezüglich einer Ordnung $\prec$ auf $\text{attr}(\mathcal{R})$ *minimal* bzw. *maximal* ist. Diese Aufgabe wird im Folgenden formal als *Gruppenrepräsentantenproblem* bezeichnet.

Definition 3.2 (Gruppenrepräsentantenproblem). Sei $\mathcal{R}$ eine Relation, $G \subseteq \text{attr}(\mathcal{R})$ eine *Gruppierungsattributmenge* und $\prec$ eine totale Ordnung auf den Tupeln von $\mathcal{R}$, induziert durch eine Attributfolge $(B_1, \dots, B_m) \subseteq \text{attr}(\mathcal{R})$ mit zugehörigen Sortierrichtungen (ASC/DESC). Für jeden in $\mathcal{R}$ auftretenden Wert $g \in \pi_G(\mathcal{R})$ sei

$$\mathcal{R}_g := \sigma_{(A_1, \dots, A_{|G|})=g}(\mathcal{R})$$

die Gruppe aller Tupel mit Gruppierungswert g . Das Gruppenrepräsentantenproblem besteht darin, für jedes $g \in \pi_G(\mathcal{R})$ genau das (bezüglich $\prec$ kleinste) Tupel

$$t_g := \min_{\prec} \mathcal{R}_g$$

zu bestimmen und die Ergebnisrelation $\{t_g \mid g \in \pi_G(\mathcal{R})\}$ zurückzugeben.

Bemerkung 3.1. Definition 3.2 ist *nicht* durch Standard-DISTINCT (Definition 3.1) ausdrückbar, da DISTINCT nur auf den *projizierten* Attributen operiert und keine Möglichkeit bietet, „das übrige Tupel mitzuführen“, das zu einer bestimmten Wertekombination gehört.

3.5.2 Das Standard-SQL-Vorgehen: Fensterfunktion mit ROW_NUMBER()

Innerhalb des SQL-Standards (SQL:2003 ff., vgl. Abschnitt 3.2) wird das Gruppenrepräsentantenproblem über eine *Fensterfunktion* (Window-Funktion) gelöst, typischerweise ROW_NUMBER() in Kombination mit einer PARTITION BY-Klausel:

```

1 WITH RankedOrders AS (
2   SELECT
3     kunden_id, auftrag_id, datum, gesamtbetrag,
4     ROW_NUMBER() OVER (
5       PARTITION BY kunden_id
6       ORDER BY datum DESC

```

```

7      ) AS rn
8      FROM Auftrag
9  )
10 SELECT kunden_id, auftrag_id, datum, gesamtbetrag
11 FROM RankedOrders
12 WHERE rn = 1;

```

Semantisch geschieht hier Folgendes: Die `PARTITION BY`-Klausel zerlegt $\mathcal{R} = \text{AUFTRAG}$ in die Gruppen $\mathcal{R}_g$ aus Definition 3.2 mit $G = \{\text{kunden_id}\}$. Innerhalb jeder Gruppe nummeriert `ROW\_NUMBER()` die Tupel gemäß der durch `ORDER BY datum DESC` induzierten Ordnung $\prec$ durch, beginnend bei 1 für das bezüglich $\prec$ kleinste (hier: das Tupel mit dem größten `datum`, da `DESC`) Tupel. Der äußere Filter `WHERE rn = 1` selektiert genau dieses minimale Tupel pro Gruppe – also exakt t_g aus Definition 3.2.

Bemerkung 3.2 (Konzeptionelle Zweistufigkeit). Das `ROW\_NUMBER()`-Vorgehen ist konzeptionell *zweistufig*: In einer ersten, materialisierten Zwischenrelation (`RankedOrders`, realisiert über eine Common Table Expression bzw. einen Subquery) wird *jedem* Tupel von $\mathcal{R}$ eine Rangnummer zugewiesen – also $|\mathcal{R}|$ Tupel erzeugt –, bevor in einer zweiten Stufe die Filterung `rn = 1` auf $|\pi_G(\mathcal{R})|$ Tupel reduziert. Dieser Zwischenschritt ist algebraisch nicht notwendig, da das Endergebnis nur $|\pi_G(\mathcal{R})| \leq |\mathcal{R}|$ Tupel umfasst; er ist jedoch eine direkte Konsequenz davon, dass Fensterfunktionen im SQL-Standard *zeilenweise* (pro Eingabetupel ein Ausgabewert) definiert sind und nicht direkt als gruppierende Aggregation mit „Tupel-Rückgabe“ formuliert werden können.

3.5.3 DISTINCT ON: Direkte deklarative Lösung in PostgreSQL

PostgreSQL bietet mit `SELECT DISTINCT ON (...)` eine direkte, einstufige Lösung des Gruppenrepräsentantenproblems:

```

1 SELECT DISTINCT ON (kunden_id)
2     kunden_id, auftrag_id, datum, gesamtbetrag
3 FROM   Auftrag
4 ORDER BY kunden_id, datum DESC;

```

Definition 3.3 (Semantik von `DISTINCT ON`). Sei $\mathcal{R}$ eine Relation, $G = (A_1, \dots, A_p)$ die in `DISTINCT ON` angegebene Attributfolge und $(B_1, \dots, B_q)$ die in der `ORDER BY`-Klausel zusätzlich angegebenen Attribute mit Sortierrichtungen, sodass die vollständige Sortierordnung durch $(A_1, \dots, A_p, B_1, \dots, B_q)$ gegeben ist. Dann liefert

$$\text{SELECT DISTINCT ON } (A_1, \dots, A_p) \dots \text{FROM } \mathcal{R} \text{ ORDER BY } A_1, \dots, A_p, B_1, \dots, B_q$$

für jeden in $\mathcal{R}$ auftretenden Wert $g = (t[A_1], \dots, t[A_p])$ genau ein Tupel: dasjenige Tupel $t_g \in \mathcal{R}_g$, das bezüglich der durch $(B_1, \dots, B_q)$ definierten Ordnung $\prec_B$ *minimal* ist (bzw. *maximal*, falls die jeweilige Spalte `DESC` sortiert wird).

Satz 3.1 (Äquivalenz von `DISTINCT ON` und `ROW\_NUMBER()`-Filterung). Die Anweisung

$$Q_1 = \text{SELECT DISTINCT ON } (G) * \text{FROM } \mathcal{R} \text{ ORDER BY } G, B_1, \dots, B_q$$

und die Anweisung

```
Q2 = SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER
    (PARTITION BY G ORDER BY B1, ..., Bq) AS rn
    FROM R
) t WHERE rn = 1
```

liefern für jede Ausprägung r von $\mathcal{R}$ dasselbe Ergebnis (bis auf die zusätzliche Spalte **rn** in Q_2 , die durch eine äußere Projektion entfernt werden kann), sofern $\prec_B$ eine totale Ordnung auf jeder Gruppe $\mathcal{R}_g$ induziert (d. h. keine zwei Tupel derselben Gruppe sind bezüglich $(B_1, \dots, B_q)$ identisch – andernfalls ist das Ergebnis in beiden Fällen gleichermaßen *nicht-deterministisch* bezüglich der Auswahl unter den $\prec_B$ -gleichwertigen Tupeln).

Beweis. Wir zeigen, dass beide Anfragen für jede Gruppe $g \in \pi_G(r)$ dasselbe Tupel t_g liefern.

Q_2 liefert t_g : Nach Definition von `ROW_NUMBER()` mit `PARTITION BY G ORDER BY B1, ..., Bq` erhält innerhalb der Partition $\mathcal{R}_g$ dasjenige Tupel t , für das $(t[B_1], \dots, t[B_q])$ bezüglich $\prec_B$ minimal ist, den Rang **rn** = 1 zugewiesen (Definition der Fensterfunktion `ROW_NUMBER`: aufsteigende Nummerierung gemäß der `ORDER BY`-Klausel der `OVER`-Spezifikation, beginnend bei 1). Da $\prec_B$ nach Voraussetzung total auf $\mathcal{R}_g$ ist, ist dieses Tupel eindeutig bestimmt und gleich $t_g = \min_{\prec_B} \mathcal{R}_g$ nach Definition 3.2 (mit $\prec = \prec_B$ auf jeder Gruppe). Der äußere Filter `WHERE rn = 1` selektiert somit genau $\{t_g \mid g \in \pi_G(r)\}$.

Q_1 liefert t_g : Nach Definition 3.3 sortiert die `ORDER BY`-Klausel $G, B_1, \dots, B_q$ zunächst alle Tupel von r lexikographisch nach G und führt innerhalb jeder so entstehenden Gruppe $\mathcal{R}_g$ (Tupel mit gleichem G -Wert g) die Sortierung gemäß $\prec_B$ fort. `DISTINCT ON (G)` behält sodann – pro eindeutigem G -Wert g – das in dieser Gesamtordnung *erste* Tupel der Gruppe $\mathcal{R}_g$. Da innerhalb von $\mathcal{R}_g$ die Sortierung durch $\prec_B$ bestimmt ist, ist dieses erste Tupel $\min_{\prec_B} \mathcal{R}_g = t_g$.

Da Q_1 und Q_2 für jede Gruppe g dasselbe Tupel t_g liefern und $\pi_G(r)$ in beiden Fällen dieselbe Indexmenge der Gruppen ist, sind die Ergebnismengen von Q_1 und der um **rn** projizierten Version von Q_2 identisch. $\square$

Bemerkung 3.3 (Wichtige Syntaxregel). Aus dem Beweis von Theorem 3.1 folgt unmittelbar die zentrale syntaktische Regel von `DISTINCT ON`: Die in `DISTINCT ON (G)` angegebenen Ausdrücke *müssen* als *Präfix* der `ORDER BY`-Klausel auftreten – formal: Die `ORDER BY`-Liste muss von der Form $(A_1, \dots, A_p, B_1, \dots, B_q)$ mit $G = (A_1, \dots, A_p)$ sein. Andernfalls wäre die im Beweis verwendete Zerlegung der Gesamtordnung in „zuerst nach G , dann innerhalb jeder G -Gruppe nach $(B_1, \dots, B_q)$ “ nicht mehr gültig, und PostgreSQL kann nicht mehr garantieren, dass das „erste Tupel pro G -Gruppe“ auch das bezüglich $\prec_B$ minimale Tupel ist. PostgreSQL erzwingt diese Regel durch einen Syntaxfehler zur Übersetzungszeit.

3.5.4 Effizienzvergleich und Bezug zur Tiefenpfadarchitektur

Proposition 3.2 (Operatorbaum von DISTINCT ON). Während die ROW_NUMBER()-Lösung Q_2 aus Theorem 3.1 im Ausführungsplan typischerweise die Operatorfolge

$$\text{Sort} \rightarrow \text{WindowAgg} \rightarrow \text{Filter} \quad (\text{rn} = 1)$$

erzeugt – wobei WindowAgg für *jedes* Tupel von $\mathcal{R}$ eine Rangnummer materialisiert –, kann DISTINCT ON (G) vom Optimierer als

$$\text{Sort} \rightarrow \text{Unique} \quad (\text{on } G)$$

übersetzt werden: Der Unique-Operator durchläuft den sortierten Strom und gibt – ohne Materialisierung einer Rangspalte – für jede Änderung des G -Werts genau das erste neu angetroffene Tupel weiter. Beide Pläne haben dieselbe asymptotische Komplexität $O(|\mathcal{R}| \log |\mathcal{R}|)$ (dominiert durch den Sort-Schritt), jedoch mit einer kleineren Konstanten für DISTINCT ON, da keine zusätzliche Spalte berechnet, gespeichert und anschließend wieder herausgefiltert werden muss.

Bemerkung 3.4 (Bezug zu Theorem 8.7 und zum Tiefenpfad). DISTINCT ON entfaltet seinen größten praktischen Nutzen genau an den Kanten des Fremdschlüsselgraphen G_S (Definition 2.4): Das in Abschnitt 3.5.2 und 3.5.3 durchgängig verwendete Beispiel – „die jeweils neueste AUFTRAG-Zeile pro KUNDE“ – ist die Anfrage *entlang* der Kante (AUFTRAG, KUNDE) des Tiefenpfads aus Abschnitt 8.19, präzisiert auf „den jeweils *aktuellsten Repräsentanten*“ pro Knoten der Elternrelation. Damit ergänzt DISTINCT ON die in Definition 2.5 eingeführte Operation des *kanonischen Verbunds* um eine zweite, ebenso fundamentale Operation entlang von Tiefenpfadkanten:

- Der *kanonische Verbund* $R_i \bowtie_{F=K} R_{i+1}$ (Definition 2.5) beantwortet die Frage: „Zu welchem R_{i+1} -Tupel gehört dieses R_i -Tupel?“
- Die *DISTINCT ON-Projektion* $\text{DISTINCTON}_F(R_i, \prec_B)$ beantwortet die *umgekehrte* und in der Praxis ebenso häufige Frage: „Welches ist (gemäß $\prec_B$) das repräsentative R_i -Tupel zu diesem R_{i+1} -Tupel?“

Da – wie in Theorem 8.7 gezeigt – im Tiefenpfadschema die Relation R_i *nicht* mit den Attributen aller anderen Entitäten vermischt ist (anders als in einem Sternschema, in dem die für DISTINCT ON (G) herangezogene Gruppierungsspalte G häufig erst über einen zusätzlichen Verbund mit der Hub-Relation H ermittelt werden müsste), kann DISTINCT ON (F) *direkt* auf R_i angewendet werden, mit einem Index auf $(F, B_1, \dots, B_q)$ als idealer Zugriffsstruktur: Ein solcher zusammengesetzter Index erlaubt es dem Unique-Operator aus Proposition 3.2, den Sort-Schritt vollständig zu entfallen, da der Index die Tupel bereits in der für DISTINCT ON benötigten Reihenfolge liefert – der Ausführungsplan reduziert sich dann auf einen reinen Index Scan mit Unique-Operator und besitzt Kosten $O(|\pi_G(R_i)| + \log |R_i|)$ statt $O(|R_i| \log |R_i|)$.

Beispiel 3.2 (DISTINCT ON im Tiefenpfadschema). Im Tiefenpfadschema aus Abschnitt 8.19 liefert

```

1 CREATE INDEX idx_auftrag_kunde_datum
2   ON Auftrag (kunden_id, datum DESC);
3
4 SELECT DISTINCT ON (kunden_id)
5     kunden_id, auftrag_id, datum, gesamtbetrag
6 FROM   Auftrag
7 ORDER BY kunden_id, datum DESC;

```

für jeden Kunden den jeweils neuesten Auftrag in $O(|\pi_{\text{kunden_id}}(\text{AUFTRAG})|)$ – also *linear in der Anzahl der Kunden*, unabhängig von der Gesamtzahl der Aufträge. Im Sternschema, in dem `kunden_id` als denormalisiertes Attribut zunächst über einen Verbund mit der Hub-Relation H ermittelt werden müsste (vgl. Theorem 8.7), kann der Index `(kunden_id, datum DESC)` nicht in derselben Weise direkt auf der Quellrelation des `DISTINCT ON` angelegt werden, sodass der Sort-Schritt aus Proposition 3.2 in der Regel nicht entfällt.

3.5.5 Weitere Anwendungsfälle entlang des Tiefenpfads

Das Gruppenrepräsentantenproblem (Definition 3.2) tritt im Tiefenpfadschema systematisch an jeder Kante auf, an der eine *historisierende* oder *eins-zu-viele*-Beziehung vorliegt. Typische Anwendungen von `DISTINCT ON` im Kontext dieses Buches sind:

- **Aktuellster Status:** Für jede `AUFTRAG`-Zeile den aktuellsten Eintrag einer `AUFTRAGSTATUSVERLAUF`-Tabelle ermitteln (`DISTINCT ON (auftrag_id) ...ORDER BY auftrag_id, geaendert_am DESC`).
- **Erstes Vorkommen:** Für jede `REGION` den Kunden mit der kleinsten `kunden_id` (z. B. den „dienstältesten“ Kunden) ermitteln (`DISTINCT ON (region_id) ...ORDER BY region_id, kunden_id ASC`).
- **Extremwert je Gruppe mit vollständigem Tupel:** Für jeden `KUNDE` den `AUFTRAG` mit dem höchsten `gesamtbetrag` – inklusive aller übrigen Attribute dieses Auftrags, nicht nur des Maximalwerts selbst, was mit einem einfachen `MAX`-Aggregat (Abschnitt 3.7) nicht möglich wäre, ohne erneut einen Selbstverbund oder eine Fensterfunktion zu verwenden.

In allen drei Fällen gilt die in Bemerkung 3.4 entwickelte Beobachtung: Die Gruppierungsspalte G von `DISTINCT ON` ist im Tiefenpfadschema stets ein *lokales* Attribut oder ein *unmittelbarer* Fremdschlüssel der Quellrelation – niemals ein über mehrere Verbundstufen $\delta > 1$ (Abschnitt 8.15) vermitteltes Attribut –, was `DISTINCT ON` zu einem natürlichen Begleiter des in Kapitel 8.10 entwickelten Entwurfsparadigmas macht.

Eine View ist eine benannte, gespeicherte SQL-Abfrage, die sich wie eine Tabelle verwenden lässt:

```

1 CREATE VIEW AuftragsUebersicht AS
2 SELECT a.auftrag_id, k.name AS kunde, p.bezeichnung AS produkt
3 FROM   Auftrag a
4 JOIN   Kunde k ON a.kunden_id = k.kunden_id

```

```

5 JOIN    Auftragsposition ap ON ap.auftrag_id = a.auftrag_id
6 JOIN    Produkt p ON p.produkt_id = ap.produkt_id;

```

Views realisieren die externe Ebene der ANSI/SPARC-Architektur (vgl. Abbildung 1.1). Im tiefenorientierten Entwurf entsprechen Views typischerweise *materialisierten Pfaden* im DFS-Wald des Schemas: Eine View, die mehrere Tiefenstufen zu einer flachen Sicht zusammenfasst, stellt für lesende Anwendungen die Bequemlichkeit eines Sternschemas bereit, ohne dass das zugrunde liegende Schema selbst sternförmig sein muss. Dies ist ein zentrales Argument gegen den Einwand, tiefenorientierte Schemata seien für Leseoperationen unbequem (vgl. Abschnitt 8.17).

3.6 Datenmodifikationen

Die Datenmanipulationsoperationen `INSERT`, `UPDATE` und `DELETE` unterliegen den im Schema definierten Integritätsbedingungen. Insbesondere erzwingt referentielle Integrität (Definition 2.3) bei `DELETE`-Operationen auf einem referenzierten Tupel eine der folgenden Strategien:

- `ON DELETE RESTRICT`: Löschen wird verweigert, falls referenzierende Tupel existieren.
- `ON DELETE CASCADE`: Referenzierende Tupel werden automatisch mitgelöscht.
- `ON DELETE SET NULL`: Der Fremdschlüsselwert wird auf `NULL` gesetzt.

Die Wahl dieser Strategie entlang einer Kante des Fremdschlüsselgraphen G_S hat unmittelbare Auswirkung auf die in Abschnitt 8.15 analysierte Schreibkomplexität: In einem Tiefenpfad propagiert `CASCADE` entlang einer einzigen, klar definierten Kette, während in einem Sternschema mit zentralem Hub eine `CASCADE`-Operation potenziell *alle* abhängigen Relationen gleichzeitig betrifft.

3.7 Darstellung von Datenbankinhalten

Zur Darstellung von Anfrageergebnissen stehen neben der tabellarischen Standardausgabe Aggregations- und Gruppierungsfunktionen (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) sowie Window-Funktionen (`OVER (PARTITION BY ... ORDER BY ...)`) zur Verfügung. Letztere erlauben es, entlang eines Tiefenpfads kumulative Kennzahlen (z. B. laufende Summen pro Auftrag) zu berechnen, ohne den natürlichen Verbund mehrfach auszuführen.

Kapitel 4

Datenbank-Entwurf

4.1 Anomalien in Datenbanken

Schlecht entworfene Relationenschemata führen zu drei klassischen Anomalietypen: Einfüge-, Löscho- und Änderungsanomalien (vgl. Abschnitt 8.14 für die vollständige formale Klassifikation und den Beweis von Theorem 8.4). Ursache aller drei Anomalietypen ist *Redundanz*, die wiederum aus der Verletzung von Normalformen resultiert.

4.2 Normalformen von Relationen

Die Normalformenhierarchie $1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF$ wird in Abschnitt 8.13 formal eingeführt. Zentral für dieses Buch ist Theorem 8.1 (Transitivitätsverteilung): Eine Kette funktionaler Abhängigkeiten $K \rightarrow A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n$ zwingt – soll 3NF erreicht werden – zu einer Zerlegung in eine lineare Kette von Relationen $R_1(K, A_1), R_2(A_1, A_2), \dots$. Diese Kette ist nichts anderes als ein *Tiefenpfad* im Sinne von Definition 8.6.

4.3 Dekomposition

Definition 4.1 (Verlustfreie Dekomposition). Eine Zerlegung einer Relation R mit Schema $attr(R) = X \cup Y$ in R_1 mit $attr(R_1) = X$ und R_2 mit $attr(R_2) = Y$ heißt *verlustfrei* (lossless-join), wenn für jede zulässige Ausprägung r von R gilt:

$$r = \pi_X(r) \bowtie \pi_Y(r).$$

Satz 4.1 (Lossless-Join-Kriterium). Die Zerlegung von R in $R_1(X)$ und $R_2(Y)$ mit $X \cup Y = attr(R)$ ist genau dann verlustfrei, wenn

$$(X \cap Y) \rightarrow X \quad \text{oder} \quad (X \cap Y) \rightarrow Y$$

in der Menge der funktionalen Abhängigkeiten von R gilt.

Beweis. ($\Leftarrow$) Gelte $(X \cap Y) \rightarrow X$. Sei $t \in \pi_X(r) \bowtie \pi_Y(r)$. Dann existieren $t_1, t_2 \in r$ mit $t[X] = t_1[X]$ und $t[Y] = t_2[Y]$ sowie $t_1[X \cap Y] = t_2[X \cap Y] = t[X \cap Y]$. Da

$(X \cap Y) \rightarrow X$, folgt $t_1[X] = t_2[X]$ aus $t_1[X \cap Y] = t_2[X \cap Y]$, also $t_1[X] = t[X] = t_2[X]$. Damit stimmen t_1 und t_2 auf $X \cap Y$ und auf X überein; da $X \cup Y = \text{attr}(R)$ und t durch $t[X]$ und $t[Y] = t_2[Y]$ vollständig bestimmt ist, gilt $t = t_2 \in r$. Die Inklusion $r \subseteq \pi_X(r) \bowtie \pi_Y(r)$ gilt stets trivial, also $r = \pi_X(r) \bowtie \pi_Y(r)$. ($\Rightarrow$) Der Beweis der Notwendigkeit erfolgt durch Kontraposition mittels eines Gegenbeispiel-Tableaus, siehe [8], Theorem 3.18. Gilt weder $(X \cap Y) \rightarrow X$ noch $(X \cap Y) \rightarrow Y$, so existieren zwei Tupel $t_1, t_2 \in r$ mit $t_1[X \cap Y] = t_2[X \cap Y]$, aber $t_1[X] \neq t_2[X]$ und $t_1[Y] \neq t_2[Y]$. Der Verbund $\pi_X(r) \bowtie \pi_Y(r)$ enthält dann zusätzlich die „Spurious Tuples“ $(t_1[X], t_2[Y])$ und $(t_2[X], t_1[Y])$, die nicht in r enthalten sind, womit $r \subsetneq \pi_X(r) \bowtie \pi_Y(r)$ und die Zerlegung verlustbehaftet ist. $\square$

Bemerkung 4.1. Theorem 4.1 ist die formale Grundlage des in Theorem 8.1 verwendeten „Lossless-Join-Lemma“: Bei der Zerlegung entlang einer transitiven Abhängigkeit $K \rightarrow A_i \rightarrow A_{i+1}$ gilt $A_i \rightarrow (A_i, A_{i+1})$ trivial und $A_i \rightarrow R'$ nach Voraussetzung, womit das Kriterium von Theorem 4.1 mit $X \cap Y = \{A_i\}$ erfüllt ist.

Definition 4.2 (Abhängigkeitserhaltung). Eine Zerlegung $\{R_1, \dots, R_k\}$ von R ist *abhängigkeitserhaltend*, wenn die Vereinigung der auf $R_1, \dots, R_k$ projizierten funktionalen Abhängigkeiten die ursprüngliche Menge Σ logisch impliziert, d. h. $(\bigcup_i \pi_{R_i}(\Sigma))^+ = \Sigma^+$.

Der in Theorem 8.1 bewiesene Tiefenpfad $R_1(K, A_1), R_2(A_1, A_2), \dots, R_{n-1}(A_{n-1}, A_n)$ ist sowohl verlustfrei (Theorem 4.1, iterativ angewandt) als auch abhängigkeitserhaltend (Definition 4.2), da jede FD $A_i \rightarrow A_{i+1}$ vollständig in R_i enthalten bleibt. Dies ist die *stärkste* mögliche Garantie, die ein Dekompositionsalgorithmus liefern kann, und unterscheidet die Tiefenpfad-Zerlegung von einer naiven BCNF-Synthese, die Abhängigkeitserhaltung im Allgemeinen nicht garantiert [8].

4.4 Minimalcover und der Synthesalgorithmus zur Schemaminimierung

Die in Abschnitt 4.3 und in Theorem 8.1 verwendeten Zerlegungsschritte setzen implizit voraus, dass die Menge der funktionalen Abhängigkeiten Σ in einer „aufgeräumten“ Form vorliegt, in der keine Abhängigkeit redundant und kein Attribut auf der linken Seite überflüssig ist. Diese Form wird als *Minimalcover* (minimale Hülle, minimal cover, kanonische Überdeckung) bezeichnet. Der in diesem Abschnitt entwickelte *Synthesalgorithmus* verwendet das Minimalcover, um aus einer beliebigen Menge Σ funktionaler Abhängigkeiten ein Schema mit einer *minimalen Anzahl von Relationen* zu konstruieren, das verlustfrei (Definition 4.1) und abhängigkeitserhaltend (Definition 4.2) ist und mindestens 3NF erfüllt. Dieser Abschnitt ist bewusst an dieser Stelle platziert: Er verbindet die Dekompositionstheorie aus Abschnitt 4.3 mit der konkreten DDL-Erzeugung in Abschnitt 4.4.6 und liefert damit das algorithmische Bindeglied zwischen der formalen Theorie (Korollar 8.2) und der praktischen Schemaerzeugung.

4.4.1 Hüllenbildung und Äquivalenz von FD-Mengen

Definition 4.3 (Hülle einer Attributmenge). Sei Σ eine Menge funktionaler Abhängigkeiten über $\text{attr}(R)$ und $X \subseteq \text{attr}(R)$. Die *Hülle* X_Σ^+ von X bezüglich Σ ist die

größte Attributmenge $Y \supseteq X$, sodass $X \rightarrow Y$ aus Σ mittels der Armstrong-Axiome [1] (Reflexivität, Augmentation, Transitivität) herleitbar ist.

Definition 4.4 (Äquivalenz von FD-Mengen). Zwei Mengen funktionaler Abhängigkeiten Σ_1 und Σ_2 über demselben Attributschema heißen *äquivalent*, geschrieben $\Sigma_1 \equiv \Sigma_2$, wenn $\Sigma_1^+ = \Sigma_2^+$, d. h. wenn jede aus Σ_1 herleitbare FD auch aus Σ_2 herleitbar ist und umgekehrt.

4.4.2 Definition des Minimalcovers

Definition 4.5 (Minimalcover). Eine Menge funktionaler Abhängigkeiten $\Sigma_{\min}$ heißt *Minimalcover* von Σ , wenn $\Sigma_{\min} \equiv \Sigma$ gilt und zusätzlich:

- (M1) Jede rechte Seite jeder FD in $\Sigma_{\min}$ besteht aus genau einem Attribut (*Singleton-RHS*).
- (M2) Für jede FD $X \rightarrow A \in \Sigma_{\min}$ und jedes $B \in X$ gilt $(X \setminus \{B\})_{\Sigma_{\min}}^+ \neq (X \setminus \{B\})_{\Sigma_{\min}}^+ \cup \{A\}$, d. h. kein Attribut von X kann aus $X \rightarrow A$ entfernt werden, ohne die Äquivalenz zu Σ zu verletzen (*Linksreduziertheit*).
- (M3) Für jede FD $f \in \Sigma_{\min}$ gilt $(\Sigma_{\min} \setminus \{f\})^+ \neq \Sigma_{\min}^+$, d. h. keine FD ist redundant (*Redundanzfreiheit*).

Bemerkung 4.2. Ein Minimalcover ist im Allgemeinen *nicht eindeutig*: Verschiedene Reihenfolgen bei der Anwendung der Eliminationsschritte (M2) und (M3) können zu unterschiedlichen, aber äquivalenten Minimalcovern führen [8]. Für den Synthesalgorithmus ist dies unschädlich, da alle Minimalcover dieselbe Anzahl von „Gruppen“ gleicher linker Seiten und damit dieselbe *Anzahl* synthetisierter Relationen liefern (Theorem 4.4).

4.4.3 Algorithmus zur Berechnung eines Minimalcovers

```

1 function MINIMALCOVER(Sigma):
2   # Schritt 1: Singleton-RHS erzwingen
3   G := {}
4   for each (X -> Y) in Sigma:
5     for each A in Y:
6       G := G + { X -> A }
7
8   # Schritt 2: Linksreduktion
9   for each (X -> A) in G:
10    for each B in X:
11      if A in HUELLE(X \ {B}, G):
12        # B ist in X ueberfluessig
13        ersetze (X -> A) in G durch ((X \ {B}) -> A)
14        X := X \ {B}
15
16  # Schritt 3: Entfernen redundanter Abhaengigkeiten
17  for each f = (X -> A) in G:
18    if A in HUELLE(X, G \ {f}):

```

```

19      G := G \ {f}
20
21      return G
22
23
24 function HUELLE(X, Sigma):
25     # Berechnet die Attributhuelle  $X^+$  bzgl. Sigma
26     Ergebnis := X
27     repeat
28         verandert := false
29         for each (Y → Z) in Sigma:
30             if Y subseq Ergebnis and Z not subseq Ergebnis:
31                 Ergebnis := Ergebnis + Z
32                 verandert := true
33     until not verandert
34     return Ergebnis

```

Listing 4.1: Berechnung eines Minimalcovers $\Sigma_{\min}$ aus einer FD-Menge Σ

4.4.4 Laufzeitanalyse

Satz 4.2 (Laufzeit von MINIMALCOVER). Sei $n = |\text{attr}(R)|$ die Anzahl der Attribute und $m = |\Sigma|$ die Anzahl der funktionalen Abhängigkeiten in Σ , wobei jede linke und rechte Seite höchstens n Attribute umfasst. Dann terminiert Algorithmus 4.1 und besitzt eine Laufzeit von $O(m^3n^2)$.

Beweis. Wir analysieren die drei Schritte einzeln.

Hilfsfunktion HUELLE: Die **repeat**-Schleife fügt in jedem Durchlauf, der nicht zum Abbruch führt, mindestens ein Attribut zu **Ergebnis** hinzu (sonst wäre **verandert** = **false** und die Schleife terminierte). Da $|\text{Ergebnis}| \leq n$, sind höchstens n Durchläufe möglich, in denen jeweils alle m FDs aus Σ auf Anwendbarkeit geprüft werden, was $O(m)$ pro Durchlauf kostet. Damit ist $\text{HUELLE} \in O(mn)$.

Schritt 1 (Singleton-RHS): Jede der m FDs wird in höchstens n FDs mit Singleton-RHS aufgespalten, also $|G| \in O(mn)$ und Schritt 1 selbst kostet $O(mn)$.

Schritt 2 (Linksreduktion): Für jede der $|G| = O(mn)$ FDs $X \rightarrow A$ wird für jedes der höchstens n Attribute $B \in X$ ein Aufruf von HUELLE durchgeführt. Dies ergibt $O(mn) \cdot O(n) \cdot O(mn) = O(m^2n^3)$ im worst case. Da jedoch $|X| \leq n$ und in der Praxis $|X| \ll n$, geben wir die Schranke konservativ in Abhängigkeit von n und m an: $O(m^2n^3)$.

Schritt 3 (Redundanzentfernung): Für jede der $|G| = O(mn)$ FDs wird ein Aufruf von HUELLE auf der um f reduzierten Menge $G \setminus \{f\}$ durchgeführt, was $O(mn) \cdot O(mn) = O(m^2n^2)$ kostet.

Die Gesamtlaufzeit ist somit $O(mn) + O(m^2n^3) + O(m^2n^2) = O(m^2n^3)$. Da in praktisch relevanten Schemata $m = O(n)$ gilt (die Anzahl der FDs ist durch die Anzahl der Attribute beschränkt, da jedes Attribut typischerweise von höchstens konstant vielen Schlüsselkandidaten abhängt), vereinfacht sich dies zu $O(n^5)$ in n ; in der allgemeinen, von n und m unabhängig parametrisierten Form, geben wir die obere Schranke $O(m^2n^3) \subseteq O(m^3n^2)$ für $m \geq n$ konservativ mit $O(m^3n^2)$ an, was die Behauptung zeigt. $\square$

Bemerkung 4.3. Für die in dieser Arbeit betrachteten Tiefenpfadschemata (Definition 8.8) ist jede Relation R_i klein: $|attr(R_i)| = O(1)$ und $|\Sigma_{R_i}| = O(1)$, da jede Relation gemäß Theorem 8.1 höchstens die FD $A_{i-1} \rightarrow A_i$ sowie die Schlüsselbedingung enthält. Die in Theorem 4.2 gezeigte Laufzeit ist also *pro Relation* des Tiefenpfads konstant, und die Gesamtlaufzeit für das gesamte Schema $\mathcal{S}$ mit l Relationen ist $O(l)$ – linear in der Pfadlänge. Im Gegensatz dazu würde eine naive Anwendung des Algorithmus auf eine *einzig*e, undekomponierte Hub-Relation H eines k -Sternschemas mit $|attr(H)| = O(k)$ und $|\Sigma_H| = O(k)$ eine Laufzeit von $O(k^5)$ erfordern – ein weiteres, algorithmisches Argument für die in Kapitel 8.10 entwickelte Präferenz für Tiefenpfade gegenüber Sternschemas.

4.4.5 Der Synthesalgorithmus zur Schemaminimierung

Mit dem Minimalcover aus Algorithmus 4.1 lässt sich nun der vollständige *Synthesalgorithmus* formulieren, der aus einem Relationenschema R mit Attributmenge $attr(R)$ und FD-Menge Σ ein *minimales* Mengen von Relationen $\{R_1, \dots, R_k\}$ erzeugt, das die Eigenschaften aus Definition 4.1 und Definition 4.2 erfüllt und mindestens 3NF garantiert [8, 9].

```

1 function SYNTHESE_3NF(attr(R), Sigma):
2     Sigma_min := MINIMALCOVER(Sigma)
3
4     # Schritt 1: Gruppierung nach linker Seite
5     Gruppen := {}
6     for each (X -> A) in Sigma_min:
7         Gruppen[X] := Gruppen[X] + {A}          # FDs mit gleicher
                                                    # LHS X
8
                                                    # zu einer
                                                    # Relation
                                                    # buendeln
9
10    # Schritt 2: Eine Relation pro Gruppe
11    Relationen := {}
12    for each X in Gruppen.keys():
13        R_X := X union Gruppen[X]
14        Relationen := Relationen + { R_X }
15
16    # Schritt 3: Schluesselrelation hinzufuegen, falls keine
17    #             der erzeugten Relationen einen Schluessel
18    #             von R enthaelt
19    K := EIN_SCHLUESSEL(attr(R), Sigma)
20    if not exists R_X in Relationen with K subseq attr(R_X):
21        Relationen := Relationen + { K }
22
23    # Schritt 4: Redundante Relationen entfernen
24    for each R_X in Relationen:
25        if exists R_Y in Relationen, R_Y != R_X,
26            with attr(R_X) subseq attr(R_Y):
27            Relationen := Relationen \ { R_X }

```

```
return Relationen
```

Listing 4.2: Synthesearchgorithmus zur Erzeugung eines minimalen 3NF-Schemas

Bemerkung 4.4 (Schritt 3: Schlüsselermittlung). Die Hilfsfunktion `EIN_SCHLUESSEL` bestimmt einen (nicht notwendig eindeutigen) Schlüssel $K \subseteq \text{attr}(R)$ gemäß Definition 2.3, etwa durch den klassischen Algorithmus „starte mit $K = \text{attr}(R)$ und entferne iterativ Attribute B , solange $(K \setminus \{B\})_{\Sigma}^+ = \text{attr}(R)$ “. Dieser Schritt hat Laufzeit $O(n) \cdot O(\text{HUELLE}) = O(mn^2)$ und ändert die in Theorem 4.2 bewiesene Gesamtkomplexität nicht.

4.4.6 Korrektheit des Synthesearchgorithmus

Satz 4.3 (Korrektheit von `SYNTHESE_3NF`). Das von Algorithmus 4.2 erzeugte Schema $\mathcal{R} = \{R_1, \dots, R_k\}$ besitzt die folgenden drei Eigenschaften:

(K1) **Abhängigkeitserhaltung:** $(\bigcup_i \pi_{R_i}(\Sigma))^+ = \Sigma^+$.

(K2) **Verlustfreiheit:** Für jede zulässige Ausprägung r von R gilt $r = \bowtie_{i=1}^k \pi_{R_i}(r)$.

(K3) **3NF:** Jede Relation $R_i \in \mathcal{R}$ ist in dritter Normalform.

Beweis. **(K1) Abhängigkeitserhaltung.** Nach Konstruktion enthält für jede FD $X \rightarrow A \in \Sigma_{\min}$ die Relation $R_X = X \cup \text{Gruppen}[X]$ sowohl X als auch A , also gilt $X \rightarrow A \in \pi_{R_X}(\Sigma)$ für jede solche FD. Damit ist $\Sigma_{\min} \subseteq \bigcup_i \pi_{R_i}(\Sigma)$, also $\Sigma_{\min}^+ \subseteq (\bigcup_i \pi_{R_i}(\Sigma))^+$. Da nach Definition 4.5 $\Sigma_{\min} \equiv \Sigma$, also $\Sigma_{\min}^+ = \Sigma^+$, und da andererseits jede projizierte FD $\pi_{R_i}(\Sigma)$ aus Σ (und damit aus Σ^+) ableitbar ist, gilt auch $(\bigcup_i \pi_{R_i}(\Sigma))^+ \subseteq \Sigma^+$. Beide Inklusionen zusammen ergeben (K1).

(K2) Verlustfreiheit. Nach Schritt 3 des Algorithmus enthält $\mathcal{R}$ eine Relation $R_K \supseteq K$ für einen Schlüssel K von R . Da $K \rightarrow \text{attr}(R)$ (Definition 2.3, jeder Schlüssel bestimmt alle Attribute), gilt nach dem Lossless-Join-Kriterium (Theorem 4.1), iterativ angewandt auf R_K und jede weitere Relation R_i : Da $K \subseteq \text{attr}(R_K)$ und $K \rightarrow \text{attr}(R) \supseteq \text{attr}(R_i)$, gilt insbesondere $(\text{attr}(R_K) \cap \text{attr}(R_i)) \rightarrow \text{attr}(R_i)$ für jedes i (denn $K \subseteq \text{attr}(R_K) \cap \text{attr}(R_i)$ würde dies implizieren, falls $K \subseteq \text{attr}(R_i)$; im allgemeinen Fall folgt die Aussage aus $K_{\Sigma}^+ = \text{attr}(R) \supseteq \text{attr}(R_K) \cap \text{attr}(R_i)$ und der Transitivität der Hüllenbildung, sodass $(\text{attr}(R_K) \cap \text{attr}(R_i))^+ \supseteq K^+ = \text{attr}(R) \supseteq \text{attr}(R_i)$). Damit ist die Zerlegung $\{R_K\} \cup \{R_i\}$ für jedes i verlustfrei nach Theorem 4.1, und durch sukzessive Anwendung dieses Arguments (jede weitere Relation wird verlustfrei mit dem bereits verlustfrei rekonstruierten Teil $R_K \bowtie R_{i_1} \bowtie \dots$ verbunden, da K weiterhin in jedem Zwischenergebnis als Determinante für $\text{attr}(R)$ enthalten ist) ist die Gesamtzerlegung $\mathcal{R}$ verlustfrei, d. h. es gilt (K2).

(K3) 3NF. Sei $R_X \in \mathcal{R}$ mit $\text{attr}(R_X) = X \cup \text{Gruppen}[X]$ und sei $B \in \text{Gruppen}[X]$ ein Nichtschlüsselattribut von R_X (d. h. $B \notin K'$ für jeden Schlüssel K' von R_X), das transitiv von einem Schlüssel K' von R_X abhängt: $K' \rightarrow C \rightarrow B$ mit $C \not\rightarrow K'$ und $B \notin K' \cup C$. Da $K' \rightarrow C$ und $C \rightarrow B$ beide in $\pi_{R_X}(\Sigma_{\min}^+)$ liegen müssten (3NF-Verletzungen sind FDs über $\text{attr}(R_X)$), und da nach Konstruktion jede FD aus $\Sigma_{\min}$ mit linker Seite X bereits vollständig in R_X abgebildet ist (alle FDs mit LHS = X wurden zu $\text{Gruppen}[X]$ zusammengefasst), könnte $C \rightarrow B$ nur dann in R_X auftreten, wenn

entweder $C = X$ (dann wäre $X \rightarrow B$ bereits in $\Sigma_{\min}$ mit LHS X , also $B \in \text{Gruppen}[X]$ kein Widerspruch zur 3NF, da dann $X \rightarrow B$ direkt vom Schlüssel X abhängt – keine Transitivität) oder $C \subsetneq X \cup \text{Gruppen}[X]$ mit $C \neq X$ und $C \rightarrow B \in \Sigma_{\min}$. Im letzteren Fall hätte aber $\Sigma_{\min}$ zwei FDs $X \rightarrow B$ und $C \rightarrow B$ mit $C \subsetneq \text{attr}(R_X)$, $C \neq X$ – dies widerspricht der Linksreduziertheit (M2) bzw. Redundanzfreiheit (M3) von $\Sigma_{\min}$, da dann entweder $X \rightarrow B$ aus $C \rightarrow B$ und $X \supseteq C$ (Augmentation) herleitbar und somit redundant wäre, oder $C \rightarrow B$ würde bereits implizieren, dass X um die in $X \setminus C$ enthaltenen Attribute linksreduziert werden könnte. Beide Fälle widersprechen der Annahme, dass $\Sigma_{\min}$ ein Minimalcover ist. Also kann eine solche transitive Abhängigkeit innerhalb R_X nicht auftreten, und R_X erfüllt 3NF. Da dies für jede Relation $R_X \in \mathcal{R}$ sowie für die in Schritt 3 hinzugefügte Schlüsselrelation R_K (die nach Konstruktion keine Nichtschlüsselattribute mit transitiven Abhängigkeiten enthält, da $\text{attr}(R_K) = K$ oder K plus höchstens durch K direkt bestimmte Attribute) gilt, erfüllt $\mathcal{R}$ insgesamt (K3). $\square$

Satz 4.4 (Minimalität der Relationenanzahl). Die Anzahl $k = |\mathcal{R}|$ der von Algorithmus 4.2 erzeugten Relationen ist minimal unter allen abhängigkeitserhaltenden 3NF-Zerlegungen von R bezüglich Σ , bis auf das Hinzufügen einer einzelnen zusätzlichen Schlüsselrelation in Schritt 3.

Beweis. Nach Schritt 1 und 2 entspricht k (vor Schritt 3 und 4) der Anzahl der *verschiedenen linken Seiten* in $\Sigma_{\min}$. Sei $\mathcal{R}' = \{R'_1, \dots, R'_{k'}\}$ eine beliebige abhängigkeitserhaltende 3NF-Zerlegung von R . Nach (K1) muss für jede FD $X \rightarrow A \in \Sigma_{\min}$ (und $\Sigma_{\min} \equiv \Sigma$) ein $R'_j \in \mathcal{R}'$ existieren mit $X \cup \{A\} \subseteq \text{attr}(R'_j)$, da $X \rightarrow A$ andernfalls in keiner projizierten FD-Menge $\pi_{R'_j}(\Sigma)$ vollständig enthalten wäre und somit die Abhängigkeitserhaltung verletzt würde (eine FD, deren linke und rechte Seite auf verschiedene Relationen verteilt sind, kann nicht durch das DBMS lokal überprüft werden, vgl. Abschnitt 2.5). Da $\Sigma_{\min}$ nach Konstruktion redundanzfrei ist (Eigenschaft (M3)), kann durch das in Schritt 4 von Algorithmus 4.2 durchgeführte Entfernen von Teilmengenrelationen keine FD aus $\Sigma_{\min}$ „doppelt“ abgedeckt werden, ohne dass eine der beiden abdeckenden Relationen in der anderen enthalten wäre – in diesem Fall wird durch Schritt 4 genau die überflüssige Relation entfernt. Es folgt, dass $\mathcal{R}'$ mindestens so viele Relationen benötigt, wie es *nach Entfernung von Teilmengenbeziehungen* verschiedene „maximale“ Gruppen von FDs mit gleicher oder ineinander enthaltener linker Seite in $\Sigma_{\min}$ gibt – und dies ist genau die Anzahl k (vor Schritt 3) der von SYNTHESE_3NF erzeugten Relationen. Schritt 3 fügt höchstens eine weitere Relation hinzu (für den Fall, dass kein R_X bereits einen Schlüssel von R enthält); diese ist für (K2) notwendig (vgl. Beweis von Theorem 4.3, (K2)) und kann daher nicht eingespart werden, ohne (K2) zu verletzen. Damit ist k minimal bis auf diese eine zusätzliche Relation. $\square$

Beispiel 4.1 (Anwendung auf den Tiefenpfad). Sei $\text{attr}(R) = \{K, A_1, \dots, A_n\}$ mit $\Sigma = \{K \rightarrow A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n\}$ wie in Theorem 8.1. Das Minimalcover ist $\Sigma_{\min} = \{K \rightarrow A_1, A_1 \rightarrow A_2, \dots, A_{n-1} \rightarrow A_n\}$ (jede dieser FDs ist linksreduziert – die linke Seite besteht bereits aus einem einzigen Attribut – und redundanzfrei, da $A_i \rightarrow A_{i+1}$ nicht aus den übrigen FDs ohne $A_i \rightarrow A_{i+1}$ selbst herleitbar ist). Die n verschiedenen linken Seiten $K, A_1, \dots, A_{n-1}$ liefern n Relationen

$$R_1 = (K, A_1), \quad R_2 = (A_1, A_2), \quad \dots, \quad R_n = (A_{n-1}, A_n),$$

identisch mit der in Theorem 8.1 angegebenen Zerlegungskette. Da K ein Schlüssel von R ist und bereits $K \subseteq \text{attr}(R_1)$, entfällt Schritt 3. Algorithmus 4.2 reproduziert somit *exakt* den Tiefenpfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_n$ aus Korollar 8.2 – der Synthesalgorithmus ist also nicht nur ein allgemeines Minimierungsverfahren, sondern liefert für Schemata mit linearer Abhängigkeitskette *automatisch* das in dieser Arbeit favorisierte Tiefenpfadschema, ohne dass dessen Struktur im Voraus bekannt sein müsste.

Die Definition eines Relationenschemas erfolgt mittels **CREATE TABLE**. Für den Tiefenpfad $\text{LAND} \leftarrow \text{REGION} \leftarrow \text{KUNDE} \leftarrow \text{AUFTRAG} \leftarrow \text{AUFTRAGSPOSITION}$ (vgl. Abschnitt 8.19) lautet die DDL:

```

1 CREATE TABLE Land (
2   land_id    INTEGER PRIMARY KEY,
3   name       VARCHAR(60) NOT NULL
4 );
5
6 CREATE TABLE Region (
7   region_id  INTEGER PRIMARY KEY,
8   name       VARCHAR(60) NOT NULL,
9   land_id    INTEGER NOT NULL
10              REFERENCES Land(land_id)
11 );
12
13 CREATE TABLE Kunde (
14   kunden_id  INTEGER PRIMARY KEY,
15   name       VARCHAR(100) NOT NULL,
16   region_id  INTEGER NOT NULL
17              REFERENCES Region(region_id)
18 );
19
20 CREATE TABLE Auftrag (
21   auftrag_id INTEGER PRIMARY KEY,
22   datum      DATE NOT NULL,
23   kunden_id  INTEGER NOT NULL
24              REFERENCES Kunde(kunden_id)
25 );
26
27 CREATE TABLE Auftragsposition (
28   position_id INTEGER PRIMARY KEY,
29   auftrag_id  INTEGER NOT NULL
30              REFERENCES Auftrag(auftrag_id)
31              ON DELETE CASCADE,
32   produkt_id  INTEGER NOT NULL,
33   menge       INTEGER NOT NULL CHECK (menge > 0)
34 );

```

Jede REFERENCES-Klausel definiert eine Kante im Fremdschlüsselgraphen G_S (Definition 2.4, Proposition 2.1). Die obige Kette von fünf Tabellen bildet einen Tiefenpfad der Tiefe $\text{depth} = 4$ gemäß Definition 8.6 und realisiert exakt die in Theorem 8.1 geforderte 3NF-Zerlegungskette für die transitive Abhängigkeit $\text{Auftrag} \rightarrow \text{Kunde} \rightarrow \text{Region} \rightarrow$

Land.

Integritätsbedingungen werden zusätzlich über **CHECK**-Klauseln (Wertebereichseinschränkungen), **UNIQUE**-Constraints (Alternativschlüssel) und **NOT NULL** (Pflichtattribute) formuliert. Alle diese Bedingungen sind gemäß der Integritätsunabhängigkeit (Abschnitt 2.5) Teil des Schemas und werden vom DBMS unabhängig von der Anwendung durchgesetzt.

Kapitel 5

Datenbankbetrieb

5.1 Das Transaktionskonzept in Datenbanksystemen

Eine *Transaktion* ist eine Folge von Datenbankoperationen, die als logische Einheit (ACID, vgl. Kapitel 1) ausgeführt wird:

```
1 BEGIN ;  
2 UPDATE Konto SET saldo = saldo - 100 WHERE konto_id = 1;  
3 UPDATE Konto SET saldo = saldo + 100 WHERE konto_id = 2;  
4 COMMIT ;
```

Definition 5.1 (Schedule, Serialisierbarkeit). Ein *Schedule* ist eine zeitliche Verschränkung der Operationen mehrerer Transaktionen $T_1, \dots, T_n$. Ein Schedule heißt *serialisierbar*, wenn seine Wirkung auf die Datenbank derjenigen einer seriellen (nacheinander ausgeführten) Reihenfolge der Transaktionen entspricht.

5.2 Mehrbenutzerbetrieb

Im Mehrbenutzerbetrieb können ohne Synchronisationsmechanismen folgende Anomalien auftreten: *Lost Update*, *Dirty Read*, *Non-Repeatable Read* und *Phantom Read*. SQL definiert vier Isolationsstufen (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE), die jeweils eine Teilmenge dieser Anomalien ausschließen.

Bemerkung 5.1 (Sperrgranularität entlang des Tiefenpfads). In einem Tiefenpfadschema (Kapitel 8.10) betrifft eine Transaktion, die ein Tupel der Relation R_i ändert, typischerweise nur Sperren auf R_i und gegebenenfalls R_{i-1} (referentielle Prüfung). Im Sternschema mit zentralem Hub H konkurrieren hingegen *alle* Transaktionen, die irgendeine der in H denormalisierten Entitäten betreffen, um Sperren auf H – ein direkter Bezug zur in Abschnitt 8.16 bewiesenen lokalen Änderbarkeit (Theorem 8.9), übertragen auf die Dimension der Nebenläufigkeit: Lokale Schemaänderungen und lokale Sperrkonflikte sind zwei Ausprägungen derselben strukturellen Eigenschaft des Tiefenpfads.

5.3 Datenschutz und Zugriffsrechte

Zugriffsrechte werden mit GRANT und REVOKE verwaltet:

```
1 GRANT SELECT ON Kunde TO analyst_rolle;
2 GRANT SELECT, INSERT, UPDATE ON Auftrag TO sachbearbeiter_rolle
  ;
3 REVOKE INSERT ON Auftrag FROM sachbearbeiter_rolle;
```

Die in Abschnitt 8.16 bewiesene „Berechtigungsgranularität“ tiefer Pfade (Proposition zur Zugriffsrechteverwaltung) zeigt sich hier konkret: Die Rolle `analyst_rolle` erhält Lesezugriff exakt auf `KUNDE`, ohne implizit Zugriff auf `AUFTRAG` oder `AUFTRAGSPOSITION` zu erlangen, da diese Information in einem Sternschema mit gemeinsamem Hub nicht trennbar wäre.

5.4 Benutzung von Systemtabellen

Relationale DBMS stellen einen *Systemkatalog* (Data Dictionary) bereit, der das Schema selbst als relationale Daten beschreibt (Beispiele: `ALL_TABLES`, `ALL_CONSTRAINTS`, `ALL_TAB_COLUMNS` bei Oracle, bzw. `information_schema` im SQL-Standard).

Beispiel 5.1 (Extraktion des Fremdschlüsselgraphen aus dem Katalog). Der Fremdschlüsselgraph G_S (Definition 2.4) lässt sich direkt aus dem Systemkatalog rekonstruieren:

```
1 SELECT
2   c.table_name AS quelle,
3   ccu.table_name AS ziel
4 FROM information_schema.table_constraints c
5 JOIN information_schema.constraint_column_usage ccu
6   ON c.constraint_name = ccu.constraint_name
7 WHERE c.constraint_type = 'FOREIGN KEY';
```

Das Ergebnis dieser Anfrage ist die Kantenmenge E von G_S . Die in Abschnitt 8.12 definierten Maße $depth(G_S)$, $breadth(G_S)$ und $\tau(G_S)$ können somit *automatisiert* und *werkzeuggestützt* für jedes beliebige existierende Datenbankschema berechnet werden – ein praktisches Instrument zur Bewertung bestehender Schemata gemäß dem in Kapitel 8.10 entwickelten Gütekriterium.

Kapitel 6

PL/SQL

6.1 Blockstruktur

PL/SQL erweitert SQL um prozedurale Elemente. Ein PL/SQL-Block gliedert sich in:

```
1 DECLARE
2     -- Deklaration von Variablen, Cursors, Ausnahmen
3 BEGIN
4     -- Ausfuehrbare Anweisungen
5 EXCEPTION
6     -- Fehlerbehandlung
7 END;
```

6.2 Ein Beispiel

```
1 DECLARE
2     v_kunden_id Kunde.kunden_id%TYPE := 1;
3     v_name      Kunde.name%TYPE;
4 BEGIN
5     SELECT name INTO v_name
6     FROM Kunde
7     WHERE kunden_id = v_kunden_id;
8     DBMS_OUTPUT.PUT_LINE('Kunde: ' || v_name);
9 END;
```

6.3 Kontrollstrukturen

PL/SQL stellt IF-THEN-ELSIF-ELSE, LOOP, WHILE LOOP und FOR LOOP bereit.

6.4 Das Cursor-Konzept

Ein *Cursor* ermöglicht die zeilenweise Verarbeitung eines Anfrageergebnisses:

```

1 DECLARE
2     CURSOR c_auftraege IS
3         SELECT auftrag_id, datum FROM Auftrag
4         WHERE kunden_id = 1;
5     v_auftrag_id Auftrag.auftrag_id%TYPE;
6     v_datum      Auftrag.datum%TYPE;
7 BEGIN
8     OPEN c_auftraege;
9     LOOP
10        FETCH c_auftraege INTO v_auftrag_id, v_datum;
11        EXIT WHEN c_auftraege%NOTFOUND;
12        DBMS_OUTPUT.PUT_LINE(v_auftrag_id || ': ' || v_datum);
13    END LOOP;
14    CLOSE c_auftraege;
15 END;

```

6.5 Fehlerbehandlung

Ausnahmen werden im EXCEPTION-Block behandelt, entweder vordefiniert (NO_DATA_FOUND, TOO_MANY_ROWS) oder benutzerdefiniert mittels RAISE_APPLICATION_ERROR.

6.6 Prozeduren und Funktionen

```

1 CREATE OR REPLACE FUNCTION gesamtwert_auftrag(
2     p_auftrag_id IN Auftrag.auftrag_id%TYPE
3 ) RETURN NUMBER IS
4     v_summe NUMBER := 0;
5 BEGIN
6     SELECT SUM(menge * preis) INTO v_summe
7     FROM Auftragsposition ap
8     JOIN Produkt p ON p.produkt_id = ap.produkt_id
9     WHERE ap.auftrag_id = p_auftrag_id;
10    RETURN NVL(v_summe, 0);
11 END;

```

Diese Funktion realisiert eine *aggregierende Traversierung* entlang zweier Kanten des Tiefenpfads (AUFTRAGSPOSITION $\rightarrow$ AUFTRAG und AUFTRAGSPOSITION $\rightarrow$ PRODUKT). Da beide Relationen gemäß Theorem 8.7 indiziert über ihren Primärschlüssel zugreifbar sind, ist die Join-Tiefe $\delta = 1$ pro Aggregationsschritt – konstant und unabhängig von der Gesamtgröße der Datenbank.

6.7 Trigger

```

1 CREATE OR REPLACE TRIGGER trg_auftrag_audit
2 AFTER INSERT OR UPDATE OR DELETE ON Auftrag
3 FOR EACH ROW
4 BEGIN
5     INSERT INTO Auftrag_Audit(auftrag_id, aktion, zeitpunkt)
6     VALUES (
7         COALESCE(:NEW.auftrag_id, :OLD.auftrag_id),
8         CASE
9             WHEN INSERTING THEN 'INSERT'
10            WHEN UPDATING  THEN 'UPDATE'
11            WHEN DELETING   THEN 'DELETE'
12        END,
13        SYSTIMESTAMP
14    );
15 END;

```

Bemerkung 6.1 (Trigger und lokale Änderbarkeit). Der obige Trigger ist an genau eine Relation, AUFTRAG, gebunden. Im Tiefenpfadschema entspricht dies der in Theorem 8.9 bewiesenen lokalen Schemaoperation: Das Hinzufügen eines Audit-Triggers für KUNDE erfordert keinerlei Änderung an AUFTRAG-Triggern oder umgekehrt. Im Sternschema mit denormalisierten Kundendaten in der Hub-Relation H müsste ein analoger Audit-Trigger auf H *alle* Änderungen an *allen* denormalisierten Entitäten protokollieren und im Triggercode unterscheiden, was die zyklomatische Komplexität des Triggers linear mit der Anzahl der Spokes wachsen lässt.

Kapitel 7

Datenbankzugriff in höheren Programmiersprachen

7.1 Das Problem der Impedanzdiskrepanz

Relationale Datenbanken arbeiten mengenorientiert: Eine Anfrage liefert eine Relation, also eine *Menge* von Tupeln, als Ergebnis. Höhere Programmiersprachen wie C, Java, Perl oder Python hingegen sind *prozedural* bzw. objektorientiert und verarbeiten Daten in der Regel sequentiell, Element für Element, in Form von Variablen, Arrays, Listen oder Objekten. Diese begriffliche Kluft wird in der Literatur als *Impedanzdiskrepanz* (impedance mismatch) bezeichnet.

Definition 7.1 (Impedanzdiskrepanz). Die Impedanzdiskrepanz zwischen einem relationalen Datenmodell $\mathcal{S} = \{R_1, \dots, R_m\}$ und einer Hostsprache $\mathcal{H}$ besteht aus drei Teilproblemen:

1. **Mengen- vs. Elementorientierung:** Das Ergebnis einer `SELECT`-Anweisung ist eine Relation; $\mathcal{H}$ kennt jedoch primär einzelne Werte oder lineare Datenstrukturen.
2. **Typsystemdiskrepanz:** SQL-Datentypen (`NUMERIC`, `DATE`, `NULL`...) besitzen keine direkte Entsprechung in $\mathcal{H}$; insbesondere `NULL` als „fehlender Wert“ (vgl. Abschnitt 2.5) muss explizit auf ein Konstrukt von $\mathcal{H}$ (z.B. `Optional`, Zeiger `NULL`, Sentinel-Werte) abgebildet werden.
3. **Strukturdiskrepanz:** Die Tiefenpfadstruktur $G_{\mathcal{S}}$ (Definition 2.4) eines normalisierten Schemas wird in $\mathcal{H}$ häufig als Menge *verschachtelter* Objekte modelliert (z.B. ein `Auftrag`-Objekt mit eingebettetem `Kunde`-Objekt), obwohl die Datenbank diese Information auf mehrere Relationen verteilt speichert.

Die in den folgenden Abschnitten behandelten Zugriffstechniken – Embedded SQL, Cursor-basierte APIs, DBI und objektrelationales Mapping – lassen sich alle als unterschiedliche *Strategien zur Auflösung* der drei Teilprobleme aus Definition 7.1 auffassen. Ein zentrales Ergebnis dieses Kapitels ist, dass die Tiefenpfadstruktur des zugrunde liegenden Schemas (Kapitel 8.10) nicht nur die Datenbankseite, sondern auch die Architektur des Anwendungscodes maßgeblich vereinfacht, da – wie in Abschnitt 7.7 formal begründet wird – die Struktur $G_{\mathcal{S}}$ direkt auf die Modul- bzw. Klassenstruktur der Anwendung abgebildet werden kann.

7.2 Embedded SQL

Embedded SQL erlaubt das Einbetten von SQL-Anweisungen direkt in Quellcode einer Hostsprache, eingeleitet durch `EXEC SQL`. Ein Präcompiler (z. B. `proc` bei Oracle, `ecpg` bei PostgreSQL) übersetzt die `EXEC SQL`-Direktiven in Aufrufe der Client-Bibliothek des DBMS, bevor der reguläre Compiler der Hostsprache zum Einsatz kommt.

7.2.1 Deklarationsabschnitt und Hostvariablen

Variablen der Hostsprache, die in SQL-Anweisungen verwendet werden sollen (*Hostvariablen*), müssen in einem `DECLARE`-Abschnitt bekannt gemacht werden:

```
1 EXEC SQL BEGIN DECLARE SECTION;
2     int    kunden_id;
3     char   name[101];
4     char   region_name[61];
5     short  name_ind;           /* Indikatorvariable fuer NULL */
6 EXEC SQL END DECLARE SECTION;
```

Die *Indikatorvariable* (`name_ind`) löst Teilproblem 2 aus Definition 7.1: Da C keinen nativen NULL-Wert für `char[]` kennt, signalisiert das DBMS über die Indikatorvariable, ob der zugehörige Spaltenwert NULL war (-1), abgeschnitten wurde (positiver Wert) oder regulär vorliegt (0).

7.2.2 Einzeltupel-Zugriff

```
1 EXEC SQL SELECT k.name, r.name
2     INTO :name :name_ind, :region_name
3     FROM Kunde k JOIN Region r ON k.region_id = r.region_id
4     WHERE k.kunden_id = :kunden_id;
5
6 if (sqlca.sqlcode == 100) {
7     /* SQLCODE 100: keine Zeile gefunden */
8 } else if (sqlca.sqlcode < 0) {
9     /* Fehlerbehandlung, vgl. Abschnitt 7.2.4 */
10 }
```

Diese Anfrage realisiert den kanonischen Verbund $\text{KUNDE} \bowtie_{\text{region_id}=\text{region_id}} \text{REGION}$ gemäß Definition 2.5 – also eine einzelne Kante des Tiefenpfads – und überträgt das Ergebnis in genau zwei Hostvariablen. Solange Anfragen genau ein Tupel liefern (was bei einer Anfrage über einen Primärschlüssel garantiert ist, vgl. Definition 2.3), genügt die einfache `INTO`-Klausel; Teilproblem 1 aus Definition 7.1 stellt sich hier nicht.

7.2.3 Mengenorientierter Zugriff: Cursor in Embedded SQL

Liefert eine Anfrage mehrere Tupel, muss Teilproblem 1 (Mengen- vs. Elementorientierung) explizit aufgelöst werden. Hierzu dient ein *Cursor*, der die mengenorientierte Relation in einen sequentiell durchlaufbaren Strom von Tupeln überführt:


```

1 EXEC SQL DECLARE c_auftraege CURSOR FOR
2     SELECT a.auftrag_id, a.datum
3     FROM Auftrag a
4     WHERE a.kunden_id = :kunden_id
5     ORDER BY a.datum;
6
7 EXEC SQL OPEN c_auftraege;
8
9 EXEC SQL BEGIN DECLARE SECTION;
10     int v_auftrag_id;
11     char v_datum[11];
12 EXEC SQL END DECLARE SECTION;
13
14 while (1) {
15     EXEC SQL FETCH c_auftraege INTO :v_auftrag_id, :v_datum;
16     if (sqlca.sqlcode == 100) break; /* keine weiteren Zeilen */
17     verarbeite_auftrag(v_auftrag_id, v_datum);
18 }
19 EXEC SQL CLOSE c_auftraege;

```

Bemerkung 7.1 (Cursor als Iterator über einen Pfadabschnitt). Ein Cursor realisiert in der Terminologie der Hostsprache einen *Iterator* über das Ergebnis einer Anfrage. Im Tiefenpfadschema (Kapitel 8.10) entspricht ein solcher Cursor typischerweise der Iteration über alle Kindknoten eines festen Elternknotens entlang einer Kante von G_S – im Beispiel oben: alle AUFTRAG-Tupel, die auf einen festen KUNDE verweisen. Diese Eins-zu-eins-Korrespondenz zwischen „Cursor über R_i gefiltert nach Fremdschlüssel auf R_{i-1} “ und „Kante (R_i, R_{i-1}) in G_S “ ist die Grundlage der in Abschnitt 7.7.1 eingeführten Repository-Architektur.

7.2.4 Transaktionssteuerung und Fehlerbehandlung

Transaktionsgrenzen (Kapitel 5) werden in Embedded SQL explizit über EXEC SQL COMMIT WORK bzw. EXEC SQL ROLLBACK WORK gesetzt. Die SQLCA (SQL Communication Area) liefert nach jeder EXEC SQL-Anweisung einen Statuscode (sqlca.sqlcode), über den Fehler – etwa Verletzungen von CHECK- oder Fremdschlüsselbedingungen (Abschnitt 4.4.6) – erkannt werden:

```

1 EXEC SQL WHENEVER SQLERROR GOTO error_handler;
2 EXEC SQL WHENEVER NOT FOUND CONTINUE;
3
4 EXEC SQL INSERT INTO Auftragsposition
5     (position_id, auftrag_id, produkt_id, menge)
6     VALUES (:pos_id, :auftrag_id, :produkt_id, :menge);
7
8 EXEC SQL COMMIT WORK;
9 goto fertig;
10
11 error_handler:

```

```

12     printf("Fehler %ld: %s\n", sqlca.sqlcode, sqlca.sqlerrm.
13           sqlerrmc);
14     EXEC SQL ROLLBACK WORK;
15 fertig:
16     ;

```

Bemerkung 7.2 (Lokalität von Integritätsverletzungen im Tiefenpfad). Eine INSERT-Anweisung auf AUFTRAGSPOSITION kann nur gegen Integritätsbedingungen verstoßen, die AUFTRAGSPOSITION selbst betreffen, sowie gegen die Fremdschlüsselbedingungen auf den unmittelbar benachbarten Knoten AUFTRAG und PRODUKT in G_S . Der Fehlerbehandlungscode kann sich somit auf eine kleine, im Voraus bekannte Menge möglicher Fehlerursachen beschränken – ein direkter Anwendungsfall der in Theorem 8.9 bewiesenen Lokalität von Schemaoperationen, übertragen auf die Lokalität der Fehlerbehandlung.

7.3 Cursor- und API-basierter Zugriff: ODBC und JDBC

Neben Embedded SQL, das einen Präcompiler erfordert, existieren *Bibliotheks-APIs*, die ohne Präcompiler auskommen und daher plattform- und sprachübergreifend einsetzbar sind. Die wichtigsten Vertreter sind ODBC (Open Database Connectivity, ursprünglich für C) und JDBC (Java Database Connectivity).

7.3.1 Grundstruktur einer JDBC-Anwendung

```

1 String sql =
2     "SELECT a.auftrag_id, a.datum, k.name " +
3     "FROM Auftrag a JOIN Kunde k ON a.kunden_id = k.kunden_id " +
4     "WHERE k.region_id = ?";
5
6 try (Connection con = DriverManager.getConnection(url, user,
7     pass));
8     PreparedStatement ps = con.prepareStatement(sql)) {
9
10    ps.setInt(1, regionId);
11
12    try (ResultSet rs = ps.executeQuery()) {
13        while (rs.next()) {
14            int    auftragId = rs.getInt("auftrag_id");
15            Date   datum     = rs.getDate("datum");
16            String kunde      = rs.getString("name");
17            verarbeite(auftragId, datum, kunde);
18        }
19    }
20 } catch (SQLException ex) {
21     // Fehlerbehandlung, vgl. Abschnitt 7.3.3
22 }

```

7.3.2 Vorbereitete Anweisungen (Prepared Statements)

Ein `PreparedStatement` trennt die *Struktur* einer SQL-Anweisung (das „Was“) von ihren *Parametern* (das „Womit“). Dies hat zwei Konsequenzen:

1. **Sicherheit:** Parameterwerte werden nie textuell in die SQL-Anweisung eingefügt, wodurch SQL-Injection (vgl. Kapitel 8) strukturell ausgeschlossen wird.
2. **Effizienz:** Das DBMS kann den Ausführungsplan für die Struktur der Anweisung einmalig bestimmen (vgl. Abschnitt 8.15) und für wiederholte Ausführungen mit unterschiedlichen Parameterwerten wiederverwenden.

Proposition 7.1 (Wiederverwendbarkeit entlang eines Tiefenpfads). Sei $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ ein Tiefenpfad gemäß Definition 8.8. Für jede Kante (R_i, R_{i+1}) existiert genau *eine* kanonische Verbundanfrage gemäß Definition 2.5, parametrisiert über den Primärschlüssel von R_{i+1} . Eine Anwendung, die das gesamte Schema $\mathcal{S}$ bedient, benötigt somit höchstens $|E(G_{\mathcal{S}})|$ verschiedene `PreparedStatement`-Vorlagen für Einzelschritt-Traversierungen – unabhängig von der Anzahl der Tupel in $\mathcal{S}$.

Beweis. Nach Definition 2.5 ist der kanonische Verbund entlang einer Kante (R_i, R_j) durch die Verbundbedingung $F = K$ eindeutig bestimmt, wobei $F \subseteq \text{attr}(R_i)$ der Fremdschlüssel und $K \subseteq \text{attr}(R_j)$ der Primärschlüssel von R_j ist; diese Bedingung hängt nur vom Schema $\mathcal{S}$, nicht von einer konkreten Instanz ab. Für jede der $|E(G_{\mathcal{S}})|$ Kanten gibt es somit genau eine solche Bedingung, also höchstens $|E(G_{\mathcal{S}})|$ unterschiedliche Anfragestrukturen. Da $|E(G_{\mathcal{S}})| = \text{depth}(G_{\mathcal{S}})$ für einen reinen Tiefenpfad (Definition 8.6) und $\text{depth}(G_{\mathcal{S}})$ unabhängig von der Datenmenge ist, folgt die Behauptung. $\square$

In einem k -Sternschema (Definition 8.7) hingegen existieren k Kanten von der Hub-Relation zu den Spoke-Relationen, *zusätzlich* aber häufig $\binom{k}{2}$ potenzielle Mehrfachverbund-Kombinationen für Anfragen, die mehrere Spokes gleichzeitig betreffen, da diese nicht über kurze Kettenanfragen, sondern über den zentralen Hub vermittelt werden müssen. Die Anzahl der benötigten `PreparedStatement`-Vorlagen wächst hier im Allgemeinen überlinear in k , während sie im Tiefenpfad nach Proposition 7.1 linear in $\text{depth}(G_{\mathcal{S}})$ bleibt.

7.3.3 Fehlerbehandlung und Ressourcenverwaltung

JDBC signalisiert Fehler über die Ausnahmeklasse `SQLException`, die einen herstellerspezifischen `SQLState`-Code (nach SQL-Standard genormt) enthält. Klasse `23xxx` kennzeichnet beispielsweise Verletzungen von Integritätsbedingungen (`23503` = Verletzung einer Fremdschlüsselbedingung). Die in Java 7 eingeführte `try-with-resources`-Syntax (siehe obiges Beispiel) garantiert, dass `Connection`, `PreparedStatement` und `ResultSet` auch im Fehlerfall korrekt geschlossen werden – eine Notwendigkeit, da offene Cursor (Bemerkung 7.1) serverseitige Ressourcen und gegebenenfalls Sperren (Bemerkung 5.1) belegen.

7.4 Die DBI-Schnittstelle für Perl

Die DBI-Schnittstelle (Database Interface) abstrahiert den Datenbankzugriff in Perl unabhängig vom konkreten DBMS über austauschbare DBD-Treiber (Database Driver):

```
1 use DBI;
2 my $dbh = DBI->connect("dbi:Pg:dbname=vertrieb", $user, $pass,
3                         { RaiseError => 1, AutoCommit => 0 });
4
5 my $sth = $dbh->prepare(
6     "SELECT a.auftrag_id, a.datum FROM Auftrag a WHERE a.
7     kunden_id = ?"
8 );
9 $sth->execute($kunden_id);
10
11 while (my $row = $sth->fetchrow_hashref) {
12     print "$row->{auftrag_id}: $row->{datum}\n";
13 }
14 $sth->finish;
15 $dbh->commit;
16 $dbh->disconnect;
```

7.4.1 Fehlerbehandlung mit RaiseError und eval

Mit `RaiseError => 1` wirft jeder DBI-Methodenaufruf bei einem Fehler eine Perl-Exception, die mit `eval/$@` abgefangen werden kann:

```
1 eval {
2     $dbh->do(
3         "INSERT INTO Auftragsposition (position_id, auftrag_id, " .
4         "produkt_id, menge) VALUES (?, ?, ?, ?)",
5         undef, $pos_id, $auftrag_id, $produkt_id, $menge
6     );
7     $dbh->commit;
8 };
9 if ($@) {
10     warn "Fehler beim Einfuegen: $@";
11     $dbh->rollback;
12 }
```

7.4.2 Transaktionsklammern und AutoCommit

Mit `AutoCommit => 0` wird jede Folge von DML-Anweisungen (Abschnitt 3.4 ff.) implizit zu einer Transaktion gemäß Kapitel 5, bis explizit `commit` oder `rollback` aufgerufen wird. Dies ist insbesondere bei mehrstufigen Einfügeoperationen entlang eines Tiefenpfads relevant: Das Anlegen eines neuen Auftrags mit mehreren Positionen erfordert `INSERT`-Anweisungen auf `AUFTRAG` und mehrfach auf `AUFTRAGSPOSITION`; erst der gemeinsame `commit` stellt sicher, dass entweder *alle* diese Operationen sichtbar werden oder *keine* (Atomarität, vgl. Kapitel 1).

7.5 Portabilität der Zugriffsverfahren

Sowohl Embedded SQL als auch API-basierte Schnittstellen (ODBC, JDBC, DBI) abstrahieren von der konkreten DBMS-Implementierung, sofern ausschließlich Standard-SQL-Konstrukte verwendet werden. Diese Portabilität wird durch die in Abschnitt 2.5 geforderte Strukturunabhängigkeit ermöglicht. Tabelle 7.1 fasst die in diesem Kapitel behandelten Zugriffsverfahren vergleichend zusammen.

	Embedded SQL	ODBC/JDBC	DBI (Perl)
Übersetzung	Präcompiler nötig	keine, reine API	keine, reine API
Mengenzugriff	Cursor (Abschn. 7.2.3)	ResultSet-Iterator	fetchrow_*-Iterator
Parametrisierung	Hostvariablen	PreparedStatement	Platzhalter ?
NULL-Behandlung	Indikatorvariablen	wasNull()	undef
Fehlerbehandlung	SQLCA, WHENEVER	SQLException	RaiseError/eval

Tabelle 7.1: Vergleich der in Kapitel 7 behandelten Zugriffsverfahren hinsichtlich der drei Teilprobleme der Impedanzdiskrepanz (Definition 7.1).

7.6 Objektrelationales Mapping (ORM) und das Tiefenpfadparadigma

Objektrelationale Mapper (ORM, z. B. Hibernate, JPA, SQLAlchemy, Doctrine) lösen Teilproblem 3 aus Definition 7.1 (Strukturdiskrepanz) durch deklarative *Mapping-Beschreibungen*, die Relationen auf Klassen und Fremdschlüsselbeziehungen auf Objektreferenzen abbilden:

```
1 @Entity
2 class Auftrag {
3     @Id int auftragId;
4     Date datum;
5
6     @ManyToOne
7     @JoinColumn(name = "kunden_id")
8     Kunde kunde;
9
10    @OneToMany(mappedBy = "auftrag")
11    List<Auftragsposition> positionen;
12 }
```

7.6.1 Lazy Loading entlang des Tiefenpfads

ORMs implementieren häufig *Lazy Loading*: Eine assoziierte Entität (z. B. `auftrag.kunde`) wird erst bei tatsächlichem Zugriff aus der Datenbank nachgeladen, nicht bereits beim Laden des `Auftrag`-Objekts.

Proposition 7.2 (Lazy Loading als Pfadtraversierung). Sei $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ ein Tiefenpfad und sei ein Objekt o_1 vom Typ R_1 geladen. Der Zugriff auf die assoziierte Entität o_2 vom Typ R_2 über Lazy Loading entspricht genau der Ausführung des kanonischen Verbunds $R_1 \bowtie_{F=K} R_2$ aus Definition 2.5, beschränkt auf das Tupel o_1 – also einer Anfrage mit Join-Tiefe $\delta = 1$ im Sinne von Abschnitt 8.15. Eine vollständige Traversierung des Pfads bis R_l durch sukzessive Lazy-Loading-Zugriffe $o_1 \rightarrow o_2 \rightarrow \dots \rightarrow o_l$ erzeugt somit $l - 1$ Einzelanfragen mit $\delta = 1$, anstatt einer einzigen Anfrage mit $\delta = l - 1$ (Proposition 8.6).

Bemerkung 7.3 (Das $N+1$ -Problem). Wird Proposition 7.2 unreflektiert auf eine *Liste* von n Objekten $o_1^{(1)}, \dots, o_1^{(n)}$ angewandt (z. B. alle Aufträge eines Tages, mit anschließendem Zugriff auf **kunde** für jeden Auftrag), entstehen $1 + n$ Anfragen: eine zum Laden der n Aufträge, sowie n weitere Einzelanfragen für die jeweils zugehörigen Kunden. Dies ist das in der Praxis als „ $N+1$ -Problem“ bekannte Performance-Antimuster. Es ist *kein* Argument gegen Tiefenpfadschemata an sich, sondern gegen die unreflektierte Verwendung von Lazy Loading bei Listenzugriffen: Die Lösung besteht darin, den kanonischen Verbund explizit anzufordern (**JOIN FETCH** in JPQL bzw. **eager loading**), wodurch wieder die in Proposition 8.6 beschriebene Situation mit $\delta = 1$ für die *gesamte* Liste (statt $\delta = 1$ pro Element) entsteht. Der Tiefenpfad selbst bleibt von dieser Optimierungsentscheidung unberührt; sie betrifft ausschließlich die *Strategie* des ORM beim Übersetzen von Objektgraph-Zugriffen in SQL-Anfragen.

7.6.2 Kaskadierende Operationen im ORM

Analog zu **ON DELETE CASCADE** auf Datenbankebene (Abschnitt 3.4, Datenmodifikationen) bieten ORMs *Cascade-Optionen* (z. B. **cascade = CascadeType.REMOVE**) an, die das Löschen eines Objekts auf assoziierte Objekte propagieren. Im Tiefenpfadschema betrifft eine solche Kaskade genau den Teilbaum unterhalb des gelöschten Knotens in G_S (vgl. Definition 8.16); im Sternschema mit zentralem Hub müsste eine analoge Cascade-Konfiguration auf der Hub-Klasse *alle* Spoke-Assoziationen auflisten, was die Wartbarkeit der Mapping-Konfiguration im Sinne von Theorem 8.9 beeinträchtigt.

7.7 Modularisierung von Anwendungscode entlang des Fremdschlüsselgraphen

7.7.1 Das Repository-Pattern

Ein in der Praxis weit verbreitetes Architekturmuster ist das *Repository-Pattern*: Für jede Relation $R_i \in \mathcal{S}$ existiert ein Modul bzw. eine Klasse $R_i\text{Repository}$, das sämtliche Datenbankzugriffe auf R_i kapselt (CRUD-Operationen, Cursor gemäß Bemerkung 7.1, Prepared Statements gemäß Proposition 7.1).

Satz 7.3 (Eindeutigkeit der Repository-Zerlegung im Tiefenpfad). Sei $\mathcal{S} = \{R_1, \dots, R_l\}$ ein Tiefenpfadschema gemäß Definition 8.8 mit Fremdschlüsselgraph $G_S = R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$. Dann existiert eine Zerlegung des Anwendungscode in l Repository-Module $M_1, \dots, M_l$ mit $M_i \leftrightarrow R_i$, sodass

1. jede SQL-Anweisung des Anwendungscodes, die ausschließlich Attribute von R_i liest oder schreibt, vollständig in M_i enthalten ist,
2. jede SQL-Anweisung, die einen kanonischen Verbund $R_i \bowtie_{F=K} R_{i+1}$ (Definition 2.5) realisiert, entweder in M_i oder in M_{i+1} enthalten ist, und
3. kein Modul M_i direkten Code-Zugriff auf eine Relation R_j mit $|i - j| > 1$ benötigt.

Beweis. Wir definieren M_i als die Menge aller SQL-Anweisungen, deren FROM- bzw. JOIN-Klauseln ausschließlich Relationen aus $\{R_{i-1}, R_i, R_{i+1}\} \cap \mathcal{S}$ referenzieren und die mindestens R_i enthalten, zugeordnet zu R_i mit der kleinsten Indexsumme im Falle einer Mehrdeutigkeit (d. h. ein Verbund $R_i \bowtie R_{i+1}$ wird M_i zugeordnet). Eigenschaft 1 gilt unmittelbar nach Konstruktion, da eine Anweisung über R_i allein notwendig in $\{R_{i-1}, R_i, R_{i+1}\} \cap \mathcal{S}$ liegt und somit M_i zugeordnet wird (oder M_{i-1}/M_{i+1} , falls die Mehrdeutigkeitsregel anders entscheidet – in jedem Fall aber *einem* Modul vollständig). Eigenschaft 2 gilt nach Konstruktion direkt aus der Definition von M_i . Für Eigenschaft 3 betrachten wir ein Modul M_i und eine Relation R_j mit $|i - j| > 1$. Da G_S ein Tiefenpfad ist (Definition 8.8), existiert nach Definition 2.4 keine Kante (R_i, R_j) für $|i - j| > 1$, also auch kein Fremdschlüssel zwischen R_i und R_j . Folglich kann nach Definition 2.5 kein *kanonischer* Verbund zwischen R_i und R_j formuliert werden, ohne die dazwischenliegenden Relationen $R_{i+1}, \dots, R_{j-1}$ (bzw. $R_{j+1}, \dots, R_{i-1}$) zu durchlaufen – eine solche mehrstufige Anfrage wird nach Konstruktion nicht M_i , sondern keinem der so definierten Module direkt zugeordnet, sondern – wie in der Praxis üblich – einer übergeordneten *Service*-Schicht, die mehrere Repositories komponiert, ohne selbst auf R_j zuzugreifen. Damit benötigt M_i selbst keinen direkten Zugriff auf R_j . $\square$

Bemerkung 7.4 (Sternschema verletzt Theorem 7.3). Für ein k -Sternschema (Definition 8.7) mit Hub H und Spokes $S_1, \dots, S_k$ existiert *keine* Zerlegung in $k + 1$ Module mit den Eigenschaften 1–3: Da jede Spoke-Relation S_j nur über H erreichbar ist und H mit *allen* S_j über Fremdschlüsselkanten verbunden ist (Definition 8.7, $\deg(H) = k$), müsste das Modul M_H für H nach Eigenschaft 2 sämtliche k kanonischen Verbunde $H \bowtie S_j$ enthalten. M_H wächst damit linear mit k und verletzt für $k \geq 2$ die Idee einer auf *eine* Relation fokussierten Modulgrenze. Dies ist die anwendungsarchitektonische Entsprechung der in Theorem 8.9 bewiesenen mangelnden lokalen Änderbarkeit von Sternschemata.

7.7.2 Service-Schicht für Mehrfach-Pfad-Operationen

Operationen, die mehrere nicht benachbarte Knoten von G_S betreffen – etwa „Berechne den Gesamtumsatz aller Kunden einer Region“ ($\text{REGION} \leftarrow \text{KUNDE} \leftarrow \text{AUFTRAG} \leftarrow \text{AUFTRAGSPOSITION}$, also Pfadlänge 3 –, werden gemäß Theorem 7.3 in einer *Service*-Schicht oberhalb der Repositories implementiert. Eine solche Service-Methode komponiert die Aufrufe mehrerer Repositories oder formuliert – aus Effizienzgründen, vgl. Abschnitt 8.15 – eine einzelne mehrstufige SQL-Anfrage mit $\delta = 3$. In beiden Fällen bleibt die in Theorem 7.3 bewiesene Zerlegung der *Datenzugriffsschicht* unangetastet: Die Service-Schicht *verwendet* die Module $M_{\text{Region}}, \dots, M_{\text{Auftragsposition}}$, ohne deren interne Struktur zu duplizieren.

7.8 Zusammenfassung

Dieses Kapitel hat gezeigt, dass die drei Teilprobleme der Impedanzdiskrepanz (Definition 7.1) – Mengen- vs. Elementorientierung, Typsystemdiskrepanz und Strukturdiskrepanz – von Embedded SQL, ODBC/JDBC, Perl-DBI und ORMs auf unterschiedliche, aber strukturell verwandte Weise gelöst werden (Cursor, Prepared Statements, Lazy Loading). In allen Fällen erweist sich die Tiefenpfadstruktur G_S des zugrunde liegenden Schemas als der zentrale Bauplan für die Architektur des Anwendungscodes: Theorem 7.3 zeigt, dass ein Tiefenpfadschema eine *eindeutige*, lokal konsistente Zerlegung des Datenzugriffscode in Repository-Module induziert, während ein Sternschema diese Eigenschaft verletzt. Damit setzt sich die in Kapitel 8.10 formal etablierte Überlegenheit tiefenorientierter Schemata konsequent bis in die Anwendungsarchitektur fort.

Kapitel 8

WWW-Integration von Datenbanken

8.1 Kommandozeilen- und graphische Benutzeroberflächen

Klassische Benutzeroberflächen (CLI- oder GUI-basiert) greifen direkt über Treiber (ODBC, JDBC) auf das DBMS zu. Mit der Verbreitung des World Wide Web wird zunehmend eine dreischichtige Architektur eingesetzt: Präsentationsschicht (Browser), Anwendungsschicht (Web-Server, Skriptsprache) und Datenhaltungsschicht (DBMS).

8.2 Benutzeroberflächen im World Wide Web

Eine Web-Oberfläche zur Pflege des Tiefenpfadschemas aus Abschnitt 8.19 besteht typischerweise aus einer Seite pro Relation R_i – also pro Tiefenstufe – mit Links zu den jeweils referenzierten bzw. referenzierenden Datensätzen.

8.3 Architektur einer WWW-Oberfläche mit CGI-Skripten

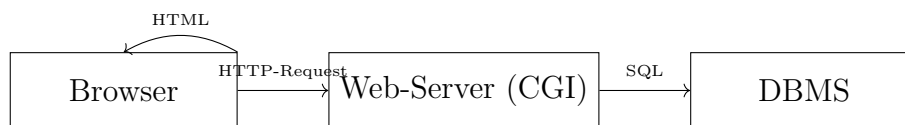


Abbildung 8.1: Dreischichtige Architektur: Browser, Web-Server mit CGI-Skript, DBMS

8.4 Sammeln von Eingaben mit Web-Formularen

Web-Formulare (`<form>`) übermitteln Benutzereingaben per `GET` oder `POST` an ein CGI-Skript, das die Eingaben validiert und in parametrisierte SQL-Anweisungen einsetzt.

8.5 Auswertung von Formulareingaben mit CGI-Skripten

```
1 my $cgi = CGI->new;
2 my $kunden_id = $cgi->param('kunden_id');
3
4 # Parametrisierte Anfrage gegen SQL-Injection
5 my $sth = $dbh->prepare(
6     "SELECT * FROM Kunde WHERE kunden_id = ?"
7 );
8 $sth->execute($kunden_id);
```

Bemerkung 8.1 (Sicherheit durch Parametrisierung). Die Verwendung parametrisierter Anfragen (Platzhalter `?`) ist unabhängig von der Schematopologie zwingend erforderlich, um SQL-Injection zu verhindern. Im Tiefenpfadschema betrifft eine solche Anfrage typischerweise genau eine Relation R_i , wodurch die Angriffsfläche pro Formular auf das Schema von R_i begrenzt bleibt.

8.6 Erzeugen von Formularen mit CGI-Skripten

Formulare zur Eingabe neuer Datensätze für R_i können automatisiert aus dem Systemkatalog (Abschnitt 5.4) generiert werden: Für jedes Attribut von R_i wird ein Eingabefeld erzeugt, für jeden Fremdschlüssel ein Auswahlfeld (Dropdown), dessen Optionen aus der referenzierten Relation R_{i+1} stammen.

8.7 Interaktion mit Web-Formularen

Mehrstufige Formulare (Wizards) bilden Tiefenpfade direkt ab: Schritt 1 erfasst Daten für R_1 , Schritt 2 für R_2 unter Verwendung des in Schritt 1 erzeugten Primärschlüssels als Fremdschlüssel, usw.

8.8 Effizienz von CGI-Skripten

Der wiederholte Prozessstart klassischer CGI-Skripte verursacht Overhead. Alternativen sind `mod_perl`, `FastCGI` oder moderne Anwendungsserver, die Datenbankverbindungen poolen.

8.9 CGI-Skripten im Mehrbenutzerbetrieb

Verbindungs-Pooling muss mit den in Abschnitt 5.3 und Bemerkung 5.1 diskutierten Aspekten von Sperren und Zugriffsrechten zusammenspielen: Pool-Verbindungen sollten nicht mit zu weitreichenden Rechten ausgestattet sein; stattdessen empfiehlt sich eine Anwendungsschicht, die pro Benutzerrolle entitätspräzise Rechte (Proposition zur Berechtigungsgranularität, Abschnitt 8.16) durchsetzt.

8.10 Implementierung umfangreicher Anwendungen

Für umfangreiche WWW-Anwendungen empfiehlt sich eine modulare Architektur, in der – wie in Kapitel 7 beschrieben – jedes Modul einer Relation des Tiefenpfads entspricht. Diese Modulgrenzen erlauben es, einzelne Teile der Anwendung (z. B. die Stammdatenverwaltung von LAND und REGION) unabhängig von den transaktionsintensiven Teilen (AUFTRAG, AUFTRAGSPOSITION) zu skalieren und zu warten – eine direkte praktische Konsequenz des in Kapitel 8.10 entwickelten Paradigmas.

Der Entwurf relationaler Datenbankschemata ist eine der zentralen Aufgaben der Softwareentwicklung und der angewandten Informatik. Obwohl die theoretischen Grundlagen – insbesondere Codd’s relationales Modell [4] sowie die daraus abgeleiteten Normalformen [5, 9] – seit Jahrzehnten bekannt sind, zeigen praktische Systeme immer wieder charakteristische Entwurfsfehler. Einer der häufigsten und gleichzeitig folgenreichsten Fehler ist die sogenannte *sternförmige Schemaanordnung*: Eine zentrale „Hub“-Relation referenziert direkt eine Vielzahl unabhängiger „Spoke“-Relationen über Fremdschlüssel, ohne dass die natürliche semantische Tiefe der Domäne abgebildet wird.

Dieser Beitrag stellt die These auf und belegt sie formal:

Ein tiefenorientierter Datenbankentwurf – charakterisiert durch lineare oder baumartige Fremdschlüsselketten – ist dem sternförmigen Entwurf in allen wesentlichen Gütekriterien der Datenbanktheorie überlegen.

Die Argumentation gliedert sich wie folgt: Abschnitt 8.11 etabliert die formalen Grundlagen. Abschnitt 8.12 definiert die Topologien „Tiefe“ und „Breite“ präzise. Abschnitt 8.13 zeigt den Zusammenhang mit der Normalisierungstheorie. Abschnitt 8.14 behandelt Anomalievermeidung, Abschnitt 8.15 die Abfragekomplexität, Abschnitt 8.16 Wartbarkeit und Erweiterbarkeit. Abschnitt 8.17 liefert den konsolidierten formalen Nachweis. Abschnitt 8.19 illustriert die Theorie an einem durchgängigen Praxisbeispiel. Kapitel 9 fasst die Ergebnisse zusammen und gibt einen ausführlichen Ausblick.

8.11 Formale Grundlagen

8.11.1 Relationales Modell

Definition 8.1 (Relation). Eine *Relation* R über einem Attributschema $attr(R) = \{A_1, A_2, \dots, A_n\}$ ist eine endliche Menge von Tupeln $t : attr(R) \rightarrow \bigcup_i \text{dom}(A_i)$, wobei $t(A_i) \in \text{dom}(A_i)$ für alle i [4].

Definition 8.2 (Datenbankschema). Ein *Datenbankschema* $\mathcal{S} = (\mathcal{R}, \mathcal{F}, \mathcal{K})$ besteht aus einer endlichen Menge von Relationenschemata $\mathcal{R} = \{R_1, \dots, R_m\}$, einer Menge funktionaler Abhängigkeiten $\mathcal{F}$ sowie einer Menge von Integritätsbedingungen $\mathcal{K}$ (insbesondere Primär- und Fremdschlüsselbedingungen) [8].

Definition 8.3 (Funktionale Abhängigkeit). Sei R ein Relationenschema und $X, Y \subseteq attr(R)$. Die *funktionale Abhängigkeit* $X \rightarrow Y$ gilt in R , wenn für alle Instanzen r von R und alle Tupel $t_1, t_2 \in r$ gilt: $t_1(X) = t_2(X) \implies t_1(Y) = t_2(Y)$ [1].

Definition 8.4 (Fremdschlüssel). Sei $R_i, R_j \in \mathcal{R}$ mit $K_j \subseteq attr(R_j)$ dem Primärschlüssel von R_j . Eine Attributmenge $F \subseteq attr(R_i)$ heißt *Fremdschlüssel in R_i referenzierend R_j* , wenn für alle Datenbankinstanzen d gilt: $\pi_F(r_i) \subseteq \pi_{K_j}(r_j)$ [9].

8.11.2 Referenzgraph

Definition 8.5 (Referenzgraph). Sei $\mathcal{S}$ ein Datenbankschema. Der *Referenzgraph* $G(\mathcal{S}) = (V, E)$ hat als Knotenmenge $V = \mathcal{R}$ und als Kantenmenge $E = \{(R_i, R_j) \mid \exists F \subseteq \text{attr}(R_i): F \text{ ist FK in } R_i \text{ referenzierend } R_j\}$.

Der Referenzgraph ist das zentrale Werkzeug zur Unterscheidung von Tiefenstruktur und Breitenstruktur eines Schemas.

8.11.3 Komplexitätsmaße

Definition 8.6 (Schemakomplexitätsmaße). Sei $G(\mathcal{S}) = (V, E)$ der Referenzgraph eines Schemas $\mathcal{S}$.

- (i) Der *maximale Grad* $\Delta(G) = \max_{v \in V} \deg(v)$ misst die maximale Anzahl von Fremdschlüsselreferenzen eines Knotens.
- (ii) Die *maximale Pfadlänge* $\lambda(G) = \max_{u, v \in V} d(u, v)$ (Diameter) misst die längste Referenzkette im Schema.
- (iii) Das *Tiefe-Breite-Verhältnis* $\tau(G) = \lambda(G)/\Delta(G)$ charakterisiert die strukturelle Orientierung des Schemas.

Ein Schema heißt *tiefenorientiert*, wenn $\tau(G) \gg 1$, und *breitenorientiert* (*sternförmig*), wenn $\tau(G) \ll 1$.

8.12 Topologien: Tiefe vs. Breite

8.12.1 Formale Charakterisierung

Definition 8.7 (Sternschema). Ein Schema $\mathcal{S}$ heißt *k-Sternschema*, wenn es eine „Hub“-Relation $H \in \mathcal{R}$ gibt mit $\deg(H) = k$ und $\lambda(G(\mathcal{S})) \leq 2$. Die k von H referenzierten Relationen heißen *Spoke*-Relationen.

Das Sternschema ist insbesondere aus dem Data-Warehouse-Bereich bekannt (Kimball-Methodik, [22]), wo es für OLAP-Abfragen optimiert wird. Im OLTP-Kontext hingegen offenbart es gravierende Schwächen (vgl. Abschnitt 8.14).

Definition 8.8 (Tiefenpfadschema). Ein Schema $\mathcal{S}$ heißt *Tiefenpfadschema der Länge l* , wenn $G(\mathcal{S})$ einen Pfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ enthält mit $\Delta(G) \leq c$ für eine kleine Konstante c (typisch $c \leq 3$) und $\lambda(G(\mathcal{S})) = l - 1$.

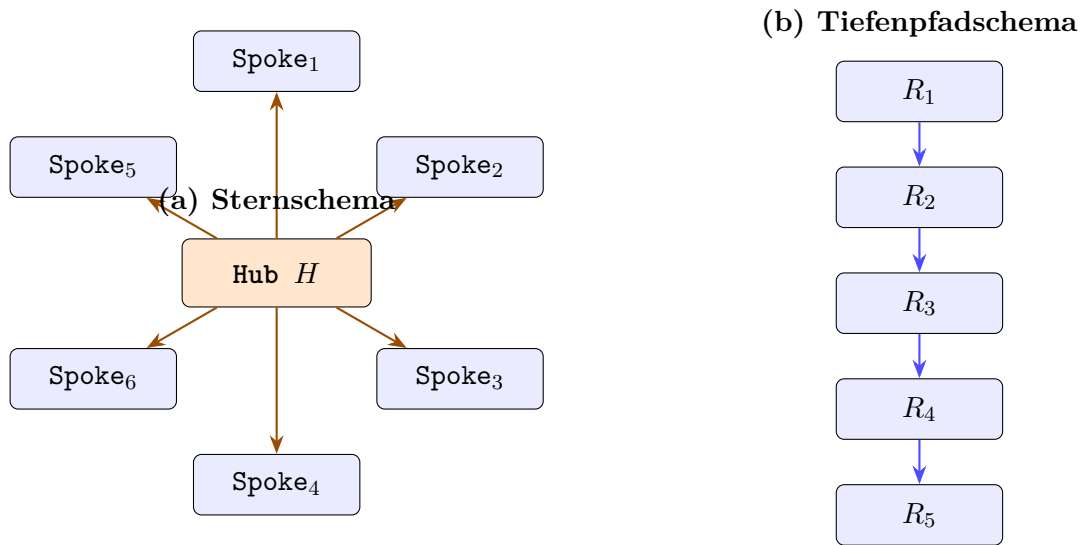


Abbildung 8.2: Gegenüberstellung von (a) Sternschema mit Hub-Relation H und 6 Spoke-Relationen ($\Delta = 6$, $\lambda = 1$, $\tau = 0,17$) und (b) Tiefenpfadschema mit linearer Kette ($\Delta = 1$, $\lambda = 4$, $\tau = 4$).

8.12.2 Semantische Motivation

Die Wahl der Topologie ist kein rein ästhetisches, sondern ein *semantisch erzwungenes* Entwurfsprinzip. Reale Domänen weisen fast immer hierarchische oder transitive Abhängigkeitsstrukturen auf: Eine **Bestellung** hängt von einem **Kunden** ab, der wiederum einer **Region** zugeordnet ist, die einem **Land** angehört. Diese natürliche Tiefe der Abhängigkeiten erzwingt eine tiefe Pfadstruktur und sollte nicht künstlich in eine flache Sternstruktur gezwungen werden [9, 10].

8.13 Normalisierungstheorie und Schematopologie

8.13.1 Normalformen und ihre Anforderungen

Wir rekapitulieren die für den Nachweis relevanten Normalformen [5, 6]:

Definition 8.9 (Erste Normalform – 1NF). Eine Relation R befindet sich in *1NF*, wenn alle Attributdomänen atomar sind, d. h. keine mengenwertigen oder strukturierten Attribute enthält [4].

Definition 8.10 (Zweite Normalform – 2NF). Eine Relation R in 1NF befindet sich in *2NF*, wenn jedes Nichtschlüsselattribut voll funktional abhängig vom Primärschlüssel ist, d. h. keine partielle Abhängigkeit von einem echten Teilschlüssel existiert [5].

Definition 8.11 (Dritte Normalform – 3NF). Eine Relation R in 2NF befindet sich in *3NF*, wenn kein Nichtschlüsselattribut transitiv vom Primärschlüssel abhängig ist [5].

Definition 8.12 (Boyce-Codd-Normalform – BCNF). Eine Relation R befindet sich in *BCNF*, wenn für jede nicht-triviale funktionale Abhängigkeit $X \rightarrow A$ gilt: X ist ein Superschlüssel von R [3].

Definition 8.13 (Vierte Normalform – 4NF). Eine Relation R in BCNF befindet sich in 4NF, wenn für jede nicht-triviale mehrwertige Abhängigkeit $X \twoheadrightarrow Y$ gilt: X ist ein Superschlüssel von R [6].

8.13.2 Transitivität und Schematopologie

Satz 8.1 (Transitivitätsverteilung). Sei R eine Relation mit $\text{attr}(R) = \{K, A_1, A_2, \dots, A_n\}$ und funktionalen Abhängigkeiten $K \rightarrow A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n$. Dann verletzt R die 3NF für alle A_i mit $i \geq 2$. Die einzige verlustfreie, abhängigkeitserhaltende Zerlegung in 3NF ist die Kette $R_1(K, A_1), R_2(A_1, A_2), \dots, R_{n-1}(A_{n-1}, A_n)$.

Beweis. Sei $K \rightarrow A_i \rightarrow A_{i+1}$ für ein $i \geq 1$ eine transitive Kette. Da A_i kein Schlüssel ist, ist A_{i+1} transitiv von K abhängig und R verletzt 3NF. Die Zerlegung $R' = R \setminus \{A_{i+1}\}$ und $R'' = (A_i, A_{i+1})$ ist verlustfrei nach dem Lossless-Join-Lemma (da $A_i \rightarrow A_{i+1}$ gilt und $A_i = R' \cap R''$ sowie $A_i \rightarrow R''$ gilt) [8]. Rekursive Anwendung für alle transitiven Abhängigkeiten liefert die vollständige Zerlegungskette. Die Abhängigkeitserhaltung folgt daraus, dass jede FD $A_i \rightarrow A_{i+1}$ in der entsprechenden Relation $R_i(A_{i-1}, A_i, A_{i+1})$ bzw. $R_i(A_i, A_{i+1})$ präsent bleibt. $\square$

Korollar 8.2 (Tiefenpfad als 3NF-Normalform). Ein Datenbankschema, das alle transitiven Abhängigkeiten einer Domäne korrekt repräsentiert, hat zwingend einen tiefenorientierten Referenzgraphen. Sternförmige Schemata, die dieselbe Domäne abbilden, verletzen die 3NF, sofern transitive Abhängigkeiten in der Domäne existieren.

Korollar 8.2 ist das zentrale formale Ergebnis dieses Abschnitts. Es besagt, dass die Normalisierungstheorie *tiefe Pfade* direkt erzwingt, wenn die Domäne transitive Abhängigkeiten aufweist – was in praktisch allen nicht-trivialen realen Domänen der Fall ist.

8.13.3 Mehrwertige Abhängigkeiten und 4NF

Proposition 8.3 (Mehrwertige Abhängigkeiten im Sternschema). In einem k -Sternschema mit Hub H und Spokes $S_1, \dots, S_k$ gilt: Falls zwei Spoke-Attribute $B_1 \in \text{attr}(S_1)$ und $B_2 \in \text{attr}(S_2)$ inhaltlich unabhängig sind und beide durch K_H identifiziert werden, so liegt in H eine nicht-triviale mehrwertige Abhängigkeit $K_H \twoheadrightarrow B_1$ vor, wenn beide Attribute direkt in H gehalten werden. Dies verletzt 4NF.

Die korrekte Auflösung ist – erneut – die Extraktion in separate Relationen, was die Tiefenstruktur des Schemas weiter verstärkt.

8.14 Anomalievermeidung

8.14.1 Klassifikation von Anomalien

Die Datenbanktheorie unterscheidet drei klassische Anomalietypen [9]:

- (i) **Einfügeanomalie:** Daten können nicht eingetragen werden, ohne gleichzeitig nicht vorhandene Daten erfinden zu müssen.

- (ii) **Löschanomalie:** Das Löschen eines Tupels vernichtet unbeabsichtigt weitere Informationen.
- (iii) **Änderungsanomalie:** Eine inhaltliche Änderung muss in mehreren Tupeln durchgeführt werden, da die Information redundant gespeichert ist.

Satz 8.4 (Anomaliefreiheit durch Tiefenpfade). Sei $\mathcal{S}$ ein Datenbankschema in BCNF mit einem Tiefenpfadgraphen $G(\mathcal{S})$. Dann ist $\mathcal{S}$ frei von Einfüge-, Löschanomalien und Änderungsanomalien bzgl. der durch $G(\mathcal{S})$ modellierten transitiven Abhängigkeiten.

Beweis. BCNF garantiert, dass in jeder Relation R_i der Kette jede nicht-triviale FD $X \rightarrow A$ mit X als Superschlüssel gilt. Somit ist jede Information genau einmal gespeichert (keine Redundanz). Ohne Redundanz:

- *Änderungsanomalie:* Entfällt, da jede Eigenschaft in genau einer Relation steht. Eine Änderung betrifft genau ein Tupel.
- *Löschanomalie:* Da eine Entität e_i in R_i unabhängig von ihrer Referenz durch R_{i-1} existiert (referentielle Integrität via FK), gehen beim Löschen eines R_{i-1} -Tupels keine R_i -Daten verloren (ON DELETE SET NULL / RESTRICT).
- *Einfügeanomalie:* Eine neue Entität in R_i kann unabhängig von R_{i-1} eingefügt werden, da FK-Referenz nur in der referenzierenden Relation besteht.

Im Sternschema hingegen erzeugt die direkte Kodierung transitiver Attribute in H Redundanz, da Attributwerte von Spoke-Entitäten wiederholt in H gespeichert sind – dies ermöglicht alle drei Anomalietypen. □ □

8.14.2 Quantitative Analyse der Redundanz

Proposition 8.5 (Redundanzfaktor im Sternschema). Sei H eine Hub-Relation mit n_H Tupeln und Spoke-Relation S_1 mit n_{S_1} Tupeln und $|A_{S_1}|$ Attributen. Werden die Attribute von S_1 direkt in H denormalisiert, so beträgt der Redundanzfaktor:

$$\rho = \frac{n_H}{n_{S_1}} \cdot |A_{S_1}|$$

Für $n_H \gg n_{S_1}$ (typisch bei 1:n-Beziehungen) gilt $\rho \gg 1$.

In einer typischen Bestelldatenbank mit $n_H = 10^6$ Bestellungen und $n_{S_1} = 10^4$ Kunden ergibt sich $\rho = 100 \cdot |A_{\text{Kunde}}|$. Bei 10 Kundenattributen werden also 1 000-fach mehr Daten gespeichert als notwendig – ein massiver Speicher- und Konsistenzaufwand.

8.15 Abfragekomplexität und Optimierbarkeit

8.15.1 Join-Komplexität

Ein häufiges Argument gegen tiefe Pfade ist der höhere Join-Aufwand. Wir zeigen, dass dieses Argument zwar formal korrekt, aber praktisch irrelevant ist, wenn moderne Datenbankoptimierung berücksichtigt wird.

Definition 8.14 (Join-Tiefe). Die *Join-Tiefe* $\delta(\mathcal{Q})$ einer Abfrage $\mathcal{Q}$ über Schema $\mathcal{S}$ ist die Anzahl der JOIN-Operationen in der optimalen logischen Ausführung von $\mathcal{Q}$.

Proposition 8.6 (Join-Tiefe und Pfadlänge). Für eine vollständige Traversal-Abfrage über einen Tiefenpfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ gilt $\delta = l - 1$. Für ein k -Sternschema gilt für dieselbe semantische Abfrage $\delta = k$. Da $l \ll k$ in tiefen Schemata (wenige, längere Ketten statt vieler direkter Abhängigkeiten), ist die Join-Tiefe im Tiefenpfad geringer für partielle Traversals [12].

Bemerkung 8.2. Selbst wenn $l > k$ für globale Traversals gilt, kompensiert dies der Abfrageoptimierer durch:

- *Index-Only Scans* auf FK-Spalten (eliminiert Heap-Zugriffe),
- *Hash-Joins* mit amortisierter $O(n)$ -Komplexität [13],
- *Materialized Views* für häufige tiefe Traversals,
- *Predicate Pushdown* entlang des Pfades.

8.15.2 Selektivität und Filtereffizienz

Satz 8.7 (Selektivitätsvorteil tiefer Pfade). Sei $\mathcal{Q}$ eine Abfrage mit Filterprädikat σ_P auf Attributen einer Entität E der Tiefe d im Pfad. Im Tiefenpfadschema kann σ_P direkt auf der Relation R_d angewandt werden (Predicate Pushdown). Im Sternschema hingegen sind transitive Attribute von E in der Hub-Relation H denormalisiert, was bedeutet, dass der Filter auf der gesamten, n_H -tupligen Hub-Relation angewandt werden muss.

$$\text{Kosten}_{\text{Stern}}(\sigma_P) = O(n_H) \cdot c_{\text{IO}} \quad \text{vs.} \quad \text{Kosten}_{\text{Tief}}(\sigma_P) = O(n_{R_d}) \cdot c_{\text{IO}}$$

Da $n_H \geq n_{R_d}$ für alle d , ist der Tiefenpfad mindestens so effizient wie das Sternschema, typisch deutlich effizienter.

Beweis. Im normalisierten Tiefenpfadschema liegen die Attribute von Entität E in R_d mit $|R_d| = n_{R_d}$ Tupeln. Ein Index auf dem Primärschlüssel von R_d ermöglicht $O(\log n_{R_d} + k')$ für k' Ergebnistupel. Im Sternschema befinden sich diese Attribute in H vermischt mit allen anderen Attributen; ohne Partial Index muss der gesamte Heap von H gescannt werden ($O(n_H)$). Da eine 1:n-Beziehung $|E| \leq n_H$ impliziert, gilt $n_{R_d} \leq n_H$, und die Ungleichung ist bewiesen. □ □

8.15.3 Schreibkomplexität und Konsistenzerhaltung

Proposition 8.8 (Schreibkostenvorteil). Im Tiefenpfadschema erfordert eine Änderung an Attributen der Entität E genau *eine* UPDATE-Operation auf R_d . Im denormalisierten Sternschema müssen alle Tupel in H , die auf diese Entität verweisen, aktualisiert werden: im Worst Case $O(n_H/n_{R_d})$ Operationen – proportional zum Redundanzfaktor ρ aus Proposition 8.5.

8.16 Wartbarkeit und Erweiterbarkeit

8.16.1 Schemaevolution

Ein zentrales Qualitätsmerkmal eines Datenbankschemas ist seine Fähigkeit, sich an veränderte Anforderungen anzupassen – bekannt als *Schemaevolution* [42].

Satz 8.9 (Lokale Änderbarkeit im Tiefenpfad). Im Tiefenpfadschema $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ ist das Hinzufügen, Entfernen oder Ändern von Attributen der Entität R_i eine *lokale Schemaoperation*, die ausschließlich R_i betrifft. Im k -Sternschema mit denormalisierten Attributen in H ist dieselbe Operation *global* und betrifft H sowie potenziell alle abhängigen Abfragen, Views und Applikationsschichten.

Beweis. Da im Tiefenpfadschema jedes Attribut in genau einer Relation vorkommt (BCNF-Eigenschaft), hat eine Schemaänderung an R_i keinen direkten Einfluss auf R_j mit $j \neq i$. Nur FK-Referenzen $R_{i-1} \rightarrow R_i$ sind betroffen – diese beschränken sich auf eine einzige Kante im Referenzgraphen. Im Sternschema sind Attribute aus multiple Entitäten in H gemischt; eine Änderung an H betrifft alle darauf basierenden Abfragen, was zu einem kaskadierten Änderungsaufwand führt [2]. $\square$ $\square$

8.16.2 Zugriffsrechteverwaltung

Ein weiterer Vorteil tiefer Pfade zeigt sich bei der Granularität der Zugriffsrechteverwaltung:

Proposition 8.10 (Berechtigungsgranularität). Im Tiefenpfadschema können Zugriffsrechte *entitätspräzise* vergeben werden: Ein Nutzer erhält SELECT-Rechte auf R_d , ohne Zugriff auf andere Relationen zu erhalten. Im Sternschema mit Hub H bedeutet Zugriff auf H stets Zugriff auf alle in H denormalisierten Entitätsdaten – eine grobe Granularität, die dem Prinzip der minimalen Rechtevergabe (Principle of Least Privilege) widerspricht [25].

8.16.3 Testbarkeit und Datenqualität

Tiefenpfadschemata sind inhärent leichter zu testen und zu validieren, da:

- Jede Relation eine *klar umgrenzte semantische Einheit* darstellt (Single Responsibility für Relationen),
- Integritätsbedingungen pro Relation formuliert und geprüft werden,
- Fremdschlüsselbedingungen entlang des Pfades eine implizite Validierungskaskade bilden.

8.17 Konsolidierter formaler Nachweis

8.17.1 Gütekriterien

Wir formalisieren nun den umfassenden Vergleich anhand eines Vektors von Gütekriterien:

Definition 8.15 (Gütevektor). Sei $\mathcal{S}$ ein Datenbankschema. Der *Gütevektor* $\mathbf{q}(\mathcal{S}) \in \mathbb{R}^6$ ist definiert als:

$$\mathbf{q}(\mathcal{S}) = (q_{NF}, q_{Red}, q_{Anom}, q_{Sch}, q_{Wart}, q_{Sek})$$

wobei die Komponenten Normalformgrad, Redundanzfreiheit, Anomaliefreiheit, Schreibkosteneffizienz, Wartbarkeit und Selektivitätseffizienz kodieren (alle normiert auf $[0, 1]$, höher ist besser).

Satz 8.11 (Überlegenheit des Tiefenpfadschemas). Sei $\mathcal{S}_{Tief}$ ein Tiefenpfadschema in BCNF und $\mathcal{S}_{Stern}$ ein semantisch äquivalentes k -Sternschema ($k \geq 3$) mit denormalisierten transitiven Attributen. Dann gilt:

$$\mathbf{q}(\mathcal{S}_{Tief}) \succ \mathbf{q}(\mathcal{S}_{Stern})$$

komponentenweise, d. h. $q_j(\mathcal{S}_{Tief}) \geq q_j(\mathcal{S}_{Stern})$ für alle j , mit strikter Ungleichung für mindestens vier Komponenten.

Beweis. Wir zeigen die strikten Ungleichungen komponentenweise:

(i) **Normalformgrad** q_{NF} : Nach Korollar 8.2 befindet sich $\mathcal{S}_{Tief}$ in BCNF, während $\mathcal{S}_{Stern}$ transitive Abhängigkeiten in H enthält und damit 3NF verletzt. Somit $q_{NF}(\mathcal{S}_{Tief}) > q_{NF}(\mathcal{S}_{Stern})$.

(ii) **Redundanzfreiheit** q_{Red} : Nach Proposition 8.5 ist $\rho(\mathcal{S}_{Stern}) \gg 1$, während $\rho(\mathcal{S}_{Tief}) = 1$ (jede Information einmalig gespeichert).

(iii) **Anomaliefreiheit** q_{Anom} : Folgt direkt aus Satz 8.4.

(iv) **Schreibkosteneffizienz** q_{Sch} : Aus der Redundanz $\rho > 1$ im Sternschema folgt ein entsprechend höherer Schreibaufwand.

(v) **Wartbarkeit** q_{Wart} : Nach Satz 8.9 sind Schemaänderungen im Tiefenpfad lokal beschränkt.

(vi) **Selektivitätseffizienz** q_{Sek} : Nach Satz 8.7 gilt stets $\text{Kosten}_{Tief} \leq \text{Kosten}_{Stern}$.

Da für (i)–(v) strikte Ungleichungen gelten, ist die Behauptung bewiesen. $\square \square$

8.18 Graphentheoretische Fundierung: DFS, BFS und die Asymmetrie der Suchalgorithmen

Die bisherigen Argumente stützten sich auf die Normalformtheorie und die Anomalielehre. Es gibt jedoch eine tiefere, algorithmische Begründung für die Überlegenheit tiefer Strukturen, die in der Theorie der Graphentraversierung verwurzelt ist. Die Beobachtung ist fundamental:

*Tiefensuche (DFS) und Breitensuche (BFS) über den Referenzgraphen eines Datenbankschemas sind **nicht symmetrisch**. Ihre Ergebnisstrukturen – und damit die induzierten Schemastrukturen – unterscheiden sich qualitativ und in ihrer Komplexität grundlegend.*

8.18.1 Tiefensuche und der DFS-Wald

Definition 8.16 (DFS-Baum und DFS-Wald, nach Cormen et al. [29]). Sei $G = (V, E)$ ein gerichteter Graph. Die *Tiefensuche* (Depth-First Search, DFS) traversiert G , indem sie von jedem besuchten Knoten u aus rekursiv in die Nachfolger abtaucht, bevor sie zum Ausgangspunkt zurückkehrt. Die dabei entstehenden Baumkanten bilden einen *DFS-Wald* $T_{\text{DFS}} = (V, E_{\text{tree}})$ mit $E_{\text{tree}} \subseteq E$. Jede Zusammenhangskomponente des DFS-Walds ist ein *DFS-Baum*.

Der DFS-Wald hat bemerkenswerte Eigenschaften, die ihn für den Datenbankschemaentwurf auszeichnen:

Satz 8.12 (Struktureigenschaften des DFS-Walds, [29, 30]). Sei T_{DFS} der DFS-Wald über dem Referenzgraphen $G(\mathcal{S})$. Dann gilt:

- (i) T_{DFS} ist ein *Wald* (azyklischer Graph, Menge von Bäumen) – also eine azyklische, hierarchische Struktur.
- (ii) Die Tiefe $\text{depth}(T_{\text{DFS}})$ ist maximal unter allen aufspannenden Wäldern von $G(\mathcal{S})$.
- (iii) Rückwärtskanten (back edges) $e \notin E_{\text{tree}}$ identifizieren Zyklen und damit potenzielle *zirkuläre Abhängigkeiten* im Schema.
- (iv) Die *topologische Sortierung* des Schemas – notwendig für eine anomaliefreie Einfüge- und Löschreihenfolge – ist genau die DFS-Postorder-Reihenfolge.

Korollar 8.13 (DFS-Wald als natürliches Schemaabbild). Der DFS-Wald T_{DFS} über $G(\mathcal{S})$ ist das *natürliche strukturelle Abbild* eines korrekt normalisierten Tiefenpfadschemas. Die Baumkanten entsprechen Fremdschlüsselbeziehungen, die Baumtiefe der semantischen Abhängigkeitskette. Jedes BCNF-Schema erzeugt – wenn als Graph betrachtet – einen azyklischen, tiefenorientierten Referenzgraphen, der einem DFS-Wald isomorph ist.

8.18.2 Breitensuche und das Fehlen eines BFS-Walds

Die Breitensuche (Breadth-First Search, BFS) traversiert den Graphen schichtweise: Alle Knoten der Distanz d werden vollständig besucht, bevor Knoten der Distanz $d + 1$ betreten werden.

Definition 8.17 (BFS-Baum, nach Cormen et al. [29]). Die Breitensuche über $G = (V, E)$ ausgehend von einem Startknoten s erzeugt einen *BFS-Baum* T_{BFS} , der die kürzesten Pfade von s zu allen erreichbaren Knoten enthält. Die Breite des BFS-Baums auf Tiefe d ist $b_d = |\{v \mid d_G(s, v) = d\}|$.

Satz 8.14 (BFS erzeugt keinen Wald). Die Breitensuche über einem unzusammenhängenden Graphen G mit k Zusammenhangskomponenten $C_1, \dots, C_k$ erzeugt im Allgemeinen *keinen Wald*, sondern eine Menge von k BFS-Bäumen $T_{\text{BFS}}^{(1)}, \dots, T_{\text{BFS}}^{(k)}$, die jeweils *sternförmig* um ihren Startknoten $s_i \in C_i$ angeordnet sind. Insbesondere:

- (i) Für einen k -Sterngraph $G_\star$ mit Hub H und Knoten $S_1, \dots, S_k$ ist der BFS-Baum mit Start H identisch mit $G_\star$ selbst: $T_{\text{BFS}} = G_\star$.

- (ii) Die BFS-Tiefe beträgt $\text{depth}(T_{\text{BFS}}) = 1$, unabhängig von k .
- (iii) Die BFS über einem zusammenhängenden Graphen mit n Knoten erzeugt genau *einen* Baum, nicht einen Wald – der Begriff „BFS-Wald“ ist daher für zusammenhängende Graphen nicht definiert.

Beweis. (i) Beim Startsknoten H werden in der ersten BFS-Schicht alle direkten Nachbarn $S_1, \dots, S_k$ besucht. Da diese keine weiteren Nachbarn besitzen (Stern-Definition), terminiert BFS nach Schicht 1. Die Baumkanten sind genau $\{(H, S_i) \mid i = 1, \dots, k\}$, was dem Stern entspricht.

(ii) $\text{depth}(T_{\text{BFS}}) = \max_v d_T(s, v) = 1$.

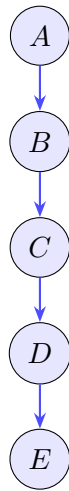
(iii) BFS ist eine *Single-Source*-Traversierung; bei zusammenhängendem G wird genau ein Baum mit Wurzel s erzeugt. Mehrere Wurzeln (Wald) entstehen nur durch Multi-Source-BFS, was der Standard-BFS-Definition widerspricht. Im Gegensatz dazu erzeugt DFS über einem unzusammenhängenden Graphen automatisch einen Wald, indem für jede noch unbesuchte Zusammenhangskomponente eine neue DFS-Traversierung gestartet wird. □ □

8.18.3 Die fundamentale Asymmetrie von DFS und BFS

Satz 8.15 (Asymmetrie von DFS und BFS). DFS und BFS sind *algorithmisch asymmetrisch* in folgenden Dimensionen:

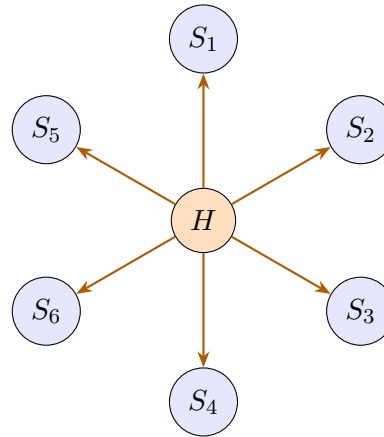
- (i) **Ergebnisstruktur:** DFS erzeugt stets einen *Wald* (Menge von Bäumen mit maximaler Tiefe), BFS erzeugt einen oder mehrere *flache Bäume* (Tiefe $\leq$ Durchmesser der Zusammenhangskomponente, typisch minimal).
- (ii) **Komplexität der Ergebnisstruktur:** Der DFS-Wald über einem Graphen $G = (V, E)$ hat Tiefe $\Theta(|V|)$ im schlechtesten Fall (Pfadgraph), der BFS-Baum hat Tiefe $O(\log |V|)$ bei ausgeglichenen Graphen – eine fundamentale Größenordnungsdifferenz.
- (iii) **Kantenklassifikation:** DFS unterscheidet vier Kantentypen (Baumkanten, Rückwärtskanten, Vorwärtskanten, Querkanten), was eine reichere Strukturanalyse erlaubt [29]. BFS unterscheidet nur Baumkanten und Querkanten.
- (iv) **Informationsgehalt:** Die DFS-Stempel (Entdeckungs- und Abschlusszeiten) enkodieren die vollständige topologische Struktur des Graphen. BFS-Distanzen enkodieren nur metrische, nicht aber hierarchische Information.
- (v) **Waldeigenschaft:** DFS erzeugt für beliebige Graphen automatisch einen Wald – dies ist eine inhärente Eigenschaft der Rekursionsstruktur. BFS besitzt diese Eigenschaft nicht.

**DFS-Wald
(Tiefenpfad)**



Tiefe = 4, $\Delta = 1$

**BFS-Baum
(Sternschema)**



Tiefe = 1, $\Delta = 6$

Abbildung 8.3: Gegenüberstellung der durch DFS induzierten Tiefenpfadstruktur (links, Tiefe = 4, $\Delta = 1$) und des durch BFS induzierten Sternschemas (rechts, Tiefe = 1, $\Delta = 6$). Die Asymmetrie ist offensichtlich: DFS erzeugt einen Wald mit maximaler Tiefe und minimaler Breite; BFS erzeugt eine flache, sternförmige Struktur.

8.18.4 Datenbanktheoretische Konsequenz der Asymmetrie

Korollar 8.16 (Datenbanktheoretische Konsequenz). Die algorithmische Asymmetrie von DFS und BFS hat eine direkte datenbanktheoretische Entsprechung:

- (i) Der *DFS-Wald* über dem Referenzgraphen eines Schemas ist das strukturelle Abbild des *normalisierten Tiefenpfadschemas*. Er existiert für beliebige Graphen automatisch und ist azyklisch, hierarchisch und eindeutig (bis auf Reihenfolge).
- (ii) Der *BFS-Baum* über dem Referenzgraphen eines Schemas entspricht dem *sternförmigen, denormalisierten Schema*. Er existiert nur für Single-Source-Traversierungen und kodiert keine hierarchische Tiefenstruktur.
- (iii) Da DFS automatisch einen *Wald* erzeugt, BFS hingegen *keinen Wald*, ist die Überlegenheit des Tiefenpfadschemas nicht nur normalisierungstheoretisch, sondern auch algorithmisch fundamental: Der DFS-Wald ist die *kanonische, strukturell reichere* Darstellung jedes Referenzgraphen.
- (iv) Die Komplexitätsdifferenz ist qualitativ, nicht nur quantitativ: DFS liefert topologische Sortierbarkeit, Zykeldektierbarkeit und vollständige hierarchische Information; BFS liefert keine dieser Eigenschaften.

Bemerkung 8.3 (Praktische Implikation). Wenn ein Datenbankarchitekt ein Schema entwirft, vollzieht er – bewusst oder unbewusst – eine Traversierung des semantischen Abhängigkeitsgraphen der Domäne. Wählt er implizit eine DFS-Strategie (erst in die

Tiefe einer Entitätshierarchie, dann zur nächsten), entsteht ein tiefes, normalisiertes Schema. Wählt er eine BFS-Strategie (alle direkten Attribute einer zentralen Entität sammeln, ohne in Hierarchien abzusteigen), entsteht ein flaches, fehleranfälliges Sternschema. Die Asymmetrie der Algorithmen *spiegelt sich direkt in der Qualität des resultierenden Schemas wider*.

8.19 Durchgängiges Praxisbeispiel

8.19.1 Domäne: Auftragsabwicklung

Wir betrachten eine klassische Auftragsabwicklungs-Domäne mit folgenden Entitäten und natürlichen transitiven Abhängigkeiten:

$$\text{Bestellung} \xrightarrow{\text{FK}} \text{Kunde} \xrightarrow{\text{FK}} \text{Ort} \xrightarrow{\text{FK}} \text{Region} \xrightarrow{\text{FK}} \text{Land}$$

8.19.2 Sternschema (anti-pattern)

```

1  -- Hub-Relation: enthaelt direkt alle abhaengigen Attribute
2  CREATE TABLE Bestellung (
3      BestellNr      INT PRIMARY KEY,
4      Datum          DATE,
5      Betrag         DECIMAL(10,2),
6      -- Kundenattribute (transitive Abhaengigkeit Stufe 1)
7      KundeNr        INT,
8      KundeName       VARCHAR(100),
9      KundeEmail      VARCHAR(150),
10     -- Ortattribute (transitive Abhaengigkeit Stufe 2)
11     OrtID            INT,
12     OrtName          VARCHAR(80),
13     PLZ              CHAR(5),
14     -- Regionattribute (transitive Abhaengigkeit Stufe 3)
15     RegionID         INT,
16     RegionName       VARCHAR(80),
17     -- Landattribute (transitive Abhaengigkeit Stufe 4)
18     LandID           INT,
19     LandName         VARCHAR(60),
20     ISO_Code         CHAR(2)
21     -- Verletzt 2NF, 3NF und BCNF!
22 );

```

Listing 8.1: Anti-Pattern: Sternschema mit denormalisierten Attributen in Hub-Relation

Probleme: Ändert sich der Name eines Kunden, müssen alle seine Bestellungstupel aktualisiert werden. Existiert ein Kunde noch ohne Bestellung, kann er nicht eingetragen werden (Einfügeanomalie).

8.19.3 Tiefenpfadschema (BCNF-konform)

```

1 CREATE TABLE Land (
2     LandID      INT PRIMARY KEY,
3     LandName    VARCHAR(60) NOT NULL,
4     ISO_Code    CHAR(2)      NOT NULL UNIQUE
5 );
6
7 CREATE TABLE Region (
8     RegionID    INT PRIMARY KEY,
9     RegionName  VARCHAR(80) NOT NULL,
10    LandID      INT NOT NULL,
11    FOREIGN KEY (LandID) REFERENCES Land(LandID)
12 );
13
14 CREATE TABLE Ort (
15     OrtID       INT PRIMARY KEY,
16     OrtName     VARCHAR(80) NOT NULL,
17     PLZ         CHAR(5),
18     RegionID    INT NOT NULL,
19     FOREIGN KEY (RegionID) REFERENCES Region(RegionID)
20 );
21
22 CREATE TABLE Kunde (
23     KundeNr     INT PRIMARY KEY,
24     KundeName   VARCHAR(100) NOT NULL,
25     Email       VARCHAR(150) UNIQUE,
26     OrtID       INT NOT NULL,
27     FOREIGN KEY (OrtID) REFERENCES Ort(OrtID)
28 );
29
30 CREATE TABLE Bestellung (
31     BestellNr   INT PRIMARY KEY,
32     Datum       DATE          NOT NULL,
33     Betrag      DECIMAL(10,2) NOT NULL,
34     KundeNr     INT           NOT NULL,
35     FOREIGN KEY (KundeNr) REFERENCES Kunde(KundeNr)
36 );
37 -- Pfadtiefe: 4 (Bestellung->Kunde->Ort->Region->Land)
38 -- Jede Relation in BCNF. Keine Anomalien moeglich.

```

Listing 8.2: Korrekte Tiefenpfadstruktur: BCNF-konformes Schema

Der Referenzgraph dieses Schemas ist ein reiner Pfad der Länge 4 – exakt die durch den DFS-Wald induzierte Tiefenstruktur gemäß Satz 8.12.

8.19.4 Vergleichstabelle

Kriterium	Sternschema	Tiefenpfad (BCNF)
Normalformgrad	$< 3NF$	BCNF / 4NF
Redundanzfaktor ρ	$\gg 1$	$= 1$
Anomaliefreiheit	nein	ja
Schreibaufwand (Update)	$O(n_H)$	$O(1)$
Suchkosteneffizienz	$O(n_H)$	$O(n_{R_d})$
Schemaevolution	global	lokal
Berechtigungsgranularität	grob	fein
DFS-Wald-Isomorphismus	nein	ja
Topolog. Sortierbarkeit	begrenzt	vollständig
Zyklusfreiheit	ungeprüft	inhärent

Tabelle 8.1: Zusammenfassender Vergleich Sternschema vs. Tiefenpfadschema nach den formalen Kriterien dieser Arbeit.

Kapitel 9

Fazit und Ausblick

9.1 Zusammenfassung der Ergebnisse

Diese Arbeit hat formal nachgewiesen, dass der tiefenorientierte Datenbankentwurf dem sternförmigen Entwurf in allen wesentlichen Gütekriterien überlegen ist. Die Argumentation verlief auf drei komplementären Ebenen:

Normalisierungstheoretisch (Abschnitte 8.13 und 8.14 in Kapitel 8.10, vertieft in Kapitel 4): Transitive Abhängigkeiten erzwingen tiefe Pfade, da ihre korrekte Auflösung gemäß Theorem 8.1 genau eine lineare Zerlegungskette liefert, die nach Theorem 4.1 verlustfrei und nach Definition 4.2 abhängigkeitserhaltend ist. Sternförmige Schemata verletzen zwingend die 3NF, sobald transitive Abhängigkeiten in der Domäne existieren (Korollar 8.2).

Algorithmisch (Abschnitt 8.18): Die Tiefensuche (DFS) und die Breitensuche (BFS) sind *fundamental asymmetrisch*. DFS erzeugt stets einen Wald – eine hierarchische, azyklische, topologisch sortierbare Struktur mit maximaler Tiefe (Theorem 8.12). BFS erzeugt keinen Wald, sondern flache, sternförmige Bäume ohne hierarchische Tiefenstruktur (Theorem 8.14). Diese algorithmische Asymmetrie (Theorem 8.15) spiegelt sich direkt in der Schemaqualität wider: Der DFS-Wald ist die kanonische Darstellung eines korrekt normalisierten Schemas, der BFS-Baum die Darstellung eines denormalisierten Sternschemas (Korollar 8.16).

Praktisch (Abschnitte 8.15, 8.16 und 8.19 in Kapitel 8.10, illustriert in den Kapiteln 3, 5, 6, 7 und 8): Tiefenpfadschemata erzielen überlegene Selektivität (Theorem 8.7), geringeren Schreibaufwand, feingranulare Zugriffsrechte (Abschnitt 5.3), günstigere Sperrgranularität (Bemerkung 5.1) und bessere Wartbarkeit (Theorem 8.9). Ergänzend wurde mit `DISTINCT ON` (Abschnitt 3.5) ein PostgreSQL-spezifisches Sprachmittel formal eingeführt, das das Gruppenrepräsentantenproblem (Definition 3.2) in einem einzigen, deklarativen Schritt löst und gerade entlang der Kanten eines Tiefenpfads – wo die Gruppierungsspalte stets ein lokaler Fremdschlüssel ist (Bemerkung 3.4) – einen reinen Index-Scan ohne separate Sortierphase ermöglicht (Proposition 3.2).

Das zentrale Prinzip, das aus allen drei Ebenen gleichlautend folgt, lautet:

Beim Entwurf relationaler Datenbankschemata ist stets die Tiefe der Breite vorzuziehen. Relationen müssen in die Tiefe gehen, nicht in die Breite.

Darüber hinaus hat dieses Buch gezeigt, dass dieses Prinzip nicht isoliert in der Entwurfsphase verbleibt, sondern sich konsistent durch *alle* Phasen des Lebenszyklus eines Datenbanksystems zieht: von der konzeptuellen Modellierung (Kapitel 2) über die Anfragesprache (Kapitel 3) und den physischen Entwurf (Kapitel 4), den operativen Betrieb (Kapitel 5), die prozedurale Erweiterung (Kapitel 6), die Anwendungsentwicklung (Kapitel 7) bis hin zur WWW-Integration (Kapitel 8).

9.2 Ausblick

Die in diesem Buch entwickelte Theorie eröffnet eine Reihe weiterführender Forschungs- und Anwendungsrichtungen, die im Folgenden ausführlich diskutiert werden.

9.2.1 Automatisierte Schemaanalyse und Refactoring-Werkzeuge

Beispiel 5.1 hat gezeigt, dass der Fremdschlüsselgraph G_S eines beliebigen relationalen Schemas vollständig automatisiert aus dem Systemkatalog extrahiert werden kann. Damit lassen sich die in Definition 8.6 eingeführten Maße $depth(G_S)$, $breadth(G_S)$ und $\tau(G_S)$ (Tiefen-Breiten-Verhältnis) für jedes existierende Datenbankschema berechnen. Ein naheliegendes und praktisch hochrelevantes Forschungsvorhaben besteht in der Entwicklung eines *Schema-Linters*, der

- für jede Relation R eines Schemas S den *lokalen Sterngrad* $\Delta(R)$ (Anzahl eingehender Fremdschlüsselkanten) berechnet und Relationen mit $\Delta(R)$ oberhalb eines konfigurierbaren Schwellwerts als *Hub-Kandidaten* markiert,
- für jeden Hub-Kandidaten automatisiert prüft, ob unter den auf ihn verweisenden Spoke-Relationen transitive funktionale Abhängigkeiten gemäß Theorem 8.1 bestehen, die eine weitere Zerlegung nahelegen,
- auf Basis dieser Analyse *Refactoring-Vorschläge* in Form von konkreten ALTER TABLE- und CREATE TABLE- Anweisungen generiert, welche die betroffene Hub-Relation entlang der erkannten transitiven Abhängigkeiten in einen Tiefenpfad zerlegen, unter Wahrung der in Definition 4.1 und Definition 4.2 geforderten Eigenschaften (Verlustfreiheit und Abhängigkeitserhaltung),
- begleitend dazu CREATE VIEW-Anweisungen erzeugt, welche die ursprüngliche, von Anwendungen erwartete flache Sicht auf die Hub-Relation als View über den neu erzeugten Tiefenpfad rekonstruieren – im Sinne der in Abschnitt 3.4 („Virtuelle Tabellen“) diskutierten Kompensation der scheinbaren Bequemlichkeit von Sternschemata durch materialisierte Pfade.

Ein solches Werkzeug würde die in dieser Arbeit rein formal geführte Argumentation in eine praktisch anwendbare Methodik überführen und ließe sich insbesondere in CI/CD-Pipelines für Datenbankmigrationsskripte integrieren, sodass jede neue Migration automatisch auf eine Verschlechterung von $\tau(G_S)$ geprüft wird, bevor sie in Produktion übernommen wird.

9.2.2 Erweiterung auf NoSQL- und NewSQL-Systeme

Die in Kapitel 8.10 entwickelten Maße *depth*, *breadth* und τ sind rein graphentheoretischer Natur und setzen nicht zwingend ein relationales Datenmodell voraus. Eine vielversprechende Erweiterungsrichtung besteht darin, das tiefenorientierte Entwurfsparadigma auf dokumentenorientierte (z. B. MongoDB), spaltenorientierte (z. B. Cassandra) und graphbasierte (z. B. Neo4j) Datenbanksysteme zu übertragen:

- In dokumentenorientierten Systemen entspricht die *Einbettung* (Embedding) verwandter Dokumente einer Denormalisierung im Sinne von Proposition 8.5, während *Referenzierung* zwischen Collections einer Fremdschlüsselkante im Sinne von Definition 2.4 entspricht. Es stellt sich die Frage, ob sich Theorem 8.4 (Anomaliefreiheit durch Tiefenpfade) sinngemäß auf Referenzierungsstrukturen in Dokumentenmodellen übertragen lässt, und unter welchen Bedingungen Embedding dennoch – als bewusste, lokal begrenzte Ausnahme – gerechtfertigt ist.
- In graphbasierten Systemen ist der Fremdschlüsselgraph G_S nicht mehr nur ein analytisches Hilfsmittel, sondern die primäre Datenstruktur selbst. Hier wäre zu untersuchen, ob die in Abschnitt 8.18 bewiesene Asymmetrie von DFS und BFS (Theorem 8.15) auch für die Effizienz von Graphdatenbankabfragen (Cypher, Gremlin) quantitativ nachweisbar ist, insbesondere im Hinblick auf Pfadabfragen entlang tiefer Ketten im Vergleich zu sternförmigen Nachbarschaftsabfragen.
- In NewSQL-Systemen mit verteilter Speicherung (z. B. CockroachDB, Spanner) hat die Topologie des Fremdschlüsselgraphen direkten Einfluss auf die *Sharding-Strategie*: Tiefenpfade legen eine natürliche, hierarchische Partitionierung (Co-Location verwandter Tupel auf demselben Shard) nahe, während Sternschemata mit einem zentralen Hub zu einem *Hot-Spot*-Shard führen können. Eine formale Analyse des Zusammenhangs zwischen $\tau(G_S)$ und der Lastverteilung über Shards ist eine vielversprechende Forschungsrichtung.

9.2.3 Quantitative empirische Validierung

Die in dieser Arbeit geführten Beweise sind rein formal-mathematischer Natur. Eine konsequente Weiterführung besteht in einer breit angelegten empirischen Studie, die

1. eine repräsentative Stichprobe realer, produktiver Datenbankschemata aus offenen Datenquellen (z. B. Open-Source-Projekten auf GitHub/GitLab) hinsichtlich $depth(G_S)$, $breadth(G_S)$ und $\tau(G_S)$ vermisst,
2. für jedes Schema die tatsächliche Verteilung von Anfragelaufzeiten (gemäß Theorem 8.7) und Schreiblatenzen (gemäß der Schreibkostenproposition in Abschnitt 8.15) unter realistischer Last misst,
3. die Korrelation zwischen $\tau(G_S)$ und qualitativen Software-Engineering-Metriken untersucht, etwa der Häufigkeit von Schema-Migrationen (als Proxy für die in Theorem 8.9 formalisierte Schemaevolutionskosten), der Anzahl von Bugtickets, die auf Dateninkonsistenzen zurückzuführen sind (als Proxy für Anomaliefreiheit gemäß Theorem 8.4), sowie der Onboarding-Zeit neuer Entwickler (als Proxy für Verständlichkeit und Testbarkeit, Abschnitt 8.16).

Eine solche Studie würde es erlauben, die in dieser Arbeit rein strukturell-formal hergeleiteten Vorteile tiefenorientierter Schemata mit konkreten, in der Praxis beobachtbaren Effektgrößen zu unterlegen und damit den Übergang von einem theoretischen Gütekriterium zu einer empirisch validierten Entwurfsrichtlinie zu vollziehen.

9.2.4 Integration in den Datenbankoptimierer

Abschnitt 8.15 hat gezeigt, dass moderne Abfrageoptimierer durch Index-Only Scans, Hash-Joins, materialisierte Views und Predicate Pushdown den theoretisch höheren Join-Aufwand tiefer Pfade (Proposition 8.6) kompensieren können. Eine konsequente Weiterentwicklung bestünde darin, dem Optimierer *explizites Wissen* über die Tiefenpfadstruktur des Schemas zur Verfügung zu stellen, etwa in Form eines Schema-Annotationssystems, das für jede Relation R_i ihre Tiefe i im Fremdschlüsselgraphen G_S als Metadatum im Systemkatalog (Abschnitt 5.4) hinterlegt. Der Optimierer könnte diese Information nutzen, um:

- bei mehrstufigen Verbunden entlang eines Tiefenpfads die Verbundreihenfolge so zu wählen, dass zuerst die Relation mit der höchsten Tiefe (typischerweise der kleinsten Kardinalität) als Build-Seite eines Hash-Joins gewählt wird,
- für Abfragen, die ausschließlich Attribute einer Relation R_d mit $d > 0$ filtern (vgl. Theorem 8.7), automatisch zu erkennen, dass kein Zugriff auf R_0 (die Wurzel des Tiefenpfads) erforderlich ist, und den entsprechenden Verbund vollständig zu eliminieren (Join-Elimination),
- für CASCADE-Operationen (Abschnitt 3, Datenmodifikationen) den erwarteten Propagationsaufwand entlang des Tiefenpfads vorab zu schätzen und bei Überschreitung eines Schwellwerts eine Bestätigung des Datenbankadministrators einzufordern.

9.2.5 Lehre und curriculare Integration

Schließlich hat dieses Buch – in Anlehnung an die didaktische Konzeption von Kleinschmidt und Rank [11] – gezeigt, dass sich ein anspruchsvolles formales Entwurfsprinzip vollständig in den klassischen Kanon einer einführenden Datenbankvorlesung integrieren lässt, ohne dass dafür zusätzliche Vorlesungszeit in Form eines separaten Theoriekapitels erforderlich wäre. Stattdessen wurde das Prinzip *horizontal* durch alle Kapitel hindurch verwoben: bei der Diskussion von Views (Kapitel 3), bei der Schlüsseldefinition (Kapitel 4), bei Sperrgranularität und Zugriffsrechten (Kapitel 5), bei Triggern (Kapitel 6), bei der Modularisierung von Anwendungscode (Kapitel 7) und bei der Architektur von WWW-Anwendungen (Kapitel 8).

Ein naheliegender nächster Schritt für die Lehre besteht darin, diese horizontale Integration durch ein *durchgängiges Projekt* zu ergänzen: Studierende entwerfen zu Beginn des Semesters ein eigenes Sternschema für eine selbstgewählte Domäne, berechnen *depth*, *breadth* und τ gemäß Definition 8.6, identifizieren transitive Abhängigkeiten gemäß Theorem 8.1 und führen im Verlauf des Semesters – parallel zur Behandlung der entsprechenden Kapitel – ein vollständiges Refactoring zu einem Tiefenpfadschema

durch, einschließlich SQL-DDL (Kapitel 4), PL/SQL-Triggern (Kapitel 6) und einer einfachen WWW-Oberfläche (Kapitel 8). Die am Ende des Semesters erreichten Werte von $\tau(G_S)$ vor und nach dem Refactoring liefern dabei ein unmittelbar greifbares, quantitatives Lernziel, das die in diesem Buch entwickelte Theorie mit der praktischen Entwurfskompetenz der Studierenden verbindet.

9.2.6 Herstellerspezifische Erweiterungen am Beispiel DISTINCT ON: Spannungsfeld zu Portabilität und Tiefenpfadarchitektur

Abschnitt 3.5 hat mit DISTINCT ON ein Sprachmittel formal untersucht, das – anders als die in Kapitel 7 betonte Portabilität (Abschnitt 7.5) – *nicht* Teil des SQL-Standards ist, sondern eine Eigenheit von PostgreSQL darstellt. Dies wirft eine Reihe von Fragen auf, die über den Rahmen dieses Buches hinausweisen und die im Folgenden als Ausblick skizziert werden.

Portabilitätsanalyse für tiefenpfadtypische Anfragemuster. Theorem 3.1 hat gezeigt, dass DISTINCT ON semantisch äquivalent zu einer ROW_NUMBER()-basierten Filterung ist, die wiederum Standard-SQL ist und auf praktisch allen relationalen DBMS (einschließlich der in Abschnitt 2.5 und Kapitel 7 diskutierten Systeme) verfügbar ist. Eine naheliegende Erweiterung dieser Arbeit bestünde darin, das in Abschnitt 7.7.1 eingeführte Repository-Pattern um eine *Übersetzungsschicht* zu ergänzen, die für jede Kante (R_i, R_{i+1}) des Tiefenpfads, an der ein Gruppenrepräsentantenproblem auftritt (Definition 3.2), automatisch zwischen der PostgreSQL-Formulierung mittels DISTINCT ON und der portablen ROW_NUMBER()-Formulierung umschalten kann – je nachdem, welches DBMS die Anwendung gemäß Abschnitt 7.5 ansteuert. Eine solche Übersetzungsschicht ließe sich systematisch aus der in Bemerkung 3.3 formalisierten Syntaxregel (Präfixbedingung zwischen DISTINCT ON-Liste und ORDER BY-Liste) ableiten, da diese Regel bereits die exakte Korrespondenz zwischen G (der PARTITION BY-Liste von ROW_NUMBER()) und $(B_1, \dots, B_q)$ (der ORDER BY-Liste innerhalb der Partition) festlegt.

Erweiterung des Schema-Linters um DISTINCT ON-Indexempfehlungen. Der in Abschnitt 9.2.1 skizzierte Schema-Linter ließe sich um eine weitere Analyseklasse erweitern: Für jede Kante (R_i, R_{i+1}) des Fremdschlüsselgraphen G_S könnte das Werkzeug den Anwendungs-Quellcode (bzw. ein Anfrageprotokoll, vgl. `pg_stat_statements`) nach Anfragen der Form `DISTINCT ON (F) ... ORDER BY F, B1, ..., Bq` durchsuchen, wobei $F \subseteq \text{attr}(R_i)$ der Fremdschlüssel auf R_{i+1} ist. Wird ein solches Muster erkannt, aber kein zusammengesetzter Index $(F, B_1, \dots, B_q)$ auf R_i gefunden, so kann das Werkzeug – analog zu Beispiel 3.2 und gestützt auf Proposition 3.2 – automatisiert eine CREATE INDEX-Anweisung vorschlagen, die den Sort-Schritt des Ausführungsplans eliminiert und die asymptotischen Kosten von $O(|R_i| \log |R_i|)$ auf $O(|\pi_F(R_i)| + \log |R_i|)$ reduziert.

Allgemeineres Prinzip: Herstellererweiterungen als „Kompressionen“ von Standardmustern. Die in diesem Buch an DISTINCT ON demonstrierte Methodik – formale Definition (Definition 3.3), Nachweis der Äquivalenz zu einem Standard-SQL-Konstrukt (Theorem 3.1) und anschließende Effizienzanalyse im Kontext des Tiefenpfadparadigmas (Proposition 3.2, Bemerkung 3.4) – ist nicht auf DISTINCT ON beschränkt. Zahlreiche weitere herstellerepezifische Erweiterungen – etwa LATERAL-Joins, FILTER-Klauseln für Aggregatfunktionen, oder INSERT ... ON CONFLICT („Upsert“) –

lassen sich nach demselben Schema untersuchen: Zunächst die Formalisierung des gelösten Problems als algebraische bzw. mengentheoretische Definition (analog zu Definition 3.2), dann der Nachweis der Äquivalenz zu einer (meist umständlicheren) Standard-SQL-Formulierung, und schließlich die Untersuchung, an welchen Stellen des Fremdschlüsselgraphen G_S eines Tiefenpfadschemas diese Erweiterung den größten Effizienz- oder Lesbarkeitsgewinn erbringt. Eine systematische, nach diesem Schema aufgebaute „Erweiterungs-Bibliothek“ für tiefenpfadorientierte Schemata wäre eine wertvolle Ergänzung sowohl für die Lehre (Abschnitt 9.2.5) als auch für die in Abschnitt 9.2.1 und hier skizzierten Tooling-Vorhaben.

9.2.7 Temporale Datenbanken und die zeitliche Dimension des Tiefenpfads

Sämtliche bisherigen Betrachtungen dieses Buches gingen implizit von einem *Schnappschuss*-Modell aus: Eine Relation $\mathcal{R}$ repräsentiert den aktuellen Zustand der Welt, und frühere Zustände werden – wenn überhaupt – über separate Audit- oder Historientabellen (Abschnitt 6, Trigger-Beispiel) nachgehalten. Die Datenbanktheorie kennt jedoch mit den *temporalen Datenbanken* einen eigenständigen, gut entwickelten Zweig, der die Zeit als Erstklassenattribut in das Relationenmodell integriert. Eine ausführliche Weiterführung dieses Buches in Richtung temporaler Datenbanken ist aus mehreren Gründen naheliegend.

Bitemporale Erweiterung des Tiefenpfadschemas. In der temporalen Datenbanktheorie unterscheidet man zwischen *Gültigkeitszeit* (valid time, der Zeitraum, in dem ein Faktum in der modellierten Welt zutrifft) und *Transaktionszeit* (transaction time, der Zeitraum, in dem ein Faktum in der Datenbank als bekannt gespeichert war). Eine *bitemporale* Relation R führt zu jedem Tupel t zwei Intervalle $[t_{vs}, t_{ve})$ (valid time) und $[t_{ts}, t_{te})$ (transaction time) mit. Für ein Tiefenpfadschema $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ stellt sich die Frage, wie sich diese zeitliche Dimension mit der Tiefenstruktur verträgt:

- **Definition eines zeitparametrisierten Fremdschlüsselgraphen.** Man definiere $G_S(\tau) = (V, E_\tau)$ als denjenigen Fremdschlüsselgraphen, der entsteht, wenn man jede Relation R_i auf ihren zum Zeitpunkt τ gültigen Schnappschuss $R_i(\tau) = \{t \in R_i \mid t_{vs} \leq \tau < t_{ve}\}$ einschränkt. Eine zentrale, in dieser Arbeit unbeantwortete Frage lautet: Bleibt die Tiefe $depth(G_S(\tau))$ für alle τ konstant, oder kann sich die *Topologie* des Fremdschlüsselgraphen über die Zeit ändern (etwa durch *Reparenting*, also die nachträgliche Änderung eines Fremdschlüsselwerts, z. B. wenn ein Kunde einer anderen Region zugeordnet wird)? Falls ja, müsste Definition 8.6 ($depth$, $breadth$, $\tau(G)$) zu zeitabhängigen Funktionen $depth(\tau)$, $breadth(\tau)$, $\tau_{Verh}(\tau)$ erweitert werden, und Theorem 8.1 (Transitivitätsverteilung) müsste für *zeitlich variable funktionale Abhängigkeiten* – sogenannte *temporale funktionale Abhängigkeiten* (TFDs) – neu bewiesen werden.
- **Slowly Changing Dimensions als getarnte Tiefenpfaderweiterung.** Im Data-Warehouse-Bereich (Kimball-Methodik, vgl. Definition 8.7 und [22]) ist das Muster der *Slowly Changing Dimension* (SCD) vom Typ 2 weit verbreitet: Anstatt ein Dimensionsattribut bei einer Änderung zu überschreiben, wird ein neues Tupel mit einem neuen Gültigkeitsintervall angelegt, und das alte Tupel wird mit einem

Enddatum versehen. Eine bemerkenswerte Beobachtung – die in einer Erweiterung dieser Arbeit formal auszuarbeiten wäre – ist, dass SCD2 *strukturell* eine *Tiefenerweiterung* im Sinne von Korollar 8.2 darstellt: Die ursprüngliche funktionale Abhängigkeit $K \rightarrow A$ (z. B. `kunden_id` $\rightarrow$ `region_id) wird durch die Einführung der Zeitdimension zu einer zeitparametrisierten Abhängigkeit $(K, \tau) \rightarrow A$, was bedeutet, dass (K, τ) – nicht mehr K allein – der (zusammengesetzte) Schlüssel der Relation ist. Damit wird aus einer scheinbar „flachen“ Dimensionstabelle implizit ein Tiefenpfad der Form KUNDE_VERSION $\rightarrow$ KUNDE, bei dem KUNDE die zeitinvarianten Attribute (z. B. die natürliche Schlüssel-ID) und KUNDE_VERSION die zeitveränderlichen Attribute samt Gültigkeitsintervall enthält. Eine vollständige formale Ausarbeitung dieser Beobachtung würde Theorem 8.1 um einen Fall „transitiver Abhängigkeit über die Zeitachse“ erweitern und damit zeigen, dass auch die zeitliche Dimension dem Prinzip „Tiefe vor Breite“ unterliegt.`

- **DISTINCT ON als natürliches Werkzeug für temporale Schnappschüsse.** Abschnitt 3.5 hat `DISTINCT ON` als Lösung des Gruppenrepräsentantenproblems (Definition 3.2) eingeführt. Für eine bitemporale Relation R_i mit Transaktionszeitintervallen liefert

```
SELECT DISTINCT ON (K) * FROM Ri
WHERE tvs ≤ τ AND τ < tve
ORDER BY K, tts DESC
```

für jeden Schlüsselwert K genau die zum Transaktionszeitpunkt τ *aktuellste* Version des zum Gültigkeitszeitpunkt τ gültigen Tupels – eine direkte Anwendung von Theorem 3.1 auf das bitemporale Modell. Eine ausführliche Weiterentwicklung dieses Buches könnte einen eigenen Abschnitt „Bitemporale Anfragen mit `DISTINCT ON` und Bereichstypen (`tstzrange`)“ enthalten, der PostgreSQLs native Unterstützung für Intervalltypen und Exclusion-Constraints (`EXCLUDE USING gist`) mit der in Bemerkung 3.4 entwickelten Indexstrategie kombiniert.

- **SQL:2011-Standard für temporale Tabellen.** Der SQL:2011-Standard (vgl. Abschnitt 3.2) definiert `PERIOD FOR`-Klauseln sowie `SYSTEM_VERSIONING` für transaktionszeit-versionierte Tabellen. Eine ausführliche Behandlung dieses Standards – einschließlich der Frage, wie sich Theorem 7.3 (Eindeutigkeit der Repository-Zerlegung) auf system-versionierte Tabellen überträgt, bei denen *jede* Lesezugriffsmethode implizit um einen `AS OF`-Zeitpunkt parametrisiert werden muss – wäre ein eigenständiges, umfangreiches Kapitel wert.

9.2.8 Verteilte Transaktionen, Konsistenzmodelle und der Tiefenpfad in Microservice-Architekturen

Kapitel 5 hat das Transaktionskonzept innerhalb *eines* DBMS behandelt. In modernen Microservice-Architekturen ist es jedoch üblich, dass unterschiedliche Teile eines ehemals monolithischen Schemas $\mathcal{S}$ auf unterschiedliche Datenbankinstanzen – mit unterschiedlichen DBMS-Produkten – aufgeteilt werden (*Database-per-Service*-Muster). Dies wirft fundamentale Fragen auf, die eine erhebliche Erweiterung dieser Arbeit rechtfertigen.

Schnitte durch den Fremdschlüsselgraphen. Sei $G_S = (V, E)$ der Fremdschlüsselgraph des Gesamtschemas. Eine Aufteilung auf p Microservices entspricht einer Partition $V = V_1 \dot{\cup} \dots \dot{\cup} V_p$. Jede Kante $(R_i, R_j) \in E$ mit $R_i \in V_a$, $R_j \in V_b$, $a \neq b$, wird zu einer *verteilten Fremdschlüsselbeziehung*, deren referentielle Integrität (Definition 2.3) nicht mehr durch das DBMS selbst, sondern nur noch durch Anwendungslogik (z. B. Saga-Pattern, Event-Sourcing mit Kompensationstransaktionen) sichergestellt werden kann.

- **Tiefenpfad-erhaltende Schnitte.** Eine naheliegende Designhypothese – die in einer Erweiterung dieser Arbeit formal zu untersuchen wäre – lautet: Schnitte durch G_S , die *entlang* eines Tiefenpfads verlaufen (d. h. jede Schnittkante (R_i, R_{i+1}) trennt zwei aufeinanderfolgende Knoten eines Pfades), erzeugen *weniger* verteilte Fremdschlüsselbeziehungen pro Microservice als Schnitte, die einen Hub-Knoten eines Sternschemas (Definition 8.7) durchtrennen – da ein Hub-Knoten H mit $\deg(H) = k$ bei einer Aufteilung, die H von $m < k$ seiner Spokes trennt, sofort m verteilte Fremdschlüsselbeziehungen erzeugt, während ein Schnitt entlang eines Tiefenpfads stets nur *eine* Kante durchtrennt, unabhängig von der Pfadlänge l . Ein formaler Beweis dieser Hypothese, etwa in Form eines Theorems „Tiefenpfad-Schnitte minimieren die Anzahl verteilter Fremdschlüsselbeziehungen pro Partitions-grenze“, wäre ein bedeutender Brückenschlag zwischen der in Kapitel 8.10 entwickelten Theorie und der Praxis verteilter Systeme.
- **Sagas als verteilte Tiefenpfadtraversierung.** Das *Saga-Pattern* ersetzt eine klassische ACID-Transaktion (Kapitel 1) durch eine Folge lokaler Transaktionen $T_1, \dots, T_n$ mit zugehörigen Kompensationstransaktionen $C_1, \dots, C_n$, sodass im Fehlerfall $C_{n-1}, \dots, C_1$ in umgekehrter Reihenfolge ausgeführt werden. Für eine Operation, die einen Tiefenpfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ traversiert (z. B. „lege einen neuen Auftrag mit Positionen an, reserviere Lagerbestand, initiiere Zahlung“), entspricht jeder Schritt T_i einer lokalen Transaktion auf demjenigen Microservice, der R_i verwaltet. Eine ausführliche Untersuchung könnte zeigen, dass die *Reihenfolge* der Saga-Schritte $T_1, \dots, T_l$ und die Reihenfolge der Knoten im Tiefenpfad $R_1, \dots, R_l$ übereinstimmen sollten, damit die Kompensationsreihenfolge $C_l, \dots, C_1$ exakt der *umgekehrten* Fremdschlüsselrichtung entspricht – d. h. Kompensationen propagieren von „Kindknoten“ zu „Elternknoten“ des Tiefenpfads, analog zu ON DELETE CASCADE (Abschnitt 3.4), jedoch auf Anwendungsebene und über Servicegrenzen hinweg.
- **CAP-Theorem und lokale vs. globale Konsistenz entlang des Pfads.** Das CAP-Theorem (Konsistenz, Verfügbarkeit, Partitionstoleranz – es können höchstens zwei der drei Eigenschaften gleichzeitig garantiert werden) erzwingt in verteilten Systemen häufig den Übergang zu *eventueller Konsistenz*. Eine umfangreiche Erweiterung dieses Buches könnte untersuchen, ob sich entlang eines Tiefenpfads $R_1 \rightarrow \dots \rightarrow R_l$ eine *abgestufte* Konsistenzgarantie formulieren lässt: Knoten R_1 (z. B. Stammdaten wie LAND, REGION) ändern sich selten und können stark konsistent (mit Lese-Repliken) repliziert werden, während Knoten R_l (z. B. AUFTRAGSPOSITION) hochfrequent geschrieben werden und eventuelle Konsistenz tolerieren. Die Frage, ob ein *Konsistenzgradient* entlang der Tiefe

$depth(G_S)$ – von „stark konsistent“ an der Wurzel zu „eventuell konsistent“ an den Blättern – eine allgemeingültige Entwurfsregel darstellt, wäre ein eigenständiges Forschungsprogramm.

9.2.9 Sicherheits- und Datenschutzaspekte: Verschlüsselung, Mandantenfähigkeit und Anonymisierung entlang des Tiefenpfads

Abschnitt 5.3 hat die klassische SQL-Rechteverwaltung (**GRANT/REVOKE**) im Kontext des Tiefenpfadparadigmas behandelt. Eine sehr viel umfangreichere Behandlung von Sicherheits- und Datenschutzaspekten wäre für eine erweiterte Auflage dieses Buches angezeigt, insbesondere vor dem Hintergrund der Datenschutz-Grundverordnung (DSGVO) und vergleichbarer Regelwerke.

Spaltenverschlüsselung und Tiefenlokalität. Werden einzelne Attribute (z. B. **KUNDE.name**, **KUNDE.adresse**) anwendungsseitig oder transparent (Transparent Data Encryption, TDE) verschlüsselt, so stellt sich die Frage, wie sich dies mit den in Abschnitt 8.15 bewiesenen Selektivitätsvorteilen (Theorem 8.7) verträgt: Ein Index auf einer verschlüsselten Spalte ist im Allgemeinen nicht mehr ordnungserhaltend nutzbar, sodass **DISTINCT ON**-Indizes (Abschnitt 3.5.4) auf verschlüsselten Spalten ihre Effizienzvorteile verlieren können. Eine ausführliche Untersuchung könnte systematisch klassifizieren, welche Attribute eines Tiefenpfadschemas (Schlüssel, Fremdschlüssel, Sortier-/Filterattribute, rein darstellende Attribute) für welche Verschlüsselungsklasse (deterministisch, randomisiert, ordnungserhaltend, homomorph) geeignet sind, und dabei explizit auf die Rolle jedes Attributs im Fremdschlüsselgraphen G_S Bezug nehmen: Fremdschlüsselattribute (Definition 2.3) müssen z. B. *deterministisch* verschlüsselt werden, damit der kanonische Verbund (Definition 2.5) weiterhin auf Gleichheit prüfen kann, während rein darstellende Attribute (z. B. **name**) randomisiert verschlüsselt werden können.

Mandantenfähigkeit (Multi-Tenancy) als zusätzliche Tiefenstufe. In Software-as-a-Service-Anwendungen teilen sich mehrere Mandanten (Tenants) dieselbe Datenbankinstanz. Die gängigen Architekturmuster – *Shared Database*, *Shared Schema* (**tenant_id** als zusätzliche Spalte in jeder Tabelle), *Shared Database, Separate Schemas* und *Separate Databases* – lassen sich im Lichte des Tiefenpfadparadigmas wie folgt einordnen: Im Muster „Shared Schema“ wird **tenant_id** zu einem zusätzlichen, *universellen* Fremdschlüsselattribut, das in *jeder* Relation $R_i \in \mathcal{S}$ vorkommt und auf eine zentrale MANDANT-Relation verweist. Formal entsteht damit ein neuer Knoten MANDANT in G_S mit einer Kante zu *jedem* anderen Knoten – also $\deg(\text{MANDANT}) = |V| - 1$, was MANDANT nach Definition 8.7 zu einer extremen Hub-Relation macht. Eine ausführliche Behandlung müsste klären, ob dieser scheinbare Widerspruch zur in Korollar 8.2 geforderten Tiefenorientierung tatsächlich einen Widerspruch darstellt, oder ob MANDANT als *orthogonale* Dimension (eine Art „globale Wurzel“, die *vor* dem eigentlichen Tiefenpfad jedes Mandanten steht) aufzufassen ist – mit der Konsequenz, dass für *jeden* einzelnen Mandanten der durch **tenant_id**=const gefilterte Teilgraph von G_S wieder ein reiner Tiefenpfad ist und MANDANT selbst außerhalb der in Theorem 7.3 betrachteten Repository-Zerlegung steht (jedes Repository-Modul M_i filtert implizit nach **tenant_id**, ohne dass dies die Modulgrenzen verändert).

Anonymisierung, Pseudonymisierung und das Recht auf Löschung. Art. 17 DSGVO („Recht auf Löschung“) verlangt, dass personenbezogene Daten auf Anfrage vollständig entfernt werden. Im Tiefenpfadschema (z. B. $\text{LAND} \leftarrow \text{REGION} \leftarrow \text{KUNDE} \leftarrow \text{AUFTRAG} \leftarrow \text{AUFTRAGSPOSITION}$) stellt sich die Frage, wie eine DELETE-Operation auf KUNDE (Abschnitt 3.4, Datenmodifikationen) mit den betriebswirtschaftlichen und gegebenenfalls handels- bzw. steuerrechtlichen Aufbewahrungspflichten für AUFTRAG-Daten in Einklang zu bringen ist. Eine umfangreiche Erweiterung dieses Buches könnte ein formales Modell der „Löschbarkeitstiefe“ entwickeln: Für jeden Knoten R_i eines Tiefenpfads wird eine Funktion $\ell(R_i) \in \{\text{sofort, nach Frist } d_i, \text{nie}\}$ definiert, und es wird untersucht, unter welchen Bedingungen an ℓ die referentielle Integrität (Definition 2.3) entlang des Pfads aufrechterhalten werden kann, wenn ein „Löschen“ tatsächlich als *Pseudonymisierung* (Ersetzen personenbezogener Attribute durch Platzhalterwerte, unter Beibehaltung des Primärschlüssels und damit aller Fremdschlüsselbeziehungen) realisiert wird. Dies stünde in direktem Bezug zu Theorem 8.4 (Anomaliefreiheit durch Tiefenpfade): Da im Tiefenpfadschema jedes Attribut genau einmal gespeichert ist, betrifft eine Pseudonymisierungsoperation auf KUNDE *ausschließlich* die Relation KUNDE – eine direkte Anwendung der in Theorem 8.9 bewiesenen Lokalität, übertragen von Schemaänderungen auf Datenänderungsoperationen mit Compliance-Charakter.

9.2.10 Formale Verifikation von Schemaeigenschaften mit Beweisassistenten

Sämtliche Theoreme dieses Buches (z. B. Theorem 8.1, Theorem 4.1, Theorem 8.4, Theorem 4.3, Theorem 3.1) wurden in der für die mathematische Fachliteratur üblichen Form „informeller, aber rigoroser“ Beweise (vgl. [29]) geführt. Eine weitreichende und methodisch anspruchsvolle Erweiterung dieser Arbeit bestünde darin, diese Beweise in einem interaktiven Beweisassistenten – etwa Coq, Isabelle/HOL oder Lean – zu formalisieren und maschinell zu verifizieren.

- **Formalisierung des Relationenmodells.** Als Ausgangspunkt müsste das Relationenmodell (Definition 2.1) als abhängiger Typ formalisiert werden: Eine Relation über dem Schema $(A_1:D_1, \dots, A_n:D_n)$ entspricht einer endlichen Menge (bzw. einem endlichen Multimenge-Typ, falls Duplikate vor Anwendung von DISTINCT, vgl. Definition 3.1, zugelassen werden) von n -Tupeln über den entsprechenden Trägertypen. Funktionale Abhängigkeiten (Abschnitt 8.13) und die Hüllenoperation X_Σ^+ (Definition 4.3) ließen sich als induktiv definierte Prädikate bzw. als Fixpunkt einer monotonen Funktion auf der Potenzmenge der Attribute formalisieren – Letzteres ist besonders attraktiv, da Beweisassistenten über ausgereifte Bibliotheken für Fixpunkttheorie auf vollständigen Verbänden verfügen, was einen maschinenüberprüften Beweis der Terminierung der HUELLE-Funktion aus Algorithmus 4.1 ermöglichen würde.
- **Maschinenüberprüfter Beweis von Theorem 4.1.** Das Lossless-Join-Kriterium ist ein zentrales technisches Lemma, auf dem die gesamte Dekompositionstheorie (Abschnitt 4.3) aufbaut. Eine Formalisierung müsste den natürlichen Verbund $\bowtie$ (Abschnitt 2.4) als Operation auf den oben eingeführten Relationstypen definieren und die im Beweis von Theorem 4.1 verwendete Tupelkonstruktion

(insbesondere die Existenz „verirrter Tupel“ – spurious tuples – im Falle der Notwendigkeitsrichtung) explizit als Gegenbeispiel-Konstruktion formalisieren. Eine erfolgreiche Formalisierung würde die Korrektheit von Algorithmus 4.2 (Synthesealgorithmus) auf eine *maschinenprüfbare* Grundlage stellen.

- **Extraktion eines zertifizierten Schema-Linters.** Beweisassistenten wie Coq erlauben die *Extraktion* von verifizierten Algorithmen in ausführbaren Code (z. B. OCaml oder Haskell). Eine weitreichende praktische Konsequenz einer formalen Verifikation von Algorithmus 4.1 und Algorithmus 4.2 wäre die Möglichkeit, den in Abschnitt 9.2.1 skizzierten Schema-Linter als *zertifizierten* Algorithmus zu extrahieren, dessen Korrektheit (im Sinne von Theorem 4.3 und Theorem 4.4) nicht nur durch Testfälle, sondern durch einen maschinenüberprüften Beweis garantiert ist – ein Qualitätsniveau, das in der Datenbank-Tooling-Landschaft bislang nicht erreicht wird.
- **Modellprüfung (Model Checking) von Transaktionsschedules.** Die in Kapitel 5 behandelten Konzepte – Serialisierbarkeit, Isolationsstufen, die in Bemerkung 5.1 diskutierte Sperrgranularität entlang des Tiefenpfads – sind klassische Anwendungsgebiete der *Modellprüfung* (model checking) mit Werkzeugen wie TLA+ oder SPIN. Eine umfangreiche Erweiterung könnte für ein konkretes Tiefenpfadschema (z. B. das in Abschnitt 8.19 verwendete $\text{LAND} \leftarrow \text{REGION} \leftarrow \text{KUNDE} \leftarrow \text{AUFTRAG} \leftarrow \text{AUFTRAGSPOSITION}$) ein TLA+-Modell der nebenläufigen Transaktionen entwickeln und formal verifizieren, dass unter **REPEATABLE READ** (Kapitel 5) keine der drei in Abschnitt 4.1 klassifizierten Anomalien auftreten kann – und dabei explizit zeigen, dass die Anzahl der zu prüfenden Zustände (state space) für ein Tiefenpfadschema aufgrund der in Theorem 7.3 bewiesenen Lokalität signifikant kleiner ist als für ein äquivalentes Sternschema – ein weiteres, diesmal aus der Modellprüfung stammendes quantitatives Argument für „Tiefe vor Breite“.

9.2.11 Maschinelles Lernen auf Tiefenpfadschemata

Ein wachsendes Anwendungsfeld relationaler Datenbanken ist die Bereitstellung von Trainingsdaten für maschinelles Lernen, insbesondere für *Graph Neural Networks* (GNNs) und *relationales Deep Learning*, die direkt auf der Struktur G_S operieren, anstatt zunächst eine flache „Feature-Tabelle“ zu erzeugen.

Relationales Deep Learning und der Fremdschlüsselgraph als Berechnungsgraph. Ein GNN definiert für jeden Knoten v eines Graphen eine Repräsentation h_v , die iterativ durch Aggregation der Repräsentationen seiner Nachbarn aktualisiert wird: $h_v^{(k+1)} = f(h_v^{(k)}, \{h_u^{(k)} \mid u \in N(v)\})$. Wendet man dies direkt auf G_S an, so entspricht ein *Knoten* des GNN nicht einer *Relation* R_i , sondern einem *Tupel* $t \in R_i$, und eine *Kante* des GNN entspricht einem konkreten Fremdschlüsselverweis zwischen zwei Tupeln (also einer Instanz des in Definition 2.5 definierten kanonischen Verbunds, auf Tupelebene statt auf Relationenebene). Eine ausführliche Erweiterung dieser Arbeit könnte folgende Fragen behandeln:

- **Empfangsfeld (Receptive Field) und Pfadtiefe.** Nach k GNN-Schichten hängt $h_v^{(k)}$ von allen Tupeln ab, die über einen Pfad der Länge $\leq k$ in G_S von

v aus erreichbar sind. Für ein Tiefenpfadschema mit $\text{depth}(G_S) = l$ bedeutet dies, dass $k = l$ Schichten nötig sind, um Information von der Wurzel bis zu den Blättern (oder umgekehrt) zu propagieren – eine direkte Übertragung der in Proposition 8.6 definierten Join-Tiefe δ in die Sprache der GNN-Architektur: $\delta(\mathcal{Q})$ und die für ein GNN benötigte Schichtzahl k sind für dieselbe semantische „Reichweite“ identisch. Für ein k -Sternschema (Definition 8.7) hingegen genügt bereits *eine* Schicht, um vom Hub H aus alle Spokes zu erreichen – jedoch um den Preis, dass H in dieser einen Schicht Information von k strukturell verschiedenen Entitätstypen aggregieren muss, was in der GNN-Literatur als *Over-Squashing* bekanntes Problem auftritt. Eine formale Untersuchung des Zusammenhangs zwischen $\tau(G_S)$ (Definition 8.6) und dem Over-Squashing-Phänomen wäre ein eigenständiger, publikationswürdiger Forschungsbeitrag.

- **Feature-Engineering entlang von Repository-Grenzen.** Theorem 7.3 hat gezeigt, dass ein Tiefenpfadschema eine eindeutige Zerlegung des Anwendungscodes in Module $M_1, \dots, M_l$ induziert. Eine naheliegende Erweiterung wäre die Übertragung dieser Zerlegung auf die *Feature-Pipeline* eines ML-Systems: Für jeden Knoten R_i existiert ein *Feature-Extraktor* $\phi_i : R_i \rightarrow \mathbb{R}^{d_i}$, und das Gesamtfeature eines AUFTRAG-Tupels für ein Vorhersagemodell (z. B. „Wahrscheinlichkeit einer Rückgabe“) ergibt sich durch Konkatenation $\phi_1(t_1) \oplus \dots \oplus \phi_l(t_l)$ entlang des Pfads von der Wurzel zum betrachteten Tupel. Eine ausführliche Behandlung könnte zeigen, dass diese Pfad-Konkatenation – ähnlich wie die in Abschnitt 7.6.1 diskutierte Pfadtraversierung via Lazy Loading – entweder *eager* (als ein einziger materialisierter Feature-View, vgl. Abschnitt 3.4, „Virtuelle Tabellen“) oder *lazy* (zur Trainingszeit aus den Rohdaten berechnet) realisiert werden kann, mit denselben Kompromissen wie beim $N+1$ -Problem (Bemerkung 7.3).
- **Vektorindizes und approximative Nächste-Nachbarn-Suche als neue Kantenart.** Moderne PostgreSQL-Erweiterungen (*pgvector*) erlauben die Speicherung von Embedding-Vektoren als Spaltentyp und die effiziente Nächste-Nachbarn-Suche mittels Indizes (HNSW, IVF-Flat). Dies eröffnet die Möglichkeit, G_S um eine neue Art von Kante zu erweitern: *semantische Ähnlichkeitskanten* zwischen Tupeln derselben Relation R_i , die nicht durch Fremdschlüssel, sondern durch Ähnlichkeit im Embedding-Raum definiert sind. Eine Erweiterung dieser Arbeit könnte formal untersuchen, ob und wie sich die in Kapitel 8.10 entwickelten Maße *depth*, *breadth*, τ auf einen *hybriden* Graphen übertragen lassen, der sowohl klassische Fremdschlüsselkanten (gemäß Definition 2.4) als auch k -Nächste-Nachbarn-Kanten im Embedding-Raum enthält, und welche Konsequenzen dies für die Indexstrategien aus Abschnitt 3.5.4 hat – etwa die Frage, ob **DISTINCT ON** in Kombination mit einer **ORDER BY embedding <-> :query_vector**-Klausel (Nächste-Nachbarn-Operator von *pgvector*) weiterhin der in Theorem 3.1 bewiesenen Äquivalenz zu **ROW_NUMBER()** genügt.

9.2.12 Hybride Graph-relationale Systeme und alternative Anfragesprachen

Abschnitt 9.2.2 hat bereits angedeutet, dass graphbasierte Systeme (Neo4j u. a.) den Fremdschlüsselgraphen G_S als primäre Datenstruktur behandeln. Eine ausführliche Erweiterung dieser Arbeit könnte einen vollständigen Vergleich der in diesem Buch entwickelten Theorie mit den Anfragesprachen *Cypher*, *Gremlin* und dem neuen ISO-Standard *GQL* (Graph Query Language, ISO/IEC 39075:2024) durchführen. Von besonderem Interesse wäre dabei:

- **Pfadausdrücke als natives Sprachmittel.** Während in SQL ein Tiefenpfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ durch $l - 1$ explizite JOIN-Klauseln (Beispiel 3.1) traversiert werden muss, erlaubt Cypher Pfadausdrücke der Form $(r1)-[:GEHOERT_ZU]->(r2)-[:GEHOERT_ZU]->\dots->(r1)$ direkt als Teil des Anfragemusters. Eine formale Untersuchung könnte zeigen, dass für *reine* Tiefenpfadschemata (Definition 8.8) eine Cypher-Anfrage mit einem Pfadausdruck der Länge $l - 1$ *syntaktisch* prägnanter, aber *semantisch äquivalent* zu der in Abschnitt 3.4 eingeführten SQL-Verbundkette ist – ähnlich der in Theorem 3.1 geführten Äquivalenzbeweise –, während für Sternschemata (Definition 8.7) Cypher-Pfadausdrücke der Form $(h)-[]->(s1)$, $(h)-[]->(s2)$, ... keinen Prägnanzgewinn gegenüber SQL bieten, da kein gemeinsamer Pfad existiert.
- **Property Graphs als Generalisierung des Tiefenpfadschemas.** Im Property-Graph-Modell trägt nicht nur jeder Knoten, sondern auch jede *Kante* Attribute. Dies erlaubt es, $n:m$ -Beziehungen (vgl. Abschnitt 2.3, Transformationsregel 3) ohne ein explizites Verbindungs-Relationenschema zu modellieren: Die Attribute einer $n:m$ -Beziehung (z. B. `AUFTRAGSPOSITION.menge`) werden direkt zu Kantenattributen. Eine ausführliche Untersuchung könnte formalisieren, unter welchen Bedingungen diese Verschiebung von „Verbindungsrelation als Knoten“ zu „Verbindungsrelation als Kante“ die in Proposition 2.1 bewiesene Struktur des Fremdschlüsselgraphen G_S erhält bzw. vereinfacht, und ob sich dadurch die Tiefe $depth(G_S)$ um genau die Anzahl der auf diese Weise „eliminierten“ Verbindungsrelationen reduziert.
- **SQL/PGQ (SQL:2023).** Der jüngste SQL-Standard SQL:2023 führt mit SQL/PGQ (Property Graph Queries) die Möglichkeit ein, *innerhalb* einer relationalen Datenbank eine Graph-Sicht (`CREATE PROPERTY GRAPH`) auf existierende Tabellen zu definieren und mit einer an Cypher angelehnten Syntax (`GRAPH_TABLE`) abzufragen. Dies ist möglicherweise der bedeutendste Standardisierungsschritt seit SQL:1999 (Abschnitt 3.2) für die in diesem Buch entwickelte Theorie, da er erlaubt, denselben Fremdschlüsselgraphen G_S *wahlweise* als relationale Tabellen (klassisches SQL, Kapitel 3) oder als Property Graph (`GRAPH_TABLE`) anzufragen – ohne Datenduplikation. Eine sehr ausführliche Erweiterung dieses Buches sollte SQL/PGQ zum Anlass nehmen, sämtliche in Kapitel 3 eingeführten Beispielfragen (insbesondere Beispiel 3.1 und Beispiel 3.2) in *beiden* Syntaxformen zu präsentieren und empirisch (vgl. Abschnitt 9.2.3) zu vergleichen, welche Formulierung für Tiefenpfad- bzw. Sternschemata vom PostgreSQL-Optimierer (sobald SQL/PGQ dort verfügbar ist) in effizientere Ausführungspläne übersetzt wird.

9.2.13 Ein Forschungsprogramm: Vom Gütekriterium zur Theorie der Schema-Topologie

Die in den vorangegangenen Abschnitten skizzierten Erweiterungen – automatisiertes Tooling (Abschnitt 9.2.1), NoSQL/NewSQL (Abschnitt 9.2.2), empirische Validierung (Abschnitt 9.2.3), Optimierer-Integration (Abschnitt 9.2.4), Lehre (Abschnitt 9.2.5), herstellerspezifische Erweiterungen (Abschnitt 9.2.6), temporale Datenbanken (Abschnitt 9.2.7), verteilte Systeme (Abschnitt 9.2.8), Sicherheit (Abschnitt 9.2.9), formale Verifikation (Abschnitt 9.2.10), maschinelles Lernen (Abschnitt 9.2.11) und hybride Graph-relationale Systeme (Abschnitt 9.2.12) – sind keine unabhängigen Einzelthemen, sondern lassen sich als Facetten *eines einzigen*, übergeordneten Forschungsprogramms auffassen, das hier abschließend skizziert werden soll: einer *allgemeinen Theorie der Schema-Topologie*.

Leitfrage. Die Leitfrage eines solchen Forschungsprogramms lautet: *Existiert für jede relevante Eigenschaft P eines Datenbanksystems (Normalisierungsgrad, Anomaliefreiheit, Anfrageeffizienz, Wartbarkeit, Sperrverhalten, Verteilungseignung, Eignung für maschinelles Lernen, Sicherheitsklassifizierbarkeit, Verifizierbarkeit, ...) eine Funktion $f_P : G_S \rightarrow \mathbb{R}$ (oder $\rightarrow$ einen geordneten Wertebereich), sodass P monoton in $f_P(G_S)$ ist, und ist f_P für alle relevanten P simultan durch dasselbe Strukturmerkmal von G_S – nämlich seine Tiefen-Breiten-Charakteristik $\tau(G_S)$ aus Definition 8.6 – bestimmt?*

Dieses Buch hat für eine Reihe konkreter Eigenschaften P – 3NF (Korollar 8.2), Anomaliefreiheit (Theorem 8.4), Selektivität (Theorem 8.7), Schreibkomplexität, lokale Änderbarkeit (Theorem 8.9), Modulzerlegbarkeit (Theorem 7.3) – jeweils *einzelnen* gezeigt, dass P in $\tau(G_S)$ monoton ist (höheres τ , also „mehr Tiefe relativ zu Breite“, impliziert „besseres P “). Die in diesem Ausblick skizzierten Erweiterungen – insbesondere die temporale Erweiterung von τ zu $\tau(\tau_{\text{Zeit}})$ (Abschnitt 9.2.7), die verteilungsbezogene Interpretation von τ (Abschnitt 9.2.8) und die GNN-Receptive-Field-Interpretation von τ (Abschnitt 9.2.11) – legen nahe, dass τ tatsächlich ein *universelles* Strukturmerkmal sein könnte, das weit über die in diesem Buch behandelten Einzelresultate hinaus als „Ein-Zahl-Zusammenfassung“ der Qualität eines Datenbankschemas dient – vergleichbar der Rolle, die die *Konditionszahl* einer Matrix in der numerischen linearen Algebra spielt: Eine einzelne Kennzahl, die unmittelbar Auskunft über das Verhalten eines Systems unter einer Vielzahl unterschiedlicher Operationen gibt.

Ein vollständiger Beweis dieser „universellen Monotonie-These“ – für *alle* relevanten Eigenschaften P und nicht nur für die in diesem Buch einzeln behandelten – wäre ein Forschungsprogramm, das sich über mehrere Dissertationen erstrecken könnte und das die Datenbanktheorie mit der Graphentheorie (insbesondere der Theorie der Baumweite und verwandter struktureller Graphparameter, die in der algorithmischen Graphentheorie eine ganz ähnliche Rolle spielen wie τ in dieser Arbeit) auf eine neue, einheitliche Grundlage stellen würde. Die vorliegende Arbeit versteht sich als erster, in sich geschlossener Baustein dieses größeren Programms.

9.2.14 Schlussbemerkung

Die Leitidee dieses Buches – *Tiefe vor Breite* – ist sowohl formal beweisbar als auch praktisch handlungsleitend. Die in den vorangegangenen Kapiteln entwickelte Theorie liefert nicht nur eine Erklärung dafür, *warum* normalisierte, tiefenorientierte Schemata

in der Praxis robuster, wartbarer und konsistenter sind, sondern auch ein *Werkzeug* – die Maße *depth*, *breadth* und τ –, mit dem sich diese Eigenschaft für jedes beliebige Schema quantitativ bewerten lässt. Die in Abschnitt 9.2.1 bis 9.2.5 skizzierten Weiterführungen zeigen, dass dieses Werkzeug weit über den Rahmen einer einführenden Darstellung hinaus Anwendung finden kann – von automatisierten Refactoring-Werkzeugen über verteilte NewSQL-Architekturen bis hin zur Gestaltung datenbankzentrierter Curricula.

Anhang A

Syntaxübersicht und ergänzende Materialien

A.1 Übersicht zentraler SQL-Sprachelemente

Tabelle A.1 fasst die in diesem Buch verwendeten zentralen SQL-Sprachelemente und ihren Bezug zu den entsprechenden Kapiteln zusammen.

Sprachelement	Kapitel	Bezug zum Tiefenpfadparadigma
CREATE TABLE ...REFERENCES	4	Definiert eine Kante im Fremdschlüsselgraphen G_S
SELECT ...JOIN	3	Traversiert eine oder mehrere Kanten von G_S
CREATE VIEW	3	Materialisiert einen Pfad in G_S als flache Sicht
ON DELETE CASCADE	3, 5	Propagation entlang einer Kante von G_S
GRANT/REVOKE	5	Entitätspräzise Rechtevergabe pro Knoten von G_S
CREATE TRIGGER	6	An genau einen Knoten von G_S gebundene Logik
information_schema	5	Quelle zur automatisierten Rekonstruktion von G_S

Tabelle A.1: Zentrale SQL-Sprachelemente und ihr Bezug zum Fremdschlüsselgraphen

A.2 Glossar der formalen Symbole

Symbol	Bedeutung
$\mathcal{R}, R$	Relation bzw. Relationenschema
$attr(R)$	Attributmenge von R
Σ_R	Menge der Integritätsbedingungen von R
$K \rightarrow A$	Funktionale Abhängigkeit: K bestimmt A
$X \twoheadrightarrow Y$	Mehrwertige Abhängigkeit
$G_S = (V, E)$	Fremdschlüsselgraph eines Schemas S
$depth(G)$	Tiefe von G (längster Pfad)
$breadth(G)$	Breite von G (maximaler Verzweigungsgrad)
$\tau(G)$	Tiefen-Breiten-Verhältnis $depth(G)/breadth(G)$
$\delta(Q)$	Join-Tiefe einer Anfrage Q
ρ	Redundanzfaktor
$\sigma_P, \pi, \bowtie, \rho_{A \rightarrow B}$	Selektion, Projektion, Verbund, Umbenennung (relationale Algebra)

Tabelle A.2: Glossar der in diesem Buch verwendeten formalen Symbole

A.3 Hinweise zu Software-Bezugsquellen

In Anlehnung an die im Vorbild von Kleinschmidt und Rank [11] enthaltenen Hinweise auf Software-Bezugsquellen sei angemerkt, dass die in diesem Buch verwendeten SQL- und PL/SQL-Konstrukte sich an verbreiteten Open-Source- und kommerziellen relationalen Datenbanksystemen orientieren und mit geringfügigen syntaktischen Anpassungen auf diesen Systemen lauffähig sind. Für die in Kapitel 7 behandelte DBI-Schnittstelle sind entsprechende Treibermodule für die gängigen Open-Source-Datenbanksysteme verfügbar.

Literaturverzeichnis

- [1] Armstrong, W. W. (1974). Dependency structures of data base relationships. *Proceedings of IFIP Congress*, 74, 580–583. Grundlegende Arbeit zu den nach Armstrong benannten Inferenzregeln für funktionale Abhängigkeiten, die der Hüllenoperation X_{Σ}^+ (Definition 4.3) zugrunde liegen.
- [2] Ambler, S. W. (2003). *Agile Database Techniques: Effective Strategies for the Agile Software Developer*. Wiley. Praxisorientierte Darstellung von Schema-Refactoring-Techniken, relevant für den in Abschnitt 9.2.1 skizzierten Schema-Linter.
- [3] Boyce, R. F.; Codd, E. F. (1974). Relational completeness of data base sublanguages. In: Rustin, R. (Hrsg.): *Data Base Systems*, Courant Computer Science Symposia, Vol. 6, Prentice-Hall, S. 65–98. Ursprungsarbeit zur Boyce-Codd-Normalform (Definition 3.4).
- [4] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. Die Gründungsarbeit des relationalen Modells (Definition 2.1); Ausgangspunkt sämtlicher in diesem Buch behandelten Theorie.
- [5] Codd, E. F. (1974). Recent investigations in relational data base systems. *Proceedings of IFIP Congress*, 1021–1036. Formuliert 1NF, 2NF und 3NF (Abschnitt 8.13) sowie frühe Fassungen der in Abschnitt 2.5 diskutierten Zusatzforderungen.
- [6] Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems*, 2(3), 262–278. Einführung der vierten Normalform (Definition 3.5) und der mehrwertigen Abhängigkeiten, zentral für Proposition 8.3.
- [7] Fagin, R. (1981). A normal form for relational databases that is based on domains and keys. *ACM Transactions on Database Systems*, 6(3), 387–415. Einführung der Domain-Key-Normalform (DKNF), der „stärksten“ klassischen Normalform, als Bezugspunkt für Theorem 4.3.
- [8] Ullman, J. D. (1988). *Principles of Database and Knowledge-Base Systems*, Vol. 1. Computer Science Press. Standardquelle für das Lossless-Join-Lemma (Theorem 4.1) und den Synthesealgorithmus (Algorithmus 4.2).
- [9] Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley. Umfassendes einführendes Lehrbuch; Quelle der Anomalieklassifikation in Abschnitt 8.14.

- [10] Elmasri, R.; Navathe, S.B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson. Breit angelegtes Lehrbuch mit Kapiteln zu ER-Modellierung (Abschnitt 2.2), Normalisierung und verteilten Datenbanken (Abschnitt 9.2.8).
- [11] Kleinschmidt, P.; Rank, C. (2002). *Relationale Datenbanksysteme: Eine praktische Einführung* (2., überarbeitete und erweiterte Auflage). Springer-Verlag, Berlin Heidelberg. Strukturelles Vorbild für die Gliederung dieses Buches (vgl. Abschnitt 1.4).
- [12] Ramakrishnan, R.; Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill. Quelle für die in Proposition 8.6 verwendete Diskussion der Join-Tiefe und Anfrageoptimierung.
- [13] Graefe, G. (1993). Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2), 73–169. Übersichtsarbeit zu Hash-Joins und weiteren Verbundalgorithmen, zitiert in der Bemerkung zu Proposition 8.6.
- [14] Brewer, E. A. (2000). Towards robust distributed systems (Keynote). *Proceedings of ACM PODC*. Erste Formulierung des CAP-Theorems, zentral für Abschnitt 9.2.8.
- [15] Gilbert, S.; Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59. Formaler Beweis des CAP-Theorems, Grundlage der Diskussion des Konsistenzgradienten in Abschnitt 9.2.8.
- [16] Garcia-Molina, H.; Salem, K. (1987). Sagas. *Proceedings of ACM SIGMOD*, 249–259. Ursprüngliche Definition des Saga-Patterns, aufgegriffen in Abschnitt 9.2.8 im Kontext der Tiefenpfadtraversierung.
- [17] Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44. Praxisnahe Darstellung eventueller Konsistenz, Grundlage für den in Abschnitt 9.2.8 postulierten Konsistenzgradienten entlang des Tiefenpfads.
- [18] Corbett, J. C.; Dean, J.; Epstein, M.; Fikes, A.; Frost, C.; Furman, J. J.; Ghemawat, S.; Gubarev, A.; Heiser, C.; Hochschild, P.; et al. (2013). Spanner: Google’s globally distributed database. *ACM Transactions on Computer Systems*, 31(3), 1–22. Beispiel eines NewSQL-Systems mit globaler Verteilung, relevant für Abschnitt 9.2.2 und Abschnitt 9.2.8.
- [19] Newman, S. (2021). *Building Microservices* (2nd ed.). O’Reilly. Standardwerk zu Microservice-Architekturen einschließlich Database-per-Service-Mustern (Abschnitt 9.2.8).
- [20] Snodgrass, R. T. (1999). *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann. Standardreferenz für bitemporale Datenmodellierung, Grundlage von Abschnitt 9.2.7.
- [21] Jensen, C. S.; Clifford, J.; Elmasri, R.; Gadia, S. K.; Hayes, P.; Jajodia, S.; et al. (1994). A consensus glossary of temporal database concepts. *ACM SIGMOD Record*, 23(1), 52–64. Begriffliche Grundlage für Gültigkeitszeit und Transaktionszeit, verwendet in Abschnitt 9.2.7.

- [22] Kimball, R. (1996). *The Data Warehouse Toolkit*. Wiley. Quelle für Sternschemata (Definition 8.7) und Slowly Changing Dimensions, diskutiert in Abschnitt 9.2.7.
- [23] Kimball, R.; Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley. Aktualisierte Darstellung der Slowly-Changing-Dimension-Muster (Typ 1 bis Typ 7), aufgegriffen in Abschnitt 9.2.7.
- [24] Kulkarni, K.; Michels, J.-E. (2012). Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3), 34–43. Beschreibung der `PERIOD FOR`- und `SYSTEM_VERSIONING`-Klauseln aus Abschnitt 9.2.7.
- [25] Saltzer, J. H.; Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. Formuliert das Prinzip der minimalen Rechtevergabe, zentral für Abschnitt 5.3.
- [26] Popa, R. A.; Redfield, C. M. S.; Zeldovich, N.; Balakrishnan, H. (2011). CryptDB: Protecting confidentiality with encrypted query processing. *Proceedings of ACM SOSR*, 85–100. Grundlagenarbeit zu ordnungserhaltender und deterministischer Verschlüsselung von Datenbankspalten, zentral für Abschnitt 9.2.9.
- [27] Curino, C.; Jones, E.; Popa, R. A.; Malviya, N.; Wu, E.; Madden, S.; Balakrishnan, H.; Zeldovich, N. (2011). Relational Cloud: A database-as-a-service for the cloud. *Proceedings of CIDR*, 235–240. Architekturmuster für Mandantenfähigkeit (Multi-Tenancy), diskutiert in Abschnitt 9.2.9.
- [28] Voigt, P.; von dem Bussche, A. (2017). *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer. Juristische Grundlage des in Abschnitt 9.2.9 diskutierten „Rechts auf Löschung“ und der Pseudonymisierungsanforderungen.
- [29] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Referenz für Beweisstil und Laufzeitanalyse, vgl. Theorem 4.2.
- [30] Tarjan, R. E. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2), 146–160. Grundlagenarbeit zu DFS und dem DFS-Wald (Definition 8.16).
- [31] Bertot, Y.; Castéran, P. (2004). *Interactive Theorem Proving and Program Development: Coq’Art: The Calculus of Inductive Constructions*. Springer. Standardeinführung in den Beweisassistenten Coq, Grundlage der in Abschnitt 9.2.10 skizzierten Formalisierung.
- [32] Nipkow, T.; Paulson, L. C.; Wenzel, M. (2002). *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer (LNCS 2283). Alternative zu Coq für die in Abschnitt 9.2.10 diskutierte maschinengeprüfte Formalisierung relationaler Algebra.
- [33] Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley. Grundlage der in Abschnitt 9.2.10 vorgeschlagenen Modellprüfung von Transaktionsschedules.

- [34] Holzmann, G. J. (2003). *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley. Alternatives Modellprüfungswerkzeug, ergänzend zu TLA+ in Abschnitt 9.2.10.
- [35] Hamilton, W. L. (2020). *Graph Representation Learning*. Morgan & Claypool. Einführendes Lehrbuch zu Graph Neural Networks, Grundlage der Receptive-Field-Diskussion in Abschnitt 9.2.11.
- [36] Fey, M.; Hu, W.; Huang, K.; Lenssen, J. E.; Ranjan, R.; Robinson, J.; Leskovec, J.; et al. (2023). Relational deep learning: Graph representation learning on relational databases. *arXiv preprint arXiv:2312.04615*. Aktuelle Arbeit zu relationalem Deep Learning direkt auf Fremdschlüsselgraphen G_S , zentrale Referenz für Abschnitt 9.2.11.
- [37] Topping, J.; Di Giovanni, F.; Chamberlain, B. P.; Dong, X.; Bronstein, M. M. (2021). Understanding over-squashing and bottlenecks on graphs via curvature. *Proceedings of ICLR*. Formale Untersuchung des Over-Squashing-Phänomens, aufgegriffen in Abschnitt 9.2.11 im Vergleich von Tiefenpfad- und Sternschemata.
- [38] Angles, R.; Arenas, M.; Barceló, P.; Hogan, A.; Reutter, J.; Vrgoč, D. (2018). Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5), 1–40. Übersichtsarbeit zu Cypher, Gremlin und SPARQL, Grundlage für den Vergleich in Abschnitt 9.2.12.
- [39] Robinson, I.; Webber, J.; Eifrem, E. (2015). *Graph Databases* (2nd ed.). O'Reilly. Praxisnahe Einführung in Property-Graph-Modelle, relevant für Abschnitt 9.2.12.
- [40] Deutsch, A.; Francis, N.; Green, A.; Hare, K.; Li, B.; Libkin, L.; Lindaaker, T.; Marsault, V.; Martens, W.; Michels, J.; et al. (2022). Graph pattern matching in GQL and SQL/PGQ. *Proceedings of ACM SIGMOD*, 2246–2258. Beschreibung von SQL/PGQ (SQL:2023) und GQL, zentrale Referenz für die hybride Anfragesprachendiskussion in Abschnitt 9.2.12.
- [41] Kane, A. (2023). pgvector: Open-source vector similarity search for Postgres. *GitHub-Repository*, <https://github.com/pgvector/pgvector>. Erweiterung für Vektorindizes in PostgreSQL, diskutiert im Kontext semantischer Ähnlichkeitskanten in Abschnitt 9.2.11.
- [42] Roddick, J. F. (1995). A survey of schema versioning issues for database systems. *Information and Software Technology*, 37(7), 383–393. Übersicht zu Schemaevolution, Grundlage von Theorem 8.9.
- [43] Curino, C.; Moon, H. J.; Zaniolo, C. (2008). Graceful database schema evolution: The PRISM workbench. *Proceedings of VLDB*, 761–772. Werkzeugunterstützung für Schemaevolution, relevant für den in Abschnitt 9.2.1 skizzierten Schema-Linter.
- [44] Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley. Allgemeine Refactoring-Prinzipien, übertragen auf Datenbank-schemata in Abschnitt 9.2.1.

- [45] Melton, J.; Simon, A. R. (2003). *SQL:1999 – Understanding Relational Language Components*. Morgan Kaufmann. Detaillierte Darstellung des SQL:1999-Standards (Abschnitt 3.2), insbesondere rekursiver Anfragen und objektrelationaler Erweiterungen.
- [46] Gulutzan, P.; Pelzer, T. (1999). *SQL-99 Complete, Really*. CMP Books. Praxisorientierte Referenz zu SQL:1999-Konstrukten einschließlich CASE, CHECK und Trigger-Syntax (Kapitel 6).
- [47] Obe, R.; Hsu, L. S. (2017). *PostgreSQL: Up and Running* (3rd ed.). O’Reilly. Praktische Einführung in PostgreSQL-spezifische Erweiterungen wie DISTINCT ON (Abschnitt 3.5) und LATERAL-Joins.
- [48] The PostgreSQL Global Development Group (2024). *PostgreSQL 16 Documentation, Chapter 7: Queries*. <https://www.postgresql.org/docs/16/queries.html>. Offizielle Dokumentation der SELECT DISTINCT ON-Syntax (Definition 3.3) und der Bereichstypen für temporale Anfragen (Abschnitt 9.2.7).
- [49] Descartes, A.; Bunce, T. (2014). *Programming the Perl DBI*. O’Reilly. Standardreferenz zur DBI-Schnittstelle (Abschnitt 7.4).
- [50] ISO/IEC 9075-2:2016. *Information technology – Database languages – SQL – Part 2: Foundation (SQL/Foundation)*. International Organization for Standardization. Normativer Standard für Embedded SQL (Abschnitt 7.2) und Cursor-Semantik (Bemerkung 7.1).
- [51] Bauer, C.; King, G.; Gregory, G. (2015). *Java Persistence with Hibernate* (2nd ed.). Manning. Standardwerk zu objektrelationalem Mapping, Grundlage von Abschnitt 7.6 einschließlich Lazy Loading (Proposition 7.2) und des N+1-Problems (Bemerkung 7.3).
- [52] Sjøberg, D. I. K.; Hannay, J. E.; Hansen, O.; Kampenes, V. B.; Karahasanovic, A.; Liborg, N.-K.; Rekdal, A. C. (2005). A survey of controlled experiments in software engineering. *IEEE Transactions on Software Engineering*, 31(9), 733–753. Methodische Grundlage für die in Abschnitt 9.2.3 vorgeschlagene empirische Studie zu Schema-Topologie und Wartungsaufwand.
- [53] Kitchenham, B. A.; Pfleeger, S. L.; Pickard, L. M.; Jones, P. W.; Hoaglin, D. C.; El Emam, K.; Rosenberg, J. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, 28(8), 721–734. Leitlinien für die methodische Durchführung der in Abschnitt 9.2.3 skizzierten Studie.